Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

DevOps Engineering on AWS

TGS Ref No: TGS-2023040474

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 14

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
7	31 July 2026	Previous release — DevOps Engineering on AWS Learner Guide.	Dr Alfred Ang
8	23 August 2026	Rebuilt from a single content source: six connected labs aligned to the six official domains, with detailed step-by-step guides, reference architectures and a TSC (ICT-DES-4006-1.1) K/A/LO mapping.	Dr Alfred Ang
9	23 August 2026	Corrected release: each of the six labs expanded to 12–16 explicit steps with prerequisites, commands, expected outputs, validation, evidence, troubleshooting, cleanup, cost/safety and acceptance criteria; GitHub via AWS CodeConnections set as the primary source path (CodeCommit optional); INSTRUCTIONS.pdf mirrors regenerated (5+ pages/lab). Aligned to the concept-led v9 slide deck (210–230 substantive slides). Full K1–K7/A1–A6/LO1–LO6 mapping	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

		preserved.	
10	23 August 2026	Redesigned to exactly 25 self-contained hands-on labs — 5 labs per day across the 5 training days (Day 1 Foundations & SDLC Automation, Labs 1–5; Day 2 Configuration Management & IaC, Labs 6–10; Day 3 Resilient Cloud Solutions, Labs 11–15; Day 4 Monitoring, Logging & Incident Response, Labs 16–20; Day 5 Security, Compliance & Capstone, Labs 21–25). Each lab has 12–16 explicit steps, a unique reference architecture, editable AWS-native source files, a .env.example (placeholders only) and a 4–6 page INSTRUCTIONS.pdf. All 25 labs mapped across LO1–LO6, K1–K7, A1–A6 and the six official domains; Lab 25 is an integrated capstone technical blueprint and evidence pack. Aligned to the v10 concept-led slide deck.	Dr Alfred Ang
11	23 August 2026	Corrected assessment timing to match the approved Assessment Plan	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

		(v5.0): Written Assessment (SAQ) 1 hour + Practical Performance (PP) 2 hours = 3 assessment hours (the PP duration and the assessment total were previously understated). Course delivery re-stated as 40 total hours = 37 instructional + 3 assessment (Days 1–4: 8 instructional hours each; Day 5: 5 instructional hours plus the 3-hour assessment). All 25 labs (5 per day), content and reference architectures unchanged.	
12	23 August 2026	Final-QA corrective release. Body text set to Arial 11pt minimum throughout (step text, captions, expected-output and code listings no longer use sub-11pt overrides); the Table of Contents is a live Word TOC field with visible entries and page numbers. Daily timing aligned to the approved schedule (9:30am–6:00pm, 30-minute lunch; tea breaks within training time). Aligned to the v12 slide deck. All 25 labs (5 per	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

		day), detailed step-by-step procedures, content and reference architectures unchanged.	
13	23 August 2026	Aligned to the expanded 589-slide v13 trainer deck, which blends the current courseware with vetted legacy and related AWS DevOps certification coverage. Detailed click-by-click procedures remain in this Learner Guide and the individual lab folders rather than the PPT. The practical structure remains 25 self-contained labs — exactly five labs per day — with unchanged K1–K7, A1–A6 and LO1–LO6 mappings. No practice exam was added.	Dr Alfred Ang
14	23 August 2026	Expanded the visual trainer deck beyond 600 slides by adding 40 vetted AWS DevOps architecture-map analyses from the related certification course. Aligned to the 629-slide v14 deck; detailed procedures remain in this Learner Guide and the 25 individual lab folders. No practice exam was added.	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Introduction.....8
Course Learning Outcomes.....8
Skills Framework Mapping.....8
Before You Start — Environment Setup.....9
Domain 01 — SDLC Automation (22%).....10
Lab 1 — Set Up a Safe AWS & Git Working Environment and Verify Identity.....10
Lab 2 — Establish a Source-Control Workflow and Branching for the Sample App.....14
Lab 3 — Run Automated Tests on Every Commit with AWS CodeBuild.....18
Lab 4 — Produce and Store Versioned Build Artifacts with a Buildspec.....22
Lab 5 — Assemble a CodePipeline with Manual Approval and a Deployment Strategy….26
Domain 02 — Configuration Management and IaC (17%).....30
Lab 6 — Deploy a Reusable VPC Network Baseline with CloudFormation.....30
Lab 7 — Deploy a Parameterized Application Stack that Imports Network Exports.....35
Lab 8 — Preview Changes with Change Sets and Detect & Reconcile Drift.....39
Lab 9 — Externalize Configuration with SSM Parameter Store.....43
Lab 10 — Multi-Account, Multi-Region Guardrails with CloudFormation StackSets.....46
Domain 03 — Resilient Cloud Solutions (15%).....51
Lab 11 — Deploy a Multi-AZ Application Load Balancer with an Auto Scaling Group…….51
Lab 12 — Attach Target-Tracking Auto Scaling and Validate Scale-Out Under Load.....56
Lab 13 — Select a DR Strategy by RTO/RPO and Implement a Pilot-Light Skeleton.....60
Lab 14 — Automate Backup, Restore and Data Protection with AWS Backup.....65
Lab 15 — Inject Failure and Verify Self-Healing with AWS FIS.....69
Domain 04 — Monitoring and Logging (15%).....74
Lab 16 — Collect Custom Metrics with the CloudWatch Agent and Build an Operations Dashboard.....74
Lab 17 — Centralise Logs in CloudWatch Logs and Analyse Them with Logs Insights.....79
Lab 18 — Turn Logs into Signals: Metric Filters, CloudWatch Alarms and SNS Notifications.....83
Domain 05 — Incident and Event Response (14%).....87
Lab 19 — Route Incidents with Amazon EventBridge.....87
Lab 20 — Automated Remediation with Lambda and Systems Manager.....91
Domain 06 — Security and Compliance (17%).....96
Lab 21 — Enforce IAM Least Privilege and Analyse Policies for Over-Permissive Access.....96
Lab 22 — Protect Data with Secrets Manager and Customer-Managed KMS Encryption.....101
Lab 23 — Produce CloudTrail Audit Evidence and Answer Who-Did-What-When.....105
Lab 24 — Run Continuous Compliance with AWS Config and Security Hub.....109
Lab 25 — Capstone — Blueprint, Well-Architected Review and Evidence Pack for the Whole Delivery System.....114
Exam & Practice Focus — Cross-Domain Topics.....119
Assessment Preparation.....119
Glossary.....120

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the WSQ course DevOps Engineering on AWS (TGS-2023040474), conducted by Tertiary Infotech Academy Pte Ltd. It provides detailed step-by-step instructions for all 25 connected hands-on labs — 5 labs on each of the 5 training days — organised by the six official DevOps-on-AWS domains: SDLC Automation; Configuration Management and Infrastructure as Code; Resilient Cloud Solutions; Monitoring and Logging; Incident and Event Response; and Security and Compliance. Every lab maps to a learning outcome and the Solution Architecture skills framework (TSC ICT-DES-4006-1.1).

The 25 labs build on one another around a single sample application: over Day 1 you stand up a safe environment and a full CI/CD pipeline; on Day 2 you codify the infrastructure; on Day 3 you make it highly available, recoverable and self-healing; on Day 4 you observe it and respond to incidents automatically; and on Day 5 you secure and audit it and assemble an integrated capstone blueprint and evidence pack. Each lab is self-contained in labs/lab-NN-*/ with a README, an INSTRUCTIONS.pdf mirror, editable AWS-native source files, a .env.example (placeholders only) and a unique reference architecture. Never commit live AWS credentials — use IAM roles, the AWS CLI profile and the .env.example placeholders provided in each lab.

Course Learning Outcomes

- LO1: Develop an AWS CI/CD pipeline proof of concept.
- LO2: Identify the technical and practical requirements in an AWS cloud infrastructure setup.
- LO3: Develop a resilient cloud solution based on AWS cloud components.
- LO4: Implement regular AWS monitoring and perform necessary troubleshooting.
- LO5: Demonstrate solutions to incidents and business problems on the AWS cloud.
- LO6: Prepare a technical blueprint to address security and compliance.

Skills Framework Mapping

This course is aligned to the Solution Architecture technical skill and competency (ICT-DES-4006-1.1). Each domain maps to one learning outcome and the underpinning knowledge (K) and ability (A) statements below.

- K1 — Process for developing proof of concepts
- K2 — Components of solution architecture
- K3 — Objectives of solution architecture
- K4 — Steps for developing solution architecture
- K5 — Tools and techniques for solution architecture modelling
- K6 — Technical blueprint design and construction process
- K7 — Interactions among various IT components

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- A1 — Develop an architectural proof of concept
- A2 — Develop a solution architecture utilising appropriate tools, techniques and models of system components and interfaces
- A3 — Identify technical and practical requirements as well as stakeholders' demands
- A4 — Prepare a technical blueprint for a solution in a given area
- A5 — Demonstrate how the recommended IT solutions and components collectively address an existing business problem or need
- A6 — Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework
- **Domain 01 — SDLC Automation —** LO1 · Knowledge K1 · Ability A1
- **Domain 02 — Configuration Management and IaC —** LO2 · Knowledge K3 · Ability A3
- **Domain 03 — Resilient Cloud Solutions —** LO3 · Knowledge K2, K4 · Ability A2
- **Domain 04 — Monitoring and Logging —** LO4 · Knowledge K5 · Ability A6
- **Domain 05 — Incident and Event Response —** LO5 · Knowledge K7 · Ability A5
- **Domain 06 — Security and Compliance —** LO6 · Knowledge K6 · Ability A4

Before You Start — Environment Setup

What you need

- An AWS account with permissions to use the services in each lab (an AWS Free Tier / sandbox account is sufficient for most labs).
- The AWS CLI v2 installed and configured with a named profile (aws configure --profile devops) — never hard-code keys in files.
- Git, and a code editor. A modern browser for the AWS Management Console.
- The lab source files from the course repository (labs/lab-01-* … labs/lab-06-*), each with editable YAML/JSON/scripts and a .env.example.

Configure the AWS CLI safely

Authenticate with a least-privilege IAM user or (preferred) IAM Identity Center / an assumed role. Copy each lab's .env.example to .env and fill in your own account ID, Region and resource names — the .env file is git-ignored and must never contain real secrets committed to source control.

```
aws configure --profile devops        # set Access Key, Secret,
region (e.g. ap-southeast-1)

aws sts get-caller-identity --profile devops   # confirm you are
authenticated

cp .env.example .env                   # then edit .env with YOUR
values (never commit real secrets)
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Conventions used in every lab

- Commands are run from the lab folder; the AWS CLI uses your 'devops' profile (append --profile devops or set AWS_PROFILE).
- Placeholders such as <alb-dns-name>, <arn>, <id> and you@example.com are replaced with your own values.
- CloudFormation stacks are named devops-* so they are easy to find and delete.
- Always run the clean-up (delete-stack) step at the end of a lab to avoid ongoing charges.
- Prefer IAM roles over long-lived access keys; store secrets in AWS Secrets Manager, not in files.

Domain 01 — SDLC Automation (22%)

CI/CD pipelines · automated testing · artifacts · deployment strategies

Key concepts

- Continuous Integration — Developers push often; CodeBuild runs the build and tests on every commit so bugs are found early, not at release.
- Continuous Delivery / Deployment — Every green build is releasable; CodePipeline and CodeDeploy move it through dev → test → prod with approvals.
- The AWS developer tools — CodeCommit / CodeBuild / CodeDeploy / CodePipeline / CodeArtifact / CodeGuru compose the managed CI/CD toolchain.
- buildspec & appspec — buildspec.yml tells CodeBuild what to build and test; appspec.yml tells CodeDeploy what and how to deploy.
- Deployment strategies — In-place, rolling, blue/green and canary trade release speed against blast radius and rollback safety.
- Artifacts — Build outputs are versioned in S3 / CodeArtifact so any release is reproducible and traceable.

Lab 1 — Set Up a Safe AWS & Git Working Environment and Verify Identity

Learning outcome: LO1. Skills mapped: K1, A1. Objective: Stand up a least-privilege AWS CLI/Git workspace and confirm your identity.

Goal: Before automating anything you need a workspace you can trust. In this lab you install and configure AWS CLI v2 with a named least-privilege 'devops' profile (or an IAM Identity Center session), prove exactly who you are with STS, pin a Region, attach a permission boundary so the profile can never exceed its remit, and lock down your Git repository so plaintext secrets can never be committed. CloudShell is your always-available fallback.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

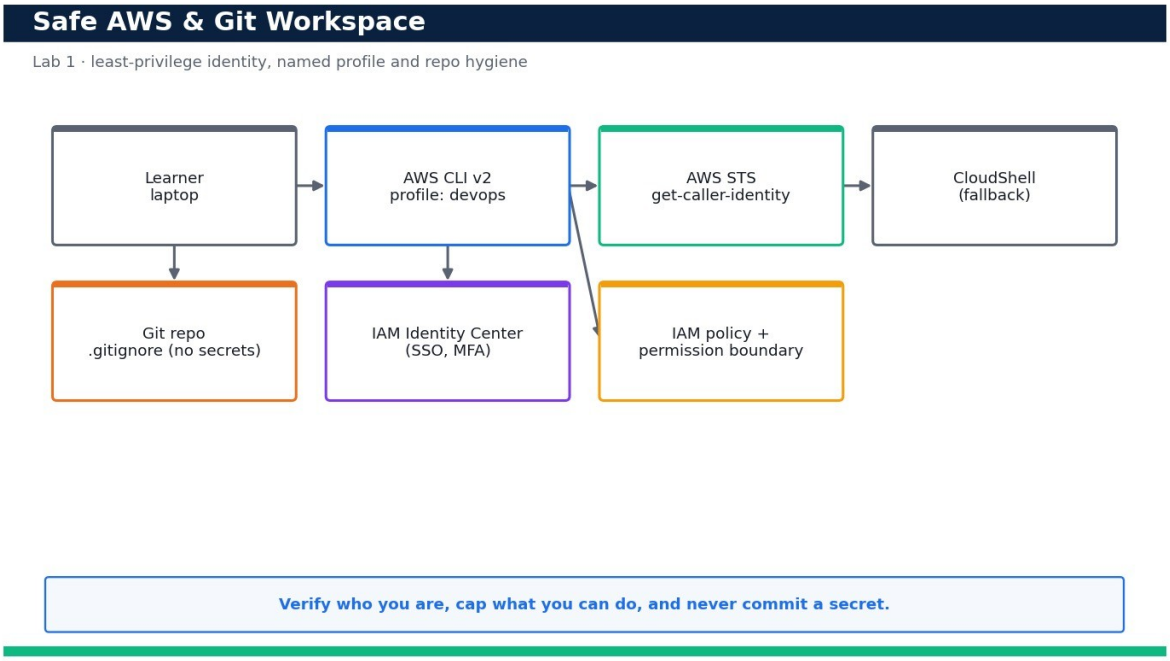


Figure — Lab 1 reference architecture: Set Up a Safe AWS & Git Working Environment and Verify Identity.

What you'll build

A named, MFA-aware, least-privilege 'devops' CLI profile plus a Git repo with a secrets-safe .gitignore, verified with an AWS caller-identity check and a clean secret scan. (Services: AWS IAM, AWS CLI v2, IAM Identity Center, CloudShell, Git.)

Step-by-step

1. Install AWS CLI v2 (macOS/Linux/Windows) and confirm the major version — v1 is not supported for this course.

```
aws --version
```

Expected: Prints something like aws-cli/2.17.x Python/3.12 — a 2.x version string.

2. Configure a NAMED least-privilege profile called 'devops' (never use root, never the default profile for real work).

```
aws configure --profile devops
```

Expected: You are prompted for Access Key, Secret Key, Default region (ap-southeast-1) and output (json); a ~/.aws/config and ~/.aws/credentials entry named [profile devops] is written.

3. Alternatively, configure IAM Identity Center (SSO) sign-in for the same profile so short-lived, MFA-backed credentials are used instead of static keys.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws configure sso --profile devops
```

Expected: The browser opens for the SSO sign-in + MFA; on success a short-lived session is cached and no long-lived secret key is stored on disk.

4. Prove exactly who you are — this is the single most important check before you touch anything.

```
aws sts get-caller-identity --profile devops
```

Expected: Returns your own UserId, Account (your <your-account-id>) and Arn — confirming the profile maps to the identity you expect.

5. Pin the Region explicitly so nothing lands in the wrong Region by accident.

```
aws configure get region --profile devops
```

Expected: Prints ap-southeast-1 (or your chosen Region) — matching what you set in step 2.

6. Create a least-privilege developer policy from the provided JSON (read-mostly plus the specific write actions this course needs).

```
aws iam create-policy --policy-name least-privilege-dev --policy-document file://policies/least-privilege-dev.json --profile devops
```

Expected: Returns the new policy Arn arn:aws:iam::<your-account-id>:policy/least-privilege-dev.

7. Create a PERMISSION BOUNDARY policy that caps the maximum permissions the devops identity can ever have, even if more policies are attached later.

```
aws iam create-policy --policy-name devops-boundary --policy-document file://policies/permission-boundary.json --profile devops
```

Expected: Returns the boundary policy Arn arn:aws:iam::<your-account-id>:policy/devops-boundary.

8. Confirm the boundary denies privilege escalation by simulating a disallowed IAM action against it.

```
aws iam simulate-custom-policy --policy-input-list file://policies/permission-boundary.json --action-names iam:CreateUser --profile devops
```

Expected: The EvalDecision for iam:CreateUser is implicitDeny — the boundary does not grant admin/IAM-user creation.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

9. Understand the MFA posture: list any virtual MFA devices in the account so you can confirm MFA is (or should be) enforced.

```
aws iam list-virtual-mfa-devices --profile devops
```

Expected: Lists MFA devices; a device bound to your user (SerialNumber ending in your username) confirms MFA is enabled for interactive sign-in.

10. Initialise or clone your working Git repo and set your commit identity.

```
git init devops-sample-app && cd devops-sample-app && git config user.email you@example.com && git config user.name 'Your Name'
```

Expected: The repo is initialised and user.name/user.email are set for this repo.

11. Add the hardened .gitignore so credentials, .env files and AWS state can never be staged.

```
cp ../.gitignore . && git add .gitignore && git status
```

Expected: .env, *.pem, .aws/ and credentials patterns are ignored; git status shows only .gitignore staged, never a secret file.

12. Run the identity-check helper script to bundle the STS check, Region check and secret scan into one verifiable command.

```
bash identity-check.sh
```

Expected: Prints the caller ARN, the active Region and 'PROFILE OK' — a single green signal that the workspace is safe to use.

13. Scan the working tree for accidental plaintext keys BEFORE your first commit (AKIA-style access-key IDs).

```
git grep -nE 'AKIA[0-9A-Z]{16}' || echo 'no access keys found'
```

Expected: Prints 'no access keys found' — confirming no AWS access-key IDs are present in tracked files.

14. As a resilience check, open AWS CloudShell in the console and re-run the identity check there (works with no local install).

```
aws sts get-caller-identity
```

Expected: CloudShell already has credentials; it returns the SAME Account/Arn, proving you have a browser-based fallback that needs no local secrets.

Validation

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

`aws sts get-caller-identity --profile devops` returns your own account and ARN; the region resolves to your chosen Region; the least-privilege-dev policy and devops-boundary policy exist in IAM; the policy simulation shows IAM-user creation is implicitly denied by the boundary; and `git grep` finds no access-key patterns in tracked files.

Test it

You can run identity-check.sh and get 'PROFILE OK' with the correct ARN and Region, the permission boundary provably blocks privilege escalation, .env and key files are git-ignored, and the same identity check succeeds from CloudShell — proving a safe, verifiable working environment.

Note: Cost & safety: Everything in this lab is free: IAM users/roles/policies, STS calls, IAM Identity Center and CloudShell incur no charge. There is nothing billable to tear down beyond removing the practice IAM entities you created. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-01-safe-env-identity/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 2 — Establish a Source-Control Workflow and Branching for the Sample App

Learning outcome: LO1. Skills mapped: K1, A1. Objective: Adopt a trunk-based Git workflow and connect the repository to AWS with CodeConnections.

Goal: Good delivery starts with disciplined source control. In this lab you fork and clone the sample web app, adopt a trunk-based workflow, cut a short-lived feature branch, commit a change, push it, open a pull request and protect the main branch. You then create an AWS CodeConnections connection to GitHub — the modern, secure source path — and confirm it reports AVAILABLE so later labs can trigger builds and pipelines straight from your repo.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

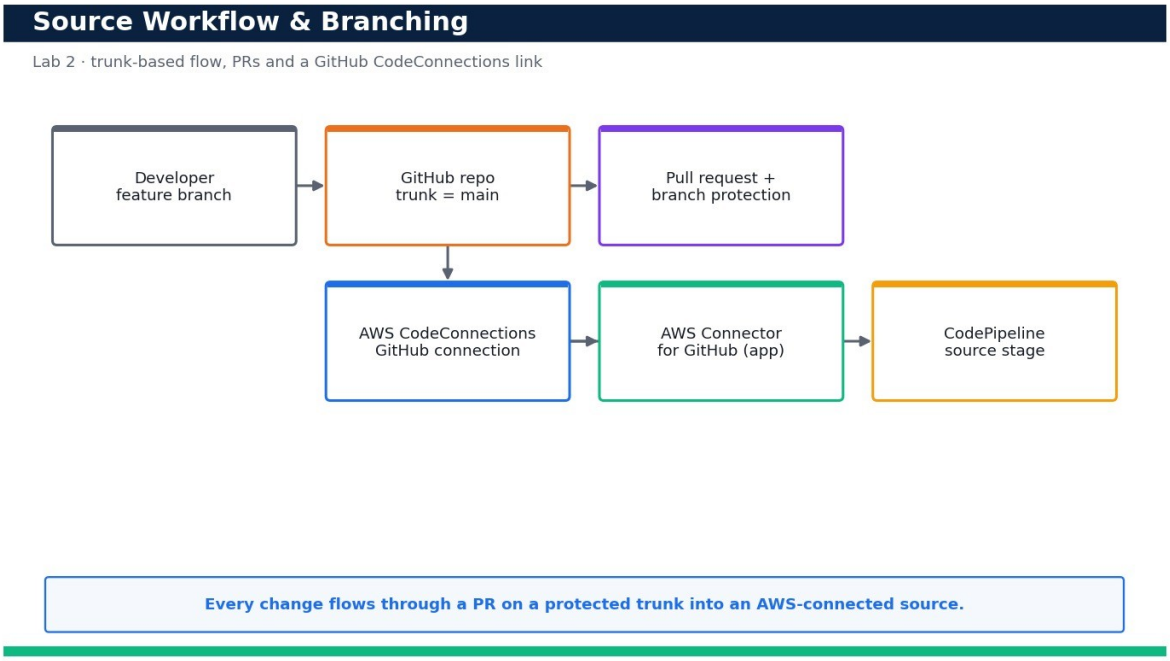


Figure — Lab 2 reference architecture: Establish a Source-Control Workflow and Branching for the Sample App.

What you'll build

A protected trunk-based GitHub repo with an open/merged PR, plus an AVAILABLE AWS CodeConnections connection to GitHub ready to drive CodeBuild and CodePipeline. (Services: GitHub, AWS CodeConnections, Git, AWS CodePipeline (source stage).)

Step-by-step

15. Fork the sample web app into your own GitHub account (via the GitHub UI), then clone YOUR fork locally.

```
git clone https://github.com/<your-gh-user>/devops-sample-app.git && cd devops-sample-app
```

Expected: The repo clones; `git remote -v` shows origin pointing at your fork and app/index.html + app/version.py are present.

16. Confirm main is your trunk and it is up to date before you branch (trunk-based development).

```
git checkout main && git pull --ff-only origin main
```

Expected: You are on branch main and it is already up to date with origin/main.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

17. Create a short-lived feature branch named for the change (never commit directly to main).

```
git checkout -b feature/bump-version
```

Expected: Switched to a new branch 'feature/bump-version'.

18. Make a small, reviewable change — bump the app version — and view the diff before committing.

```
sed -i '' 's/1.0.0/1.1.0/' app/version.py && git diff
app/version.py
```

Expected: The diff shows VERSION changed from "1.0.0" to "1.1.0".

19. Commit the change with a clear, conventional message.

```
git commit -am 'feat: bump app version to 1.1.0'
```

Expected: One file changed; a commit is created on feature/bump-version.

20. Push the feature branch to your fork to make it reviewable.

```
git push -u origin feature/bump-version
```

Expected: The branch is pushed and GitHub prints a URL to open a pull request.

21. Confirm the branch exists on the remote so a pull request can be opened against it.

```
git ls-remote --heads origin feature/bump-version
```

Expected: The remote branch ref is listed, confirming the branch exists on GitHub and is ready for a PR/review.

22. Enable branch protection on main in GitHub (require a PR and at least one approval before merge) so nobody can push straight to trunk.

Expected: In GitHub > Settings > Branches, a protection rule for main requires a pull request and 1 approving review; direct pushes to main are now blocked.

23. Create an AWS CodeConnections connection to GitHub — the primary, modern source path (CodeCommit is an AWS-native alternative, not required).

```
aws codeconnections create-connection --provider-type GitHub --
connection-name devops-github --profile devops
```

Expected: Returns a ConnectionArn with ConnectionStatus PENDING.

24. Complete the handshake by authorising the AWS Connector for GitHub app in the browser (Console > Developer Tools > Connections > Update pending connection).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: After you authorise and install the AWS Connector on your fork, the connection status changes from PENDING to AVAILABLE.

25. Confirm the connection is AVAILABLE before any later lab wires it into a build or pipeline.

```
aws codeconnections get-connection --connection-arn <connection-arn> --query 'Connection.ConnectionStatus' --profile devops
```

Expected: The command prints "AVAILABLE".

26. List your connections so you can copy the ARN into .env for reuse in Labs 3–5.

```
aws codeconnections list-connections --query 'Connections[?ConnectionName==`devops-github`]' --profile devops
```

Expected: The devops-github connection is listed with its ARN and ConnectionStatus AVAILABLE.

27. Merge the PR in GitHub after approval, then update your local trunk to prove the loop is closed.

```
git checkout main && git pull --ff-only origin main && grep VERSION app/version.py
```

Expected: main now contains VERSION = "1.1.0" — the reviewed change landed on the protected trunk.

28. Commit the branching guide so the team follows the same rules.

```
git add docs/branching-workflow.md && git commit -m 'docs: branching workflow' && git push
```

Expected: docs/branching-workflow.md is committed and the workflow is captured in the repo.

Validation

Your fork shows a feature branch and a merged PR; main has branch protection requiring a PR + review; `aws codeconnections get-connection` returns AVAILABLE; and the connection ARN is stored in .env for reuse. Direct pushes to main are rejected by the protection rule.

Test it

You can push a feature branch, open and merge a PR into a protected main, and prove an AWS CodeConnections connection to GitHub is AVAILABLE — the source foundation every later pipeline depends on.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Cost & safety: AWS CodeConnections has no per-connection charge and GitHub is free for public/small repos. There is nothing metered here — the only teardown is deleting the connection and the practice branch. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-02-source-workflow-branching/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 3 — Run Automated Tests on Every Commit with AWS CodeBuild

Learning outcome: LO1. Skills mapped: K1, A1. Objective: Author a buildspec and a CodeBuild project that automatically lints and tests the app.

Goal: Automated tests are the gate that keeps broken code out of production. In this lab you author a buildspec.yml with install/pre_build/build phases that install dependencies, run a linter and run pytest, then create a CodeBuild project that pulls the source through your CodeConnections link. You start a build, read the phase-by-phase logs in CloudWatch, deliberately break a test to watch the build fail, then fix it and confirm the build goes green again.

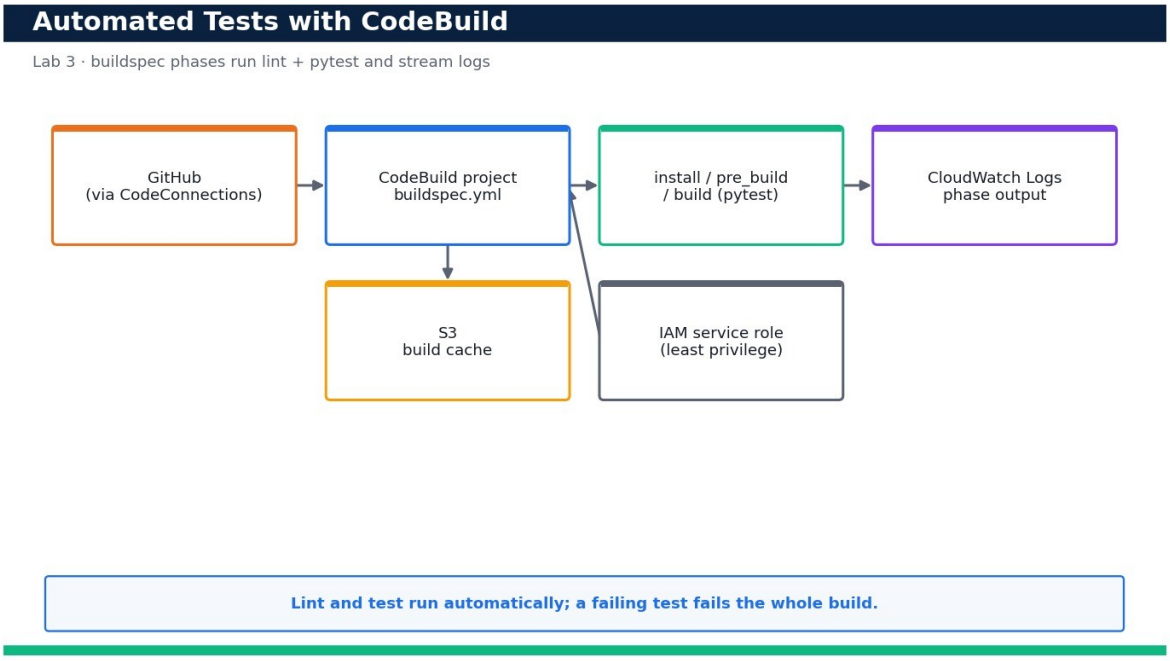


Figure — Lab 3 reference architecture: Run Automated Tests on Every Commit with AWS CodeBuild.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A CodeBuild project that runs the linter and pytest on demand, streams phase logs to CloudWatch, and fails loudly when a test is broken. (Services: AWS CodeBuild, Amazon S3, AWS IAM, Amazon CloudWatch Logs.)

Step-by-step

29. Review buildspec.yml and confirm it has install, pre_build and build phases that install deps, lint, then run pytest.

```
cat buildspec.yml
```

Expected: You can see runtime-versions python 3.12, `pip install -r requirements.txt`, a pyflakes lint step, and `pytest -q` in the build phase.

30. Run the tests locally first to establish the green baseline before wiring CodeBuild.

```
pip install -r requirements.txt && python -m pytest -q
```

Expected: pytest reports '2 passed' (test_greeting and test_version).

31. Create a least-privilege IAM service role for CodeBuild with a CodeBuild trust policy, then note its ARN.

```
aws iam create-role --role-name devops-codebuild-role --assume-
role-policy-document '{"Version":"2012-10-17","Statement":
[{"Effect":"Allow","Principal":
{"Service":"codebuild.amazonaws.com"},"Action":"sts:AssumeRole"}]
}' --profile devops
```

Expected: Returns the role Arn arn:aws:iam::<your-account-id>:role/devops-codebuild-role.

32. Attach the inline least-privilege policy (CloudWatch Logs + read the CodeConnections source) shipped in policies/codebuild-logs.json.

```
aws iam put-role-policy --role-name devops-codebuild-role --
policy-name codebuild-logs --policy-document
file://policies/codebuild-logs.json --profile devops
```

Expected: The command returns with no error; the role can now write logs and use the connection.

33. Create the CodeBuild project sourced from GitHub through your CodeConnections connection (substitute your connection ARN and account ID).

```
aws codebuild create-project --name devops-app-tests --source
type=GITHUB,location=https://github.com/<your-gh-user>/devops-
sample-
app.git,buildspec=buildspec.yml,auth='{type=CODECONNECTIONS,resou
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
rce=<connection-arn>}' --artifacts type=NO_ARTIFACTS --
environment
type=LINUX_CONTAINER,image=aws/codebuild/standard:7.0,computeType
=BUILD_GENERAL1_SMALL --service-role arn:aws:iam::<your-account-
id>:role/devops-codebuild-role --profile devops
```

Expected: Returns the project with name devops-app-tests, image aws/codebuild/standard:7.0 and source type GITHUB via the connection.

34. Confirm the project points at buildspec.yml and the correct managed image.

```
aws codebuild batch-get-projects --names devops-app-tests --query
'projects[0].{image:environment.image,spec:source.buildspec}' --
profile devops
```

Expected: Shows image aws/codebuild/standard:7.0 and buildspec buildspec.yml.

35. Start a build and capture the build ID for log tailing.

```
aws codebuild start-build --project-name devops-app-tests --query
'build.id' --output text --profile devops
```

Expected: Returns a build id like devops-app-tests:<uuid>; the build enters IN_PROGRESS.

36. Watch the build reach SUCCEEDED and confirm each phase passed.

```
aws codebuild batch-get-builds --ids <build-id> --query
'builds[0].{status:buildStatus,phases:phases[].
{phase:phaseType,status:phaseStatus}}' --profile devops
```

Expected: buildStatus is SUCCEEDED and INSTALL, PRE_BUILD and BUILD all show SUCCEEDED.

37. Read the phase logs in CloudWatch to see the actual pytest output.

```
aws logs tail /aws/codebuild/devops-app-tests --since 10m --
profile devops
```

Expected: The log shows the linter output and 'pytest' reporting '2 passed'.

38. Deliberately break a test to prove the gate works, commit and push it.

```
sed -i '' 's/Hello from the DevOps sample app/WRONG/'
tests/test_app.py && git commit -am 'test: break greeting
assertion' && git push
```

Expected: The broken assertion is committed and pushed to your branch.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

39. Start another build on the broken commit and confirm it FAILS at the build (test) phase.

```
aws codebuild start-build --project-name devops-app-tests --query 'build.buildStatus' --profile devops
```

Expected: The build starts, then batch-get-builds shows buildStatus FAILED with the BUILD phase FAILED — the gate blocked the bad code.

40. Inspect the failure log to see the exact failing assertion (this is what a reviewer would read).

```
aws logs tail /aws/codebuild/devops-app-tests --since 5m --profile devops | grep -i -A2 assert
```

Expected: The log shows the AssertionError for test_greeting and '1 failed'.

41. Fix the test, commit, push and run a final build to return to green.

```
git checkout tests/test_app.py && git commit -am 'test: restore greeting assertion' && git push && aws codebuild start-build --project-name devops-app-tests --profile devops
```

Expected: The new build reaches SUCCEEDED with all phases green again.

42. Confirm the build history so you can show the fail-then-pass story as evidence.

```
aws codebuild list-builds-for-project --project-name devops-app-tests --query 'ids[0:3]' --profile devops
```

Expected: Returns the recent build IDs; batch-get-builds on them shows the SUCCEEDED -> FAILED -> SUCCEEDED sequence.

Validation

The CodeBuild project builds from GitHub via CodeConnections; a clean commit yields buildStatus SUCCEEDED with INSTALL/PRE_BUILD/BUILD all green and pytest showing '2 passed'; a broken test yields buildStatus FAILED at the BUILD phase; and the CloudWatch logs show the exact assertion failure and the subsequent pass.

Test it

You can start a build that runs the linter and pytest automatically, read the phase logs, break a test to see the build FAIL, and fix it to see the build SUCCEED — proving automated tests gate the code.

Note: Cost & safety: CodeBuild bills per build-minute on a small on-demand compute (BUILD_GENERAL1_SMALL); a handful of sub-two-minute builds costs cents. The only

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

stored artifacts are tiny log streams and an optional S3 cache. Delete the project when done. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-03-codebuild-automated-tests/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 4 — Produce and Store Versioned Build Artifacts with a Buildspec

Learning outcome: LO1. Skills mapped: K1, A1. Objective: Extend the buildspec to package versioned artifacts and store them in an encrypted, versioned bucket.

Goal: A pipeline is only trustworthy if you can identify and reproduce exactly what it built. In this lab you extend the buildspec to package the app into a versioned artifact named by commit and semantic version, then upload it to an encrypted S3 artifact bucket with Block Public Access and object versioning enabled. You verify each object version, and prove you can retrieve and reproduce a prior build — the foundation for rollback in Lab 5.

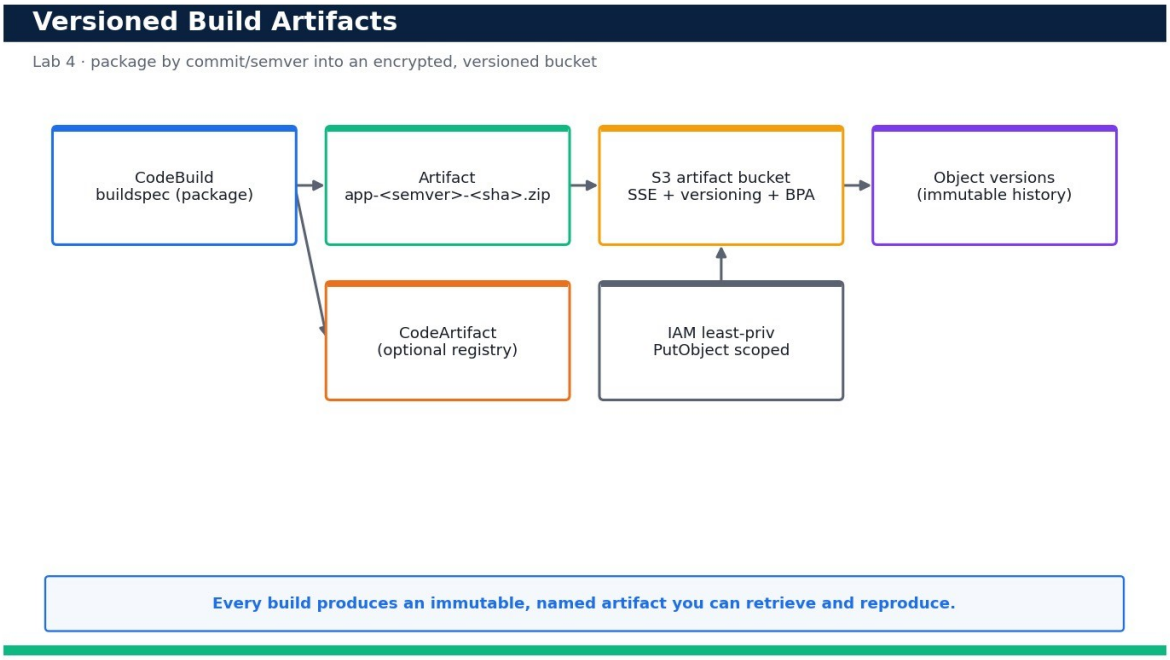


Figure — Lab 4 reference architecture: Produce and Store Versioned Build Artifacts with a Buildspec.

What you'll build

An encrypted, version-enabled S3 artifact bucket holding immutable, commit/semver-named build artifacts that you can list, retrieve and reproduce. (Services: AWS CodeBuild, Amazon S3, AWS CodeArtifact, AWS IAM.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

43. Review the extended buildspec.yml and note the packaging step that names the artifact by semver and commit SHA.

```
cat buildspec.yml
```

Expected: The build phase computes SEMVER from app/version.py and SHA from CODEBUILD_RESOLVED_SOURCE_VERSION, then zips the app to app-<semver>-<sha>.zip.

44. Deploy the encrypted, versioned, public-access-blocked artifact bucket from the provided template.

```
aws cloudformation deploy --template-file artifact-bucket.yaml --stack-name devops-artifacts --profile devops
```

Expected: Stack devops-artifacts reaches CREATE_COMPLETE and outputs the ArtifactBucket name devops-cicd-artifacts-<your-account-id>.

45. Confirm default server-side encryption is enabled on the bucket.

```
aws s3api get-bucket-encryption --bucket devops-cicd-artifacts-<your-account-id> --profile devops
```

Expected: Returns a rule with SSEAlgorithm AES256 (or aws:kms) — objects are encrypted at rest.

46. Confirm Block Public Access is fully on so artifacts can never be exposed.

```
aws s3api get-public-access-block --bucket devops-cicd-artifacts-<your-account-id> --profile devops
```

Expected: All four flags (BlockPublicAcls, BlockPublicPolicy, IgnorePublicAcls, RestrictPublicBuckets) are true.

47. Confirm object versioning is enabled so each upload is retained as an immutable version.

```
aws s3api get-bucket-versioning --bucket devops-cicd-artifacts-<your-account-id> --profile devops
```

Expected: Returns Status: Enabled.

48. Package the first artifact locally using version.sh (mirrors what the buildspec does) to see the naming.

```
bash version.sh
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: Prints the computed name, e.g. devops-sample-app-1.1.0-a1b2c3d.zip, and creates that zip in ./dist.

49. Upload the first versioned artifact to the bucket under a stable key.

```
aws s3 cp dist/devops-sample-app-1.1.0-a1b2c3d.zip s3://devops-cicd-artifacts-<your-account-id>/artifacts/app.zip --profile devops
```

Expected: The object is uploaded and S3 returns a VersionId for artifacts/app.zip.

50. Bump the app version, repackage and upload again to the SAME key to create a second object version.

```
sed -i '' 's/1.1.0/1.2.0/' app/version.py && bash version.sh && aws s3 cp dist/devops-sample-app-1.2.0-*.zip s3://devops-cicd-artifacts-<your-account-id>/artifacts/app.zip --profile devops
```

Expected: A second upload succeeds and returns a new (different) VersionId for the same key.

51. List all object versions to prove the immutable history exists.

```
aws s3api list-object-versions --bucket devops-cicd-artifacts-<your-account-id> --prefix artifacts/app.zip --query 'Versions[].{VersionId:VersionId,Latest:IsLatest,Size:Size}' --profile devops
```

Expected: Two versions are listed; exactly one has IsLatest true (the 1.2.0 build).

52. Also start a real CodeBuild run so the pipeline-produced artifact is stored the same way (belt and braces).

```
aws codebuild start-build --project-name devops-app-tests --artifacts-override type=S3,location=devops-cicd-artifacts-<your-account-id>,path=artifacts,name=app.zip,packaging=ZIP --profile devops
```

Expected: The build runs and uploads its packaged artifact to the artifact bucket, producing another object version.

53. Reproduce a PRIOR version: download the earlier 1.1.0 object version by its VersionId.

```
aws s3api get-object --bucket devops-cicd-artifacts-<your-account-id> --key artifacts/app.zip --version-id <old-version-id> restored-app.zip --profile devops
```

Expected: The exact earlier artifact is downloaded to restored-app.zip.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

54. Verify the restored artifact really is the older version by inspecting its contents.

```
unzip -p restored-app.zip app/version.py | grep VERSION
```

Expected: Prints VERSION = "1.1.0" — proving you reproduced the prior build byte-for-byte.

55. (Optional AWS-native registry) Create a CodeArtifact domain + repository if you want a managed package store as well.

```
aws codeartifact create-domain --domain devops --profile devops
&& aws codeartifact create-repository --domain devops --
repository app-artifacts --profile devops
```

Expected: The domain and repository are created; CodeArtifact can host packages alongside the S3 artifact bucket.

56. Confirm nothing is publicly reachable by attempting an unauthenticated read (should be denied).

```
curl -s -o /dev/null -w '%{http_code}\n' https://devops-cicd-
artifacts-<your-account-id>.s3.amazonaws.com/artifacts/app.zip
```

Expected: Returns 403 (AccessDenied) — Block Public Access and no public policy mean the artifact is private.

Validation

The artifact bucket reports SSE enabled, all four Block Public Access flags true, and versioning Enabled; two named artifacts exist as distinct object versions under artifacts/app.zip; you can download a specific prior VersionId and confirm it contains the older VERSION; and an unauthenticated GET returns 403.

Test it

You can produce a commit/semver-named artifact, store it in an encrypted, versioned, private bucket, list its version history, and download and reproduce a prior version — proving artifacts are identifiable, immutable and recoverable.

Note: Cost & safety: An S3 artifact bucket with versioning stores small zip files (cents/month) and CodeBuild bills per build-minute. Enabling versioning keeps old objects until you delete them — empty the bucket in cleanup so it stops accruing storage. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-04-versioned-artifacts-buildspec/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 5 — Assemble a CodePipeline with Manual Approval and a Deployment Strategy

Learning outcome: LO1. Skills mapped: K1, A1. Objective: Wire an end-to-end pipeline with a manual approval gate, a deployment strategy and auto-rollback.

Goal: Now you connect everything into one governed release path. Using a CloudFormation template plus a pipeline definition, you build Source (CodeConnections) -> Build (CodeBuild) -> Manual Approval -> Deploy (CodeDeploy). You choose between in-place and blue/green deployment, run the pipeline end to end, approve the release, verify the deployed app with a health check, and finally wire an alarm-based automatic rollback so a bad release self-reverts.

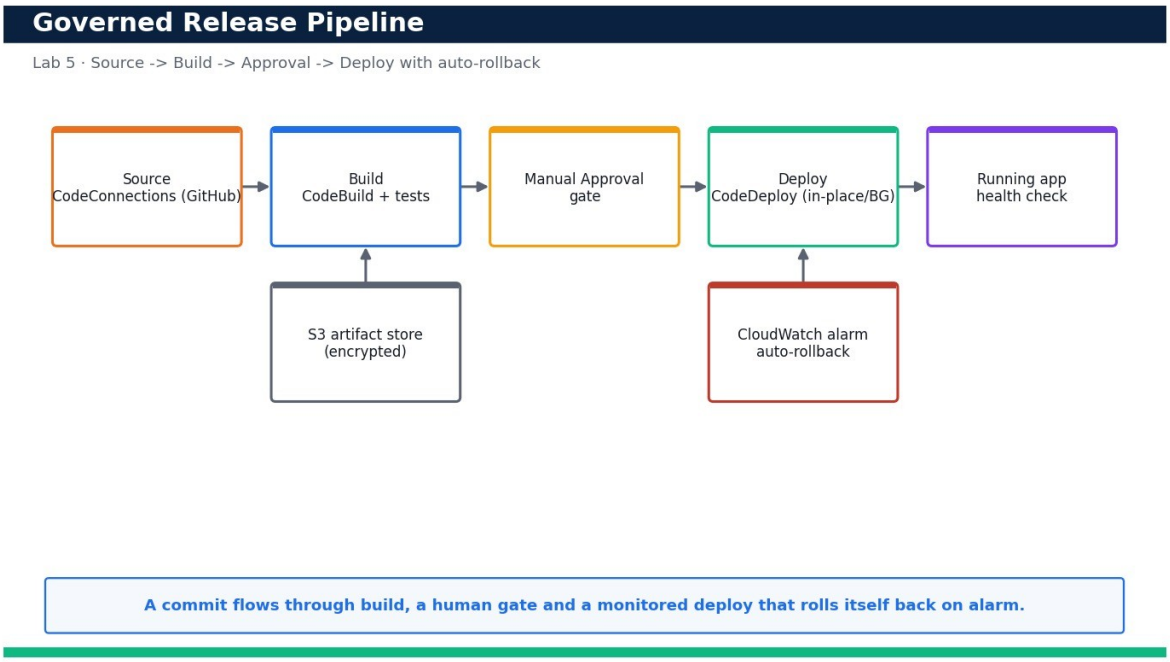


Figure — Lab 5 reference architecture: Assemble a CodePipeline with Manual Approval and a Deployment Strategy.

What you'll build

A four-stage CodePipeline (Source -> Build -> Approve -> Deploy) with a manual approval gate, a chosen deployment strategy and alarm-based automatic rollback. (Services: AWS CodePipeline, CodeBuild, CodeDeploy, CodeConnections, Amazon S3, Amazon CloudWatch, AWS CloudFormation.)

Step-by-step

57. Review pipeline.yaml, pipeline-definition.json and appspec.yml so you understand the four stages and the deploy hooks.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cat appspec.yml pipeline-definition.json
```

Expected: appspec.yml shows BeforeInstall/AfterInstall/ApplicationStart/ValidateService hooks; pipeline-definition.json shows Source, Build, Approve and Deploy stages.

58. Decide your deployment strategy: in-place (simplest) vs blue/green (zero-downtime). In-place redeploys onto the same instances; blue/green shifts traffic to a fresh set and keeps the old set for instant rollback.

Expected: You have chosen a strategy (IN_PLACE with WITH_TRAFFIC_CONTROL for this lab) and understand its rollback and downtime trade-offs.

59. Deploy the pipeline foundation stack (encrypted artifact bucket, CodeBuild project, CodeDeploy app and least-privilege roles).

```
aws cloudformation deploy --template-file pipeline.yaml --stack-name devops-cicd --capabilities CAPABILITY_NAMED_IAM --parameter-overrides ConnectionArn=<connection-arn> --profile devops
```

Expected: Stack devops-cicd reaches CREATE_COMPLETE and outputs the ArtifactBucket and CodeDeploy application name.

60. Create the CodeDeploy deployment group targeting your tagged instance(s) with the chosen in-place strategy.

```
aws deploy create-deployment-group --application-name devops-sample-app --deployment-group-name devops-dg --service-role-arn arn:aws:iam::<your-account-id>:role/devops-codedeploy-role --deployment-style deploymentType=IN_PLACE,deploymentOption=WITH_TRAFFIC_CONTROL --ec2-tag-filters Key=App,Value=devops-sample-app,Type=KEY_AND_VALUE --profile devops
```

Expected: A deploymentGroupId is returned; the group lists your target instances by tag and shows the chosen deployment type.

61. Create the pipeline from the definition, wiring the Source action to your AVAILABLE CodeConnections connection.

```
aws codepipeline create-pipeline --cli-input-json file://pipeline-definition.json --profile devops
```

Expected: The pipeline devops-cicd-pipeline is created and immediately starts its first execution from the connected repo.

62. Confirm the Source action uses the connection webhook (DetectChanges true), not polling.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws codepipeline get-pipeline --name devops-cicd-pipeline --query
'pipeline.stages[0].actions[0].configuration' --profile devops
```

Expected: Shows ConnectionArn, FullRepositoryId, BranchName and DetectChanges true.

63. Watch the first run: Source then Build should succeed (Build runs the tests from Lab 3).

```
aws codepipeline get-pipeline-state --name devops-cicd-pipeline
--query 'stageStates[?stageName==`Build`].latestExecution.status'
--profile devops
```

Expected: Source and Build report Succeeded; the pipeline then pauses at the Approve stage.

64. Find the pending approval token so you can approve the release.

```
aws codepipeline get-pipeline-state --name devops-cicd-pipeline
--query 'stageStates[?
stageName==`Approve`].actionStates[0].latestExecution.token' --
output text --profile devops
```

Expected: Returns the approval token for the ManualApproval action (status InProgress/pending).

65. Approve the Manual Approval action so the release proceeds to Deploy.

```
aws codepipeline put-approval-result --pipeline-name devops-cicd-
pipeline --stage-name Approve --action-name ManualApproval --
result summary=ok,status=Approved --token <token> --profile
devops
```

Expected: The Deploy stage transitions to InProgress; CodeDeploy starts deploying the artifact.

66. Confirm CodeDeploy reports a successful deployment with your chosen strategy.

```
aws deploy list-deployments --application-name devops-sample-app
--query 'deployments[0]' --output text --profile devops
```

Expected: Returns the latest deployment ID; get-deployment on it shows status Succeeded and the IN_PLACE/BLUE_GREEN style you chose.

67. Verify the deployed app end to end with the health check.

```
bash scripts/health_check.sh http://<deploy-target-dns>/
```

Expected: Prints HTTP 200 and the deployed version — the app is live and healthy.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

68. Create a CloudWatch alarm on deployment/app error rate that CodeDeploy can watch for rollback.

```
aws cloudwatch put-metric-alarm --alarm-name devops-deploy-errors
--namespace AWS/ApplicationELB --metric-name
HTTPCode_Target_5XX_Count --statistic Sum --period 60 --
evaluation-periods 1 --threshold 5 --comparison-operator
GreaterThanThreshold --profile devops
```

Expected: The alarm devops-deploy-errors is created in state OK.

69. Wire alarm-based automatic rollback into the deployment group so a bad release self-reverts.

```
aws deploy update-deployment-group --application-name devops-
sample-app --current-deployment-group-name devops-dg --auto-
rollback-configuration
enabled=true,events=DEPLOYMENT_FAILURE,DEPLOYMENT_STOP_ON_ALARM
--alarm-configuration enabled=true,alarms=[{name=devops-deploy-
errors}] --profile devops
```

Expected: The deployment group shows auto-rollback enabled on failure and on the alarm.

70. Prove the whole loop: push a change and watch Source -> Build -> Approve -> Deploy run automatically after approval.

```
git commit -am 'feat: headline change' && git push
```

Expected: A new execution starts within seconds; after you approve, the deployed app serves your change and the health check passes.

71. Prove the safety net: simulate a failing deployment (or trip the alarm) and confirm CodeDeploy rolls back to the last good revision.

```
aws deploy get-deployment --deployment-id <bad-deployment-id> --
query 'deploymentInfo.rollbackInfo' --profile devops
```

Expected: rollbackInfo shows a rollback was triggered and the app is restored to the previous, healthy revision.

Validation

The CodePipeline shows Source, Build, Approve and Deploy; a run reaches the Approve gate and only proceeds after `put-approval-result`; CodeDeploy reports Succeeded with the chosen strategy; the health check returns HTTP 200 with the deployed version; and the deployment group has alarm-based automatic rollback enabled that reverts a failing release.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

You can run the pipeline end to end, be blocked at a manual approval until you approve, see the app deployed and healthy, and watch an alarm-based rollback automatically restore the previous version when a deploy goes bad — a governed, self-healing release path.

Note: Cost & safety: This lab runs a v2 CodePipeline, CodeBuild minutes, one t3.micro deploy target, an encrypted S3 artifact bucket and a CloudWatch alarm. Kept to a single pipeline and torn down the same day it is a few US$; the deploy instance and build minutes are the main line items. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-05-codepipeline-approval-deploy/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Domain 02 — Configuration Management and IaC (17%)

Infrastructure as Code · reusable templates · multi-account & multi-Region

Key concepts

- Infrastructure as Code — Declare infrastructure in version-controlled templates so environments are repeatable, reviewable and disposable.
- CloudFormation — Declarative JSON/YAML stacks with parameters, mappings, outputs, change sets, drift detection and automatic rollback.
- Reusable components — Nested stacks and modules parameterise a pattern once and reuse it across teams and Regions.
- CDK vs CloudFormation vs Terraform — CDK generates CloudFormation from real code; Terraform is multi-cloud — all describe desired state.
- Configuration management — Systems Manager (State Manager, Automation, Parameter Store) keeps running instances in their desired configuration.
- Multi-account & multi-Region — Organizations, StackSets and Control Tower create, onboard and secure accounts at scale.

Lab 6 — Deploy a Reusable VPC Network Baseline with CloudFormation

Learning outcome: LO2. Skills mapped: K3, A3. Objective: Author, validate and deploy a reusable VPC network baseline with CloudFormation and publish cross-stack exports (LO2, K3, A3).

Goal: Your team keeps building VPCs by hand and no two look alike. In this lab you codify a golden network baseline as a single CloudFormation template — a VPC with

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

two public and two private subnets spread across two Availability Zones, an Internet Gateway and route tables — and publish its identifiers as cross-stack Exports so every downstream stack can import them. You validate the template, deploy it from a parameter file and inspect the resulting outputs and exports.

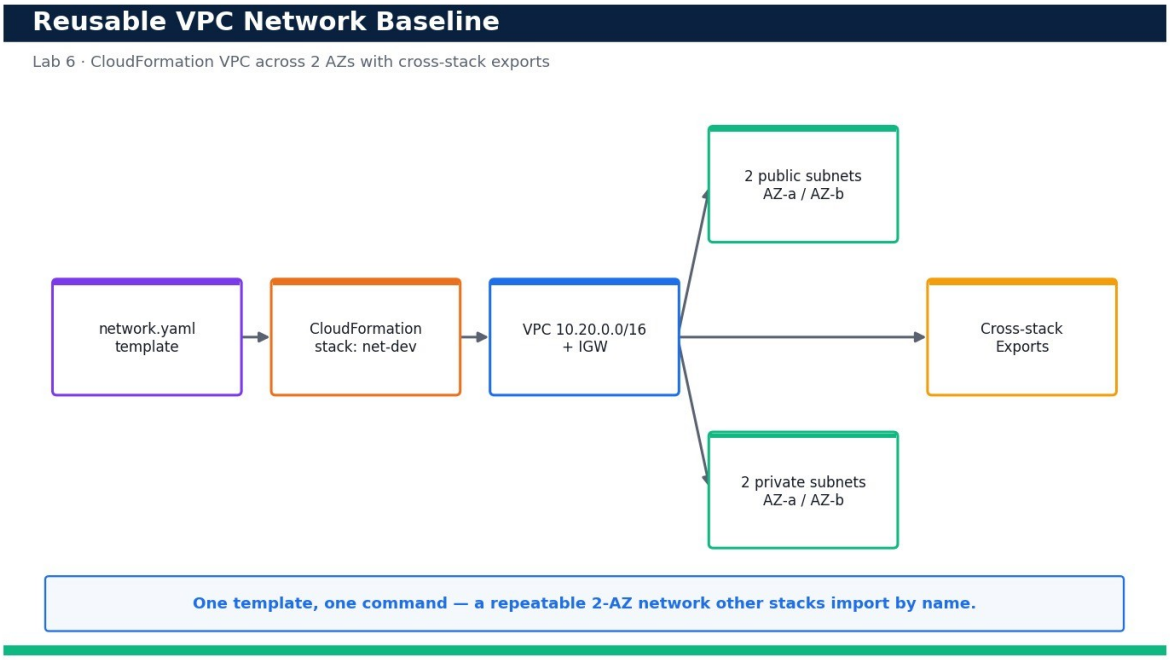


Figure — Lab 6 reference architecture: Deploy a Reusable VPC Network Baseline with CloudFormation.

What you'll build

A version-controlled network.yaml that deploys a 2-AZ VPC (2 public + 2 private subnets, IGW, route tables) and publishes VpcId, subnet IDs and the VPC CIDR as reusable cross-stack Exports. (Services: AWS CloudFormation, Amazon VPC, Amazon EC2.)

Step-by-step

72. Change into the lab folder and confirm your identity and Region before you touch anything.

```
aws sts get-caller-identity --profile devops
```

Expected: Your own Account, UserId and Arn are printed — confirming you are operating in the right account.

73. Validate the template syntax and capabilities before deploying so you catch errors locally.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws cloudformation validate-template --template-body
file://network.yaml --profile devops
```

Expected: The command returns the template Description and the list of Parameters with no error.

74. Lint the template's logical structure with the CloudFormation linter (cfn-lint) if installed.

```
cfn-lint network.yaml
```

Expected: cfn-lint exits 0 with no findings, confirming the template is well-formed.

75. Review the dev parameter file so you know exactly what values will be injected.

```
cat parameters-dev.json
```

Expected: The file lists EnvName=dev and VpcCidr=10.20.0.0/16 as ParameterKey/ParameterValue pairs.

76. Deploy the network baseline as a named stack using the deploy helper script (it wraps aws cloudformation deploy).

```
bash deploy.sh dev
```

Expected: The script prints 'Deploying net-dev...' and CloudFormation reports the stack reaching CREATE_COMPLETE.

77. Confirm the stack finished successfully.

```
aws cloudformation describe-stacks --stack-name net-dev --query
'Stacks[0].StackStatus' --profile devops
```

Expected: The command prints "CREATE_COMPLETE".

78. Inspect the stack Outputs to see the identifiers the baseline produced.

```
aws cloudformation describe-stacks --stack-name net-dev --query
'Stacks[0].Outputs' --profile devops
```

Expected: You see OutputKeys VpcId, PublicSubnetAId, PublicSubnetBId, PrivateSubnetAId, PrivateSubnetBId and VpcCidr with real IDs.

79. List the cross-stack Exports so you can confirm they are published for other stacks to import.

```
aws cloudformation list-exports --query "Exports[?
starts_with(Name,'dev-')]" --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: Exports dev-VpcId, dev-PublicSubnetAId, dev-PublicSubnetBId, dev-PrivateSubnetAId, dev-PrivateSubnetBId and dev-VpcCidr are listed with their values.

80. Confirm the VPC exists and DNS support is enabled.

```
aws ec2 describe-vpcs --filters Name=tag:Name,Values=dev-vpc --
query 'Vpcs[0].{Id:VpcId,Cidr:CidrBlock,Dns:EnableDnsSupport}' --
profile devops
```

Expected: The VPC is returned with CIDR 10.20.0.0/16 and Dns set to true.

81. Verify the four subnets landed in two distinct Availability Zones.

```
aws ec2 describe-subnets --filters Name=vpc-id,Values=<vpc-id> --
query 'Subnets[].
{Az:AvailabilityZone,Cidr:CidrBlock,Public:MapPublicIpOnLaunch}'
--profile devops
```

Expected: Four subnets are listed across two AZs; the two public subnets show Public=true and the two private show Public=false.

82. Check that the Internet Gateway is attached to the VPC.

```
aws ec2 describe-internet-gateways --filters Name=attachment.vpc-
id,Values=<vpc-id> --query
'InternetGateways[0].Attachments[0].State' --profile devops
```

Expected: The attachment State is "available", confirming the IGW is attached.

83. Confirm the public route table has a default route (0.0.0.0/0) to the Internet Gateway.

```
aws ec2 describe-route-tables --filters Name=vpc-id,Values=<vpc-
id> --query 'RouteTables[].Routes[?
DestinationCidrBlock==`0.0.0.0/0`]' --profile devops
```

Expected: One route table shows a 0.0.0.0/0 route whose GatewayId is your igw-... , confirming internet egress for public subnets.

84. Prove reusability: deploy a second, isolated copy of the same baseline for the test environment.

```
bash deploy.sh test
```

Expected: A separate stack net-test reaches CREATE_COMPLETE using VpcCidr 10.30.0.0/16 and publishes test-* exports.

85. Detect drift immediately after deployment to establish a clean baseline for later labs.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws cloudformation detect-stack-drift --stack-name net-dev --
profile devops
```

Expected: A StackDriftDetectionId is returned; a follow-up describe shows StackDriftStatus IN_SYNC.

86. Render a change-free change set to prove the deployed stack matches the template exactly.

```
aws cloudformation deploy --template-file network.yaml --stack-
name net-dev --parameter-overrides EnvName=dev
VpcCidr=10.20.0.0/16 --no-execute-changeset --profile devops
```

Expected: CloudFormation reports 'No changes to deploy' (or an empty change set), confirming template and stack are in sync.

Validation

In the CloudFormation console stack net-dev shows CREATE_COMPLETE with an Outputs tab listing the VPC, subnet and CIDR values, and the Exports appear under CloudFormation → Exports as dev-*. describe-vpcs, describe-subnets and describe-route-tables confirm a 2-AZ VPC with a public route to the IGW, and drift detection reports IN_SYNC.

Test it

Deploying network.yaml with parameters-dev.json produces a reusable 2-AZ VPC whose IDs are published as cross-stack Exports; re-running the deploy reports no changes and a second (test) deployment reproduces an identical, isolated network — proving the baseline is repeatable and importable.

Note: Cost & safety: A VPC, subnets, route tables and an Internet Gateway are all free of charge; no NAT Gateway or EC2 is created in this lab, so it is effectively $0. Delete the stack when finished to keep your account tidy. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-06-cloudformation-network-baseline/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 7 — Deploy a Parameterized Application Stack that Imports Network Exports

Learning outcome: LO2. Skills mapped: K3, A3. Objective: Deploy a parameterized application stack that consumes cross-stack exports and reuse it for dev and test via parameter files (LO2, K3, A3).

Goal: With the network baseline from Lab 6 in place, you now deploy the application tier on top of it — without copy-pasting a single subnet or VPC ID. The app template uses Fn::ImportValue to pull the network's exports, an SSM public parameter to always resolve the latest Amazon Linux AMI, and CloudFormation Parameters (with a Mappings-driven instance size and a Conditions-driven Auto Scaling group) so the very same template produces a dev single instance and a test Auto Scaling group just by swapping a parameter file.

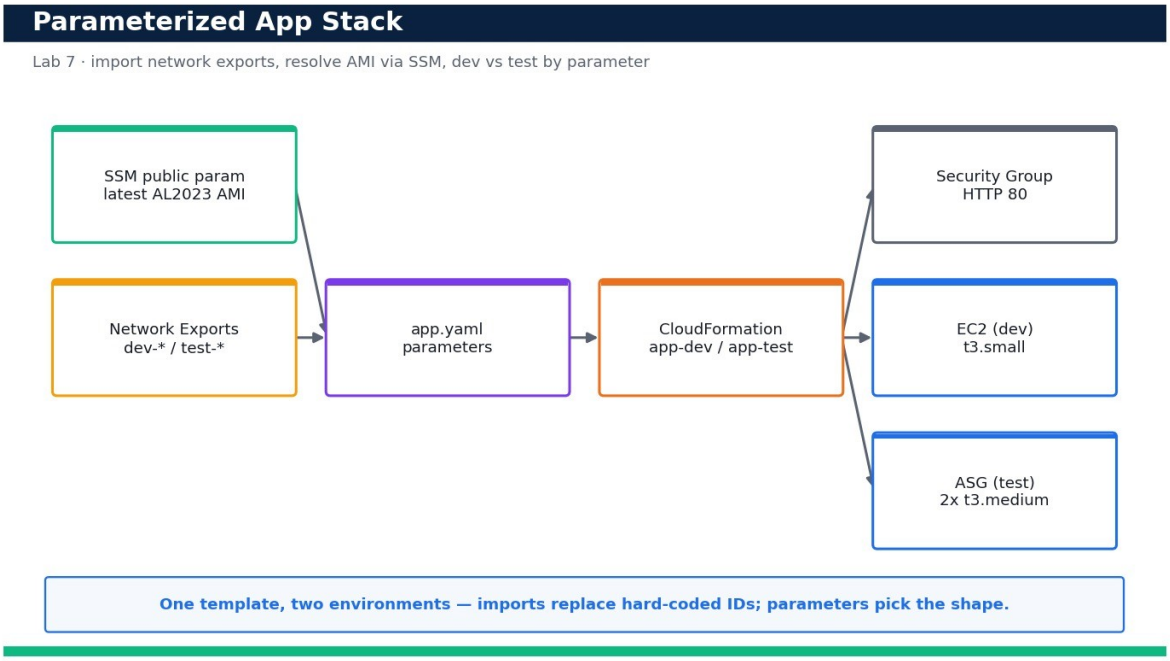


Figure — Lab 7 reference architecture: Deploy a Parameterized Application Stack that Imports Network Exports.

What you'll build

A parameterized app.yaml that imports the network exports, resolves the latest AMI via SSM and deploys a security group plus either a single EC2 instance (dev) or an Auto Scaling group (test) from the same template. (Services: AWS CloudFormation, Amazon EC2, AWS Systems Manager (SSM public parameters).)

Step-by-step

87. Confirm the Lab 6 network stack still exists and its exports are available to import.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws cloudformation list-exports --query "Exports[?Name=='dev-VpcId']" --profile devops
```

Expected: The dev-VpcId export is listed with a vpc-... value, confirming the network baseline is present.

88. Validate the app template before deploying.

```
aws cloudformation validate-template --template-body file://app.yaml --profile devops
```

Expected: The Description and Parameters (EnvName, InstanceSize, DeployMode) are returned with no error.

89. Inspect the dev parameter file to see the values that select a single instance.

```
cat parameters.json
```

Expected: The file sets EnvName=dev, InstanceSize=small and DeployMode=single.

90. Preview which AMI the SSM public parameter will resolve to, so you know the deployment is using the latest image.

```
aws ssm get-parameters --names /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 --query 'Parameters[0].Value' --profile devops
```

Expected: A current ami-... ID for Amazon Linux 2023 is printed.

91. Deploy the dev application stack; it will import the network exports automatically.

```
aws cloudformation deploy --template-file app.yaml --stack-name app-dev --parameter-overrides EnvName=dev InstanceSize=small DeployMode=single --capabilities CAPABILITY_NAMED_IAM --profile devops
```

Expected: Stack app-dev reaches CREATE_COMPLETE.

92. Confirm the security group was created in the imported VPC (not a default VPC).

```
aws cloudformation describe-stack-resources --stack-name app-dev --logical-resource-id AppSg --query 'StackResources[0].PhysicalResourceId' --profile devops
```

Expected: A sg-... ID is returned; describe-security-groups shows it belongs to the dev VPC.

93. Verify the dev stack created exactly one EC2 instance (single mode) placed in an imported public subnet.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws ec2 describe-instances --filters
Name=tag:aws:cloudformation:stack-name,Values=app-dev
Name=instance-state-name,Values=running --query
'Reservations[].Instances[].
{Id:InstanceId,Subnet:SubnetId,Type:InstanceType}' --profile
devops
```

Expected: Exactly one running instance of type t3.small is returned in the dev-public-a subnet.

94. Read the stack Outputs to get the app endpoint.

```
aws cloudformation describe-stacks --stack-name app-dev --query
'Stacks[0].Outputs' --profile devops
```

Expected: An AppEndpoint output (public DNS name) and the SecurityGroupId are returned.

95. Confirm the app is serving over HTTP once the instance has bootstrapped.

```
curl -fsS http://<app-dev-public-dns>/ | head -n 1
```

Expected: The instance returns HTTP 200 and a line such as 'OK from ip-10-20-1-xx' proving the web server is up.

96. Now reuse the SAME template for the test environment with a different parameter file that selects an Auto Scaling group.

```
aws cloudformation deploy --template-file app.yaml --stack-name
app-test --parameter-overrides file://params-test.json --
capabilities CAPABILITY_NAMED_IAM --profile devops
```

Expected: Stack app-test reaches CREATE_COMPLETE using DeployMode=asg and InstanceSize=medium.

97. Prove the Conditions logic worked: the test stack has an Auto Scaling group instead of a standalone instance.

```
aws autoscaling describe-auto-scaling-groups --query
"AutoScalingGroups[?contains(AutoScalingGroupName,'app-test')].
{Name:AutoScalingGroupName,Desired:DesiredCapacity,Instances:leng
th(Instances)}" --profile devops
```

Expected: One ASG for app-test is listed with DesiredCapacity 2 and 2 healthy instances.

98. Confirm the ASG spread its instances across BOTH imported private subnets (multi-AZ).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws ec2 describe-instances --filters
Name=tag:aws:cloudformation:stack-name,Values=app-test --query
'Reservations[].Instances[].Placement.AvailabilityZone' --profile
devops
```

Expected: Two distinct Availability Zones are returned, confirming the ASG used both imported subnets.

99. Demonstrate safe change management: change only a parameter (bump the dev instance size) and let CloudFormation compute the update.

```
aws cloudformation deploy --template-file app.yaml --stack-name
app-dev --parameter-overrides EnvName=dev InstanceSize=medium
DeployMode=single --capabilities CAPABILITY_NAMED_IAM --profile
devops
```

Expected: app-dev reaches UPDATE_COMPLETE and the instance type is replaced with t3.medium via the Mappings lookup.

100. Confirm the two environments are truly independent by comparing their statuses.

```
aws cloudformation describe-stacks --query "Stacks[?
StackName=='app-dev'||StackName=='app-test'].
{Name:StackName,Status:StackStatus}" --profile devops
```

Expected: Both app-dev and app-test are listed independently at *_COMPLETE, proving one template drives two isolated environments.

Validation

CloudFormation shows app-dev and app-test both at a *_COMPLETE status; the dev stack contains a single EC2 instance in an imported subnet while the test stack contains a 2-instance, multi-AZ Auto Scaling group — all built from the same app.yaml, differing only by parameter file. The security groups live in the imported VPC and the app answers HTTP 200.

Test it

The same app.yaml, deployed with parameters.json, yields a dev single instance, and with params-test.json yields a multi-AZ Auto Scaling group — both importing the network exports and resolving the latest AMI via SSM. Changing a single parameter drives a controlled UPDATE_COMPLETE, proving the stack is fully parameterized.

Note: Cost & safety: Dev deploys one t3.small; the test path deploys two t3.medium instances in an Auto Scaling group, billed per second while running. Delete both stacks the same day; the security group and SSM lookup are free. Always run the clean-up step; never commit real credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-07-parameterized-app-stack/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 8 — Preview Changes with Change Sets and Detect & Reconcile Drift

Learning outcome: LO2. Skills mapped: K3, A3. Objective: Preview a stack modification safely with a change set before executing, then detect out-of-band drift and reconcile it (LO2, K3, A3).

Goal: Production changes are risky when you cannot see them coming, and infrastructure silently drifts when people click in the console. In this lab you take the app stack from Lab 7, generate a change set to preview a before-you-execute diff, execute it, then deliberately make an out-of-band change directly on a resource. You run CloudFormation drift detection to catch the drift, inspect exactly what differs, and reconcile the stack back to its declared state.

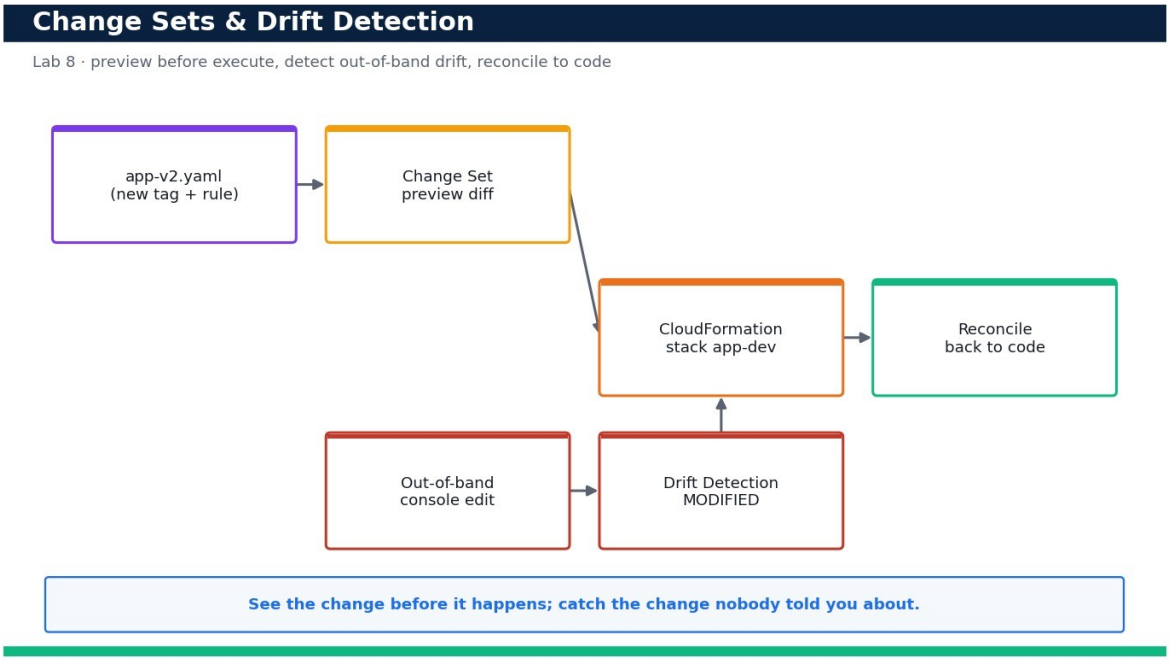


Figure — Lab 8 reference architecture: Preview Changes with Change Sets and Detect & Reconcile Drift.

What you'll build

A repeatable workflow: create-change-set.sh renders a reviewable preview of a stack change, and drift-check.sh detects and reports out-of-band drift so you can reconcile infrastructure back to code. (Services: AWS CloudFormation.)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

101. Confirm the app-dev stack from Lab 7 is present and healthy before you change it.

```
aws cloudformation describe-stacks --stack-name app-dev --query
'Stacks[0].StackStatus' --profile devops
```

Expected: The status is CREATE_COMPLETE or UPDATE_COMPLETE.

102. Diff the new template against the current one locally so you understand your intended change.

```
diff app.yaml app-v2.yaml || true
```

Expected: The diff shows an added CostCenter tag and a narrowed SSH ingress rule (203.0.113.0/24 instead of 0.0.0.0/0).

103. Create a change set to PREVIEW the modification without applying it (using the helper script).

```
bash create-change-set.sh app-dev cs-add-guardrails
```

Expected: The script reports the change set cs-add-guardrails reaching CREATE_COMPLETE and prints a summary of changes.

104. Inspect exactly what the change set will do before you commit to it.

```
aws cloudformation describe-change-set --stack-name app-dev --
change-set-name cs-add-guardrails --query
'Changes[].ResourceChange.
{Action:Action,Resource:LogicalResourceId,Replace:Replacement}'
--profile devops
```

Expected: The change list shows AppSg being Modify with Replacement=False and no resource deletions — a safe change.

105. Execute the change set now that you have reviewed it.

```
aws cloudformation execute-change-set --stack-name app-dev --
change-set-name cs-add-guardrails --profile devops
```

Expected: The stack transitions to UPDATE_IN_PROGRESS then UPDATE_COMPLETE.

106. Confirm the tag and tightened rule are in effect.

```
aws cloudformation describe-stacks --stack-name app-dev --query
'Stacks[0].Tags' --profile devops
```

Expected: The Tags list now includes CostCenter=devops-training.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

107. Now simulate a colleague making an out-of-band change: add an unmanaged ingress rule directly to the security group.

```
aws ec2 authorize-security-group-ingress --group-id <app-sg-id>
--protocol tcp --port 22 --cidr 0.0.0.0/0 --profile devops
```

Expected: AWS confirms the SecurityGroupRules were added — the live resource no longer matches the template.

108. Kick off drift detection on the stack.

```
bash drift-check.sh app-dev
```

Expected: The script starts detection, polls until it finishes, and prints StackDriftStatus DRIFTED.

109. List which resources drifted.

```
aws cloudformation describe-stack-resource-drifts --stack-name
app-dev --stack-resource-drift-status-filters MODIFIED --query
'StackResourceDrifts[].
{Resource:LogicalResourceId,Status:StackResourceDriftStatus}' --
profile devops
```

Expected: AppSg is reported as MODIFIED.

110. Drill into the exact property difference so you know what changed out-of-band.

```
aws cloudformation describe-stack-resource-drifts --stack-name
app-dev --query 'StackResourceDrifts[?
LogicalResourceId==`AppSg`].PropertyDifferences' --profile devops
```

Expected: A PropertyDifference of type ADD is shown for the SecurityGroupIngress list — the extra port-22 rule that is not in the template.

111. Reconcile the drift the correct way: remove the out-of-band rule so the resource matches code again.

```
aws ec2 revoke-security-group-ingress --group-id <app-sg-id> --
protocol tcp --port 22 --cidr 0.0.0.0/0 --profile devops
```

Expected: AWS confirms the rule was revoked, restoring the security group to its declared state.

112. Re-run drift detection to prove the stack is reconciled.

```
bash drift-check.sh app-dev
```

Expected: StackDriftStatus is now IN_SYNC.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

113. As an alternative reconciliation path, prove that re-applying the template also removes drift.

```
aws cloudformation deploy --template-file app-v2.yaml --stack-
name app-dev --parameter-overrides EnvName=dev
InstanceSize=medium DeployMode=single --capabilities
CAPABILITY_NAMED_IAM --profile devops
```

Expected: CloudFormation reports either 'No changes' (already reconciled) or overwrites any remaining drift back to the template.

114. Clean up the change set you created (executed change sets remain listed until deleted).

```
aws cloudformation delete-change-set --stack-name app-dev --
change-set-name cs-add-guardrails --profile devops
```

Expected: The change set is deleted; the stack itself remains at UPDATE_COMPLETE and IN_SYNC.

Validation

The change set is visible under the stack's Change sets tab with a readable diff before execution; after executing, the stack is UPDATE_COMPLETE with the CostCenter tag. After the out-of-band rule is added, drift detection reports DRIFTED with AppSg MODIFIED and shows the exact extra ingress rule; after reconciliation, drift detection reports IN_SYNC.

Test it

A change set previews the modification (added tag + narrowed rule) with no unexpected replacements before it is executed; an out-of-band security-group edit is caught by drift detection as MODIFIED with the precise property difference; and removing the rule (or re-applying the template) returns the stack to IN_SYNC.

Note: Cost & safety: This lab reuses the Lab 7 app-dev stack and only adds a tag and a tightened security-group rule; there are no new billable resources beyond the existing instance. Change sets and drift detection are free. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-08-changesets-drift-detection/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 9 — Externalize Configuration with SSM Parameter Store

Learning outcome: LO2. Skills mapped: K3, A3. Objective: Externalize application configuration into SSM Parameter Store (String and SecureString), read it at runtime, and compare Parameter Store with Secrets Manager (LO2, K3, A3).

Goal: Hard-coding configuration and credentials into code and templates is a recipe for leaks and rebuilds. In this lab you externalize your application's configuration into AWS Systems Manager Parameter Store — plain String and String List values for non-sensitive settings and a KMS-encrypted SecureString for a sensitive value. You create a hierarchy of parameters (deployed both by CLI and by CloudFormation), have an app read them at runtime, learn where State Manager and Automation fit for keeping configuration applied, and compare Parameter Store with Secrets Manager to choose the right tool.

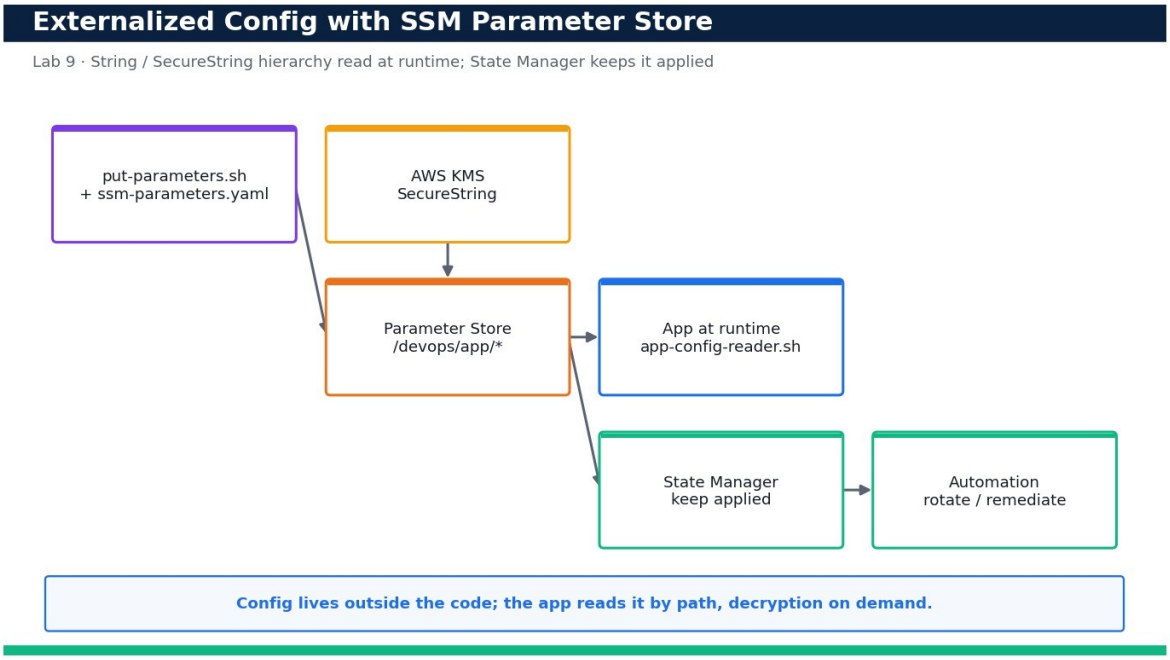


Figure — Lab 9 reference architecture: Externalize Configuration with SSM Parameter Store.

What you'll build

A /devops/app/* parameter hierarchy in SSM (String, StringList and KMS-encrypted SecureString) plus an app-config-reader.sh that pulls the whole configuration at runtime with decryption. (Services: AWS Systems Manager (Parameter Store, State Manager, Automation), AWS KMS, AWS CloudFormation.)

Step-by-step

115. Confirm your identity and the Region where the parameters will live.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws sts get-caller-identity --profile devops
```

Expected: Your Account and Arn are printed.

116. Create the non-sensitive configuration values (String and StringList) using the put-parameters helper.

```
bash put-parameters.sh
```

Expected: The script reports each parameter created: /devops/app/log-level, /devops/app/feature-flags and /devops/app/db/endpoint.

117. Create the sensitive value as a KMS-encrypted SecureString (the AWS-managed alias/aws/ssm key).

```
aws ssm put-parameter --name /devops/app/db/password --type
SecureString --value '<placeholder-set-your-own>' --key-id
alias/aws/ssm --overwrite --profile devops
```

Expected: A Version number is returned; the value is stored encrypted at rest.

118. List the whole parameter hierarchy by path to confirm the tree.

```
aws ssm get-parameters-by-path --path /devops/app --recursive --
query 'Parameters[].{Name:Name,Type:Type}' --profile devops
```

Expected: The four parameters are listed with types String, StringList and SecureString under /devops/app.

119. Read a non-sensitive parameter the way an app would.

```
aws ssm get-parameter --name /devops/app/log-level --query
'Parameter.Value' --profile devops
```

Expected: The command prints "INFO".

120. Show that a SecureString is NOT decrypted by default (safe-by-default behaviour).

```
aws ssm get-parameter --name /devops/app/db/password --query
'Parameter.Value' --profile devops
```

Expected: An encrypted, base64-looking ciphertext is returned — not the plaintext.

121. Now read it WITH decryption, as the app's IAM role would (requires ssm:GetParameter + kms:Decrypt).

```
aws ssm get-parameter --name /devops/app/db/password --with-
decryption --query 'Parameter.Value' --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: The plaintext value is returned only because --with-decryption was supplied and you have kms:Decrypt.

122. Run the app config reader to pull the entire configuration at runtime in one place.

```
bash app-config-reader.sh
```

Expected: The script prints LOG_LEVEL=INFO, FEATURE_FLAGS=beta,newui and a masked DB_PASSWORD, proving runtime config loading.

123. Deploy an additional parameter via CloudFormation so config is also version-controlled as IaC.

```
aws cloudformation deploy --template-file ssm-parameters.yaml --
stack-name app-config --parameter-overrides EnvName=dev --profile
devops
```

Expected: Stack app-config reaches CREATE_COMPLETE and creates /devops/app/dev/max-connections.

124. Confirm the CloudFormation-managed parameter exists.

```
aws ssm get-parameter --name /devops/app/dev/max-connections --
query 'Parameter.Value' --profile devops
```

Expected: The command prints "100".

125. Update a parameter value and observe the version increment (parameters keep a history).

```
aws ssm put-parameter --name /devops/app/log-level --type String
--value DEBUG --overwrite --profile devops
```

Expected: Version 2 is returned; the value is now DEBUG.

126. View the parameter's version history to prove auditable configuration change.

```
aws ssm get-parameter-history --name /devops/app/log-level --
query 'Parameters[].{Version:Version,Value:Value}' --profile
devops
```

Expected: Two entries are listed: Version 1 = INFO and Version 2 = DEBUG.

127. Concept check — list State Manager associations so you understand how SSM keeps configuration continuously applied to targets.

```
aws ssm list-associations --query 'Associations[].
{Name:Name,Targets:Targets}' --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: Existing associations (if any) are listed; you note that State Manager enforces desired config on a schedule and Automation runbooks remediate/rotate.

128. Compare with Secrets Manager: confirm the API is available and note the trade-off (built-in rotation vs cost).

```
aws secretsmanager list-secrets --query 'SecretList[].Name' --
profile devops
```

Expected: The (possibly empty) secret list returns successfully — you record that Secrets Manager adds managed rotation and cross-service integration at a per-secret cost, whereas SecureString is cheaper for simple encrypted config.

Validation

get-parameters-by-path shows the /devops/app hierarchy with String, StringList and SecureString types; the SecureString returns ciphertext without --with-decryption and plaintext with it; app-config-reader.sh loads the full configuration at runtime; the CloudFormation-managed parameter exists; and get-parameter-history proves versioned, auditable changes.

Test it

An application reads all of its configuration from the /devops/app/* Parameter Store hierarchy at runtime — plain values in the clear and the sensitive value only after KMS decryption — with no config baked into code. You can state when to use Parameter Store SecureString versus Secrets Manager, and where State Manager and Automation keep that configuration continuously applied.

Note: Cost & safety: Standard-tier Parameter Store parameters and the AWS-managed KMS key are free; you are billed only for API calls at negligible volumes. Advanced-tier parameters and higher throughput would cost more but are not used here. Delete the parameters when finished. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-09-ssm-parameter-store-config/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 10 — Multi-Account, Multi-Region Guardrails with CloudFormation StackSets

Learning outcome: LO2. Skills mapped: K3, A3. Objective: Design and deploy a multi-account, multi-Region guardrail using CloudFormation StackSets with AWS Organizations service-managed permissions, and express a preventive guardrail as an SCP (LO2, K3, A3).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: Scaling infrastructure to many accounts by hand does not work. In this lab you build a governance blueprint: using CloudFormation StackSets with AWS Organizations service-managed permissions, you deploy a detective guardrail stack (an AWS Config managed rule and an SNS notification topic) to every account in an Organizational Unit across two Regions from a single operation. You then express a matching preventive guardrail as a Service Control Policy, and note where AWS Control Tower automates this whole pattern. Where you lack a real Organization, the same commands and templates form a ready-to-run blueprint.

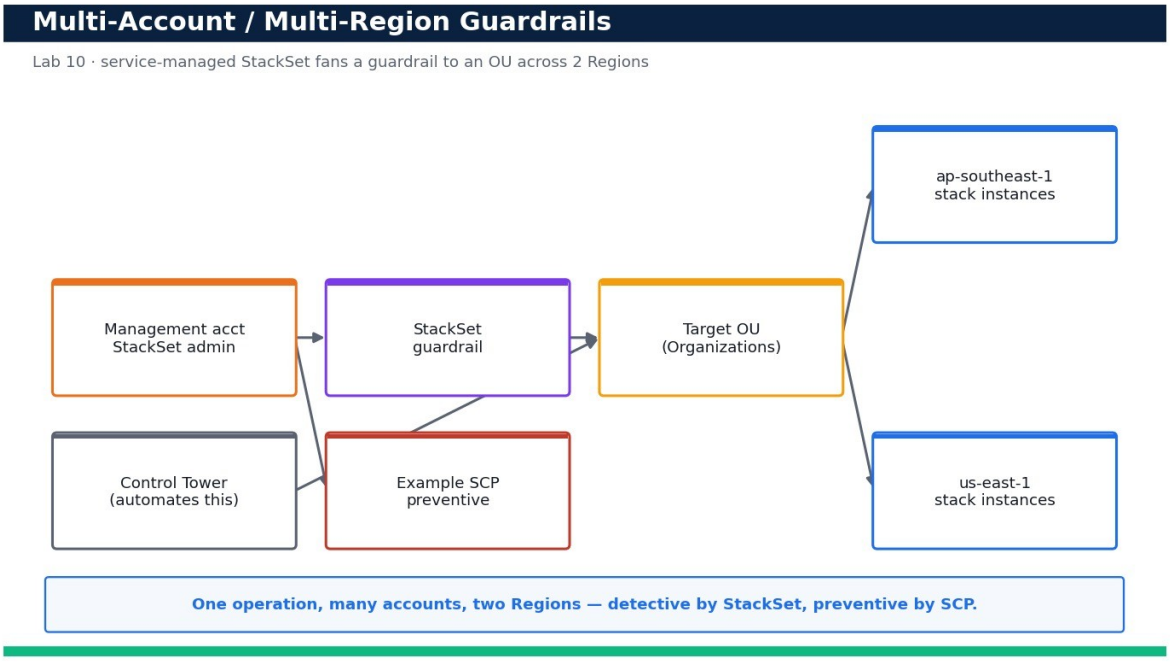


Figure — Lab 10 reference architecture: Multi-Account, Multi-Region Guardrails with CloudFormation StackSets.

What you'll build

A StackSet (service-managed) that fans a guardrail stack out to an OU across two Regions in one operation, plus an example SCP that enforces the same guardrail preventively. (Services: AWS CloudFormation StackSets, AWS Organizations, AWS Control Tower (concept).)

Step-by-step

129. Confirm you are operating from the Organizations management (or delegated administrator) account.

```
aws organizations describe-organization --query 'Organization.
{Id:Id,Feature:FeatureSet}' --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: The organization Id and FeatureSet ALL are returned (ALL is required for service-managed StackSets and SCPs).

130. Enable trusted access so CloudFormation StackSets can deploy across the Organization.

```
aws cloudformation activate-organizations-access --profile devops
```

Expected: The call succeeds with no error; StackSets can now use service-managed permissions.

131. Identify the target Organizational Unit you will deploy the guardrail to.

```
aws organizations list-organizational-units-for-parent --parent-
id <root-id> --query 'OrganizationalUnits[].{Name:Name,Id:Id}' --
profile devops
```

Expected: Your OUs are listed; note the OU Id (ou-xxxx-xxxxxxxx) of the workloads OU.

132. Validate the guardrail template before creating the StackSet.

```
aws cloudformation validate-template --template-body
file://stackset-template.yaml --profile devops
```

Expected: The Description and Parameters are returned with no error.

133. Create the StackSet with service-managed permissions and auto-deployment enabled.

```
aws cloudformation create-stack-set --stack-set-name org-
guardrails --template-body file://stackset-template.yaml --
permission-model SERVICE_MANAGED --auto-deployment
Enabled=true,RetainStacksOnAccountRemoval=false --capabilities
CAPABILITY_NAMED_IAM --profile devops
```

Expected: A StackSetId is returned, confirming the StackSet was created under service-managed permissions.

134. Deploy stack instances to the target OU across BOTH Regions in a single operation (the deploy-stackset script wraps this).

```
bash deploy-stackset.sh org-guardrails <ou-id> ap-southeast-1 us-
east-1
```

Expected: An OperationId is returned and the script reports instances being created for the OU in both Regions.

135. Track the deployment operation until it succeeds.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws cloudformation describe-stack-set-operation --stack-set-name
org-guardrails --operation-id <operation-id> --query
'StackSetOperation.Status' --profile devops
```

Expected: The Status progresses to SUCCEEDED.

136. List the stack instances to confirm the guardrail landed in each target account and Region.

```
aws cloudformation list-stack-instances --stack-set-name org-
guardrails --query 'Summaries[].
{Account:Account,Region:Region,Status:Status}' --profile devops
```

Expected: One row per account-and-Region pair is listed with Status CURRENT — proving fan-out to the OU across two Regions.

137. Confirm the detective guardrail (Config rule) is actually present in a target account/Region.

```
aws configservice describe-config-rules --config-rule-names s3-
bucket-public-read-prohibited --profile devops
```

Expected: The managed Config rule s3-bucket-public-read-prohibited is returned in ACTIVE state (view from within a target account).

138. Now express the SAME guardrail preventively as a Service Control Policy — review the example SCP first.

```
cat example-scp.json
```

Expected: The SCP denies removing the S3 public-access block and denies public-ACL puts, complementing the detective Config rule.

139. Create the SCP in the Organization (SCPs must be enabled as a policy type first).

```
aws organizations create-policy --name deny-public-s3 --type
SERVICE_CONTROL_POLICY --description 'Preventive S3 public-access
guardrail' --content file://example-scp.json --profile devops
```

Expected: A PolicyId (p-xxxxxxxx) is returned for the new SCP.

140. Attach the SCP to the same target OU so prevention and detection cover the same scope.

```
aws organizations attach-policy --policy-id <policy-id> --target-
id <ou-id> --profile devops
```

Expected: The attach call succeeds; the SCP now constrains every account in the OU.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

141. Verify the SCP is attached to the OU.

```
aws organizations list-policies-for-target --target-id <ou-id> --
filter SERVICE_CONTROL_POLICY --query 'Policies[].Name' --profile
devops
```

Expected: deny-public-s3 (plus FullAWSAccess) is listed for the OU.

142. Prove the blueprint scales to new accounts automatically: describe the StackSet's auto-deployment setting.

```
aws cloudformation describe-stack-set --stack-set-name org-
guardrails --query 'StackSet.AutoDeployment' --profile devops
```

Expected: AutoDeployment shows Enabled=true — any account later added to the OU receives the guardrail automatically (the pattern AWS Control Tower automates end-to-end).

Validation

list-stack-instances shows the guardrail stack CURRENT in every target account across ap-southeast-1 and us-east-1 from a single operation; the AWS Config managed rule is ACTIVE in a target account; the example SCP is created and attached to the OU (visible via list-policies-for-target); and the StackSet's AutoDeployment is Enabled so new accounts inherit the guardrail.

Test it

From the management account you deploy one guardrail stack to an entire OU across two Regions in a single StackSet operation (detective control), attach a matching SCP to the same OU (preventive control), and confirm auto-deployment will extend the guardrail to any newly added account — the multi-account, multi-Region pattern AWS Control Tower operationalizes.

Note: Cost & safety: The guardrail stack deploys an AWS Config managed rule and an SNS topic per account per Region; Config rule evaluations and SNS are low-cost but multiply by the number of target accounts and Regions. Deploy to a small OU (or a single sandbox account) and delete the stack instances promptly to control spend. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-10-multiaccount-region-stacksets/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Domain 03 — Resilient Cloud Solutions (15%)

High availability · scalability · automated recovery (RTO/RPO)

Key concepts

- High availability — Spread compute across multiple Availability Zones behind an Elastic Load Balancer so one AZ failure is survivable.
- Elasticity & Auto Scaling — Auto Scaling groups add and remove capacity on demand using target-tracking and step scaling policies.
- Decoupling — SQS, SNS and EventBridge decouple producers from consumers so components fail and scale independently.
- RTO & RPO — Recovery Time / Point Objectives drive the DR pattern: backup & restore, pilot light, warm standby or multi-site active/active.
- Data durability — Multi-AZ RDS, DynamoDB global tables, S3 cross-Region replication and automated snapshots protect state.
- Automated recovery — EC2 auto-recovery, ASG health checks and Route 53 health-check failover restore service without human action.

Lab 11 — Deploy a Multi-AZ Application Load Balancer with an Auto Scaling Group

Learning outcome: LO3. Skills mapped: K2, K4, A2. Objective: Deploy a highly available multi-AZ Application Load Balancer fronting an Auto Scaling group and prove requests are served from both Availability Zones (LO3, K2, K4, A2).

Goal: A single instance in a single Availability Zone is a single point of failure. In this lab you codify a highly available web tier as one CloudFormation template: an internet-facing Application Load Balancer spanning two AZs, a launch template describing identical instances, an Auto Scaling group that keeps a minimum of two instances balanced across both AZs, and ELB health checks that replace anything unhealthy. You then curl the load balancer repeatedly and confirm responses come back from instances in both AZs.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

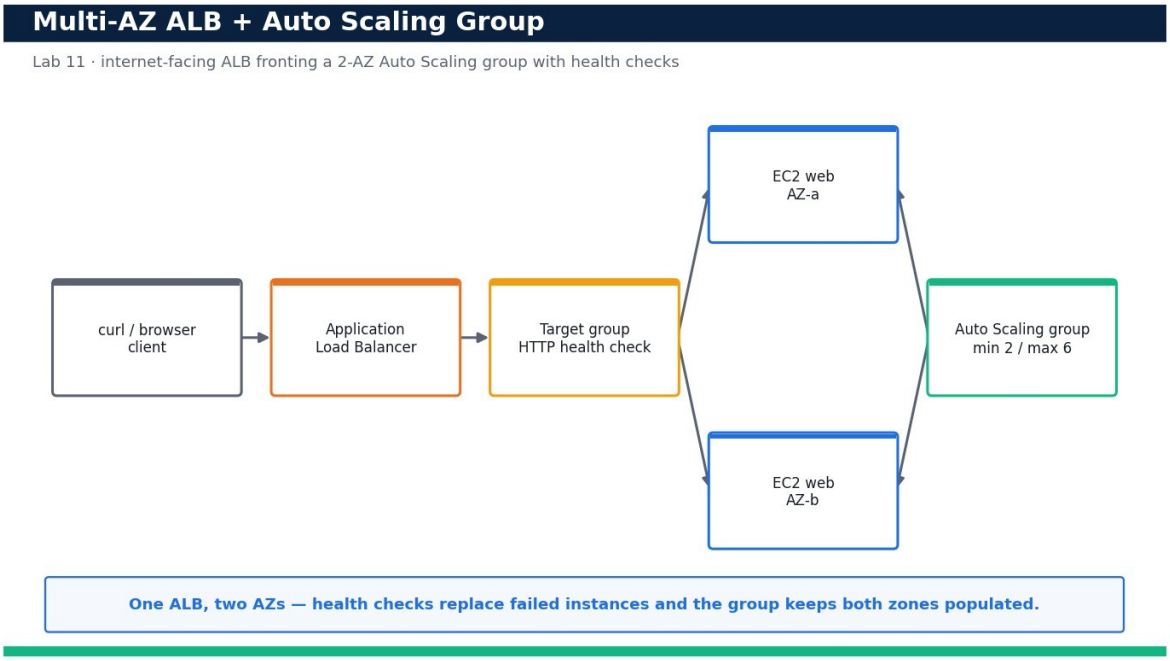


Figure — Lab 11 reference architecture: Deploy a Multi-AZ Application Load Balancer with an Auto Scaling Group.

What you'll build

A running multi-AZ stack — an ALB, a target group with health checks, a launch template and an Auto Scaling group (min 2 / max 6) spread across two AZs — whose DNS name serves traffic from instances in both AZs. (Services: Amazon EC2, Elastic Load Balancing (ELBv2 / ALB), AWS Auto Scaling, AWS CloudFormation.)

Step-by-step

143. Change into the lab folder and confirm your identity and Region before you touch anything.

```
aws sts get-caller-identity --profile devops
```

Expected: Your own Account, UserId and Arn are printed, confirming you are operating in the right account and Region.

144. List the default VPC subnets and pick two in two different Availability Zones for the ALB and Auto Scaling group.

```
aws ec2 describe-subnets --filters Name=default-for-
az,Values=true --query 'Subnets[].
{Az:AvailabilityZone,Id:SubnetId}' --profile devops
```

Expected: At least two subnets are listed, each in a distinct AZ (e.g. ap-southeast-1a and ap-southeast-1b).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

145. Validate the high-availability template before deploying so you catch errors locally.

```
aws cloudformation validate-template --template-body file://ha-asg.yaml --profile devops
```

Expected: The command returns the template Description and its Parameters (VpcId, SubnetAId, SubnetBId) with no error.

146. Deploy the stack, passing the VPC and the two AZ subnet IDs you selected.

```
aws cloudformation deploy --template-file ha-asg.yaml --stack-name ha-web --parameter-overrides VpcId=<vpc-id> SubnetAId=<subnet-a> SubnetBId=<subnet-b> --capabilities CAPABILITY_IAM --profile devops
```

Expected: CloudFormation reports the stack ha-web reaching CREATE_COMPLETE.

147. Read the stack Outputs and capture the load balancer DNS name and the Auto Scaling group name.

```
aws cloudformation describe-stacks --stack-name ha-web --query 'Stacks[0].Outputs' --profile devops
```

Expected: Outputs AlbDnsName (an *.elb.amazonaws.com name) and AsgName (devops-ha-asg) are returned.

148. Confirm the Auto Scaling group is configured for both AZs and a minimum of two instances.

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names devops-ha-asg --query 'AutoScalingGroups[0].{Min:MinSize,Desired:DesiredCapacity,Max:MaxSize,AZs:AvailabilityZones}' --profile devops
```

Expected: Min=2, Desired=2, Max=6 and AvailabilityZones lists the two AZs you chose.

149. Confirm the launch template exists and defines the instance type and AMI.

```
aws ec2 describe-launch-templates --query "LaunchTemplates[?LaunchTemplateName=='devops-ha-lt'].LaunchTemplateId" --profile devops
```

Expected: A launch template ID (lt-...) is returned for devops-ha-lt.

150. Wait until both instances are InService and healthy behind the target group.

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names devops-ha-asg --query
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
'AutoScalingGroups[0].Instances[].
{Id:InstanceId,Az:AvailabilityZone,State:LifecycleState,Health:He
althStatus}' --profile devops
```

Expected: Two instances are listed, one per AZ, each with LifecycleState=InService and HealthStatus=Healthy.

151. Check the target group health directly from the load balancer's point of view.

```
aws elbv2 describe-target-health --target-group-arn <target-
group-arn> --query 'TargetHealthDescriptions[].
{Id:Target.Id,State:TargetHealth.State}' --profile devops
```

Expected: Both registered targets report TargetHealth.State=healthy.

152. Curl the load balancer once and confirm it returns HTTP 200 from a web instance.

```
curl -fsS http://<alb-dns-name>/
```

Expected: The response body is 'OK from ip-10-...' (the private hostname of whichever instance served the request).

153. Hit the load balancer 20 times in a row and count how many distinct instances answered — the proof of load balancing.

```
for i in $(seq 20); do curl -s http://<alb-dns-name>/; echo; done
| sort | uniq -c
```

Expected: The output shows two distinct 'OK from ip-...' lines with counts that add up to 20, proving requests hit instances in BOTH AZs.

154. Confirm the health-check settings on the target group so you understand what marks an instance unhealthy.

```
aws elbv2 describe-target-groups --names devops-ha-tg --query
'TargetGroups[0].
{Path:HealthCheckPath,Interval:HealthCheckIntervalSeconds,Healthy
:HealthyThresholdCount,Unhealthy:UnhealthyThresholdCount}' --
profile devops
```

Expected: The target group shows HealthCheckPath=/ , a 15s interval and healthy/unhealthy thresholds (2 and 3).

155. Confirm the Auto Scaling group uses the ELB health-check type so ALB health drives replacement, not just EC2 status.

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-
group-names devops-ha-asg --query 'AutoScalingGroups[0].
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
{HealthCheckType:HealthCheckType,Grace:HealthCheckGracePeriod}'
--profile devops
```

Expected: HealthCheckType=ELB and HealthCheckGracePeriod is 60, confirming failed health checks trigger replacement.

156. Manually raise DesiredCapacity to 3 and watch a third instance launch into whichever AZ is under-populated.

```
aws autoscaling set-desired-capacity --auto-scaling-group-name
devops-ha-asg --desired-capacity 3 --profile devops
```

Expected: A third instance launches; describe-auto-scaling-groups shows three InService instances still balanced across the two AZs.

157. Return DesiredCapacity to 2 so the lab stays near-free ahead of the scaling exercise in Lab 12.

```
aws autoscaling set-desired-capacity --auto-scaling-group-name
devops-ha-asg --desired-capacity 2 --profile devops
```

Expected: The group scales back to two InService instances, one in each AZ.

Validation

In the EC2 console the Auto Scaling group devops-ha-asg shows two (or three) InService instances split across two AZs, the target group shows all targets healthy, and the Load Balancers view shows the ALB active in both AZs. From the CLI, curling the ALB DNS name repeatedly returns responses from more than one instance and HealthCheckType is ELB.

Test it

Curling the ALB DNS name 20 times returns responses from two distinct instances located in two different Availability Zones, target health is healthy for both, and raising then lowering DesiredCapacity adds and removes an instance while keeping both AZs populated — proving the tier is genuinely multi-AZ and self-balancing.

Note: Cost & safety: Two t3.micro instances plus one Application Load Balancer run for the duration of the lab; the ALB is billed per hour plus a small LCU charge and the instances per second. Keeping DesiredCapacity at 2 and deleting the stack the same day keeps this to roughly US$1–2. Delete the stack when finished. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-11-multiaz-alb-autoscaling/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 12 — Attach Target-Tracking Auto Scaling and Validate Scale-Out Under Load

Learning outcome: LO3. Skills mapped: K2, K4, A2. Objective: Attach a target-tracking scaling policy to an Auto Scaling group and validate automatic scale-out under generated load and scale-in when the load stops (LO3, K2, K4, A2).

Goal: A static two-instance web tier survives an AZ failure but not a traffic spike. In this lab you attach a target-tracking scaling policy that keeps average CPU at 50% to the Auto Scaling group you built in Lab 11, generate CPU load on the instances, and watch CloudWatch alarms trip and the group scale out. When the load stops, you confirm the group scales back in to its minimum — proving the tier grows and shrinks with demand.

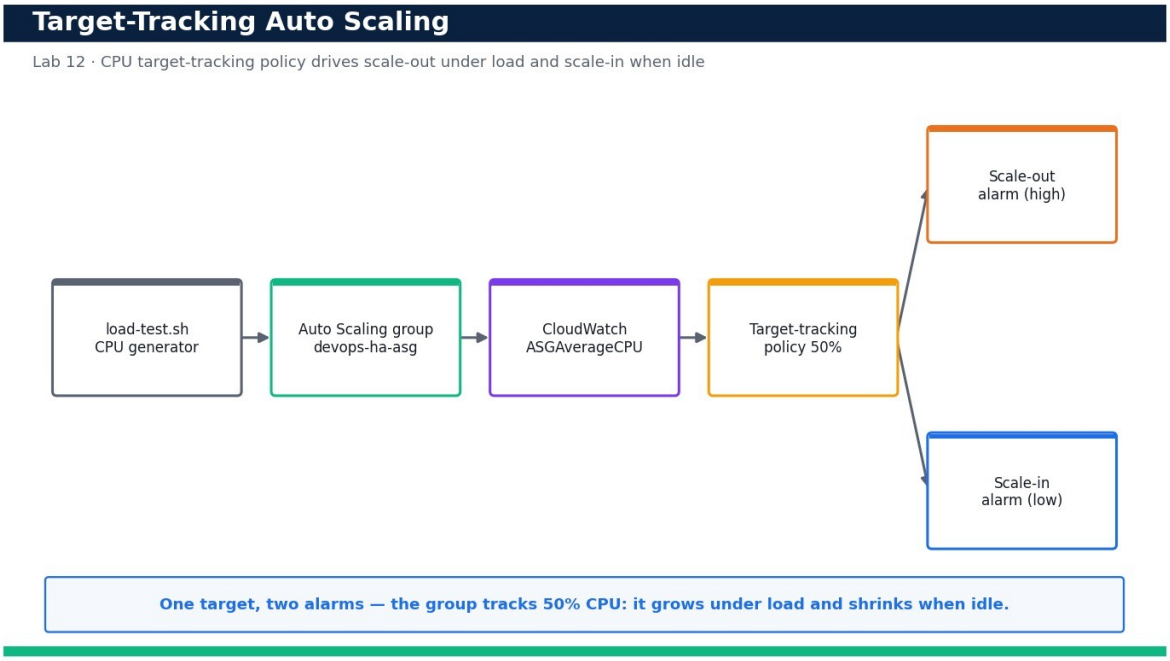


Figure — Lab 12 reference architecture: Attach Target-Tracking Auto Scaling and Validate Scale-Out Under Load.

What you'll build

A target-tracking scaling policy (50% average CPU) on devops-ha-asg with its two auto-created CloudWatch alarms, plus a repeatable load test that drives observable scale-out and scale-in. (Services: AWS Auto Scaling, Amazon CloudWatch, Amazon EC2.)

Step-by-step

158. Confirm the Lab 11 Auto Scaling group is present and currently at its minimum of two instances.

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-
group-names devops-ha-asg --query 'AutoScalingGroups[0].
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
{Desired:DesiredCapacity,Min:MinSize,Max:MaxSize}' --profile
devops
```

Expected: The group reports Desired=2, Min=2, Max=6.

159. Review the scaling-policy.json document so you understand the target metric and value before applying it.

```
cat scaling-policy.json
```

Expected: The file defines PolicyType TargetTrackingScaling, PredefinedMetricType ASGAverageCPUUtilization and TargetValue 50.0.

160. Attach the target-tracking scaling policy with the helper script (it wraps put-scaling-policy).

```
bash put-scaling-policy.sh
```

Expected: put-scaling-policy returns a PolicyARN and Alarms — a high (scale-out) and a low (scale-in) CloudWatch alarm auto-created for the policy.

161. Confirm the policy is attached to the group.

```
aws autoscaling describe-policies --auto-scaling-group-name
devops-ha-asg --query 'ScalingPolicies[].
{Name:PolicyName,Type:PolicyType,Target:TargetTrackingConfigurati
on.TargetValue}' --profile devops
```

Expected: The policy cpu-target-tracking-50 is listed with PolicyType TargetTrackingScaling and Target 50.0.

162. List the CloudWatch alarms the policy created so you can watch them trip.

```
aws cloudwatch describe-alarms --alarm-name-prefix
TargetTracking-devops-ha-asg --query 'MetricAlarms[].
{Name:AlarmName,State:StateValue,Threshold:Threshold}' --profile
devops
```

Expected: Two alarms are listed (an AlarmHigh and an AlarmLow), both currently in state OK or INSUFFICIENT_DATA.

163. Record the baseline average CPU across the group before generating load.

```
aws cloudwatch get-metric-statistics --namespace AWS/EC2 --
metric-name CPUUtilization --dimensions
Name=AutoScalingGroupName,Value=devops-ha-asg --statistics
Average --period 60 --start-time $(date -u -v-10M +%FT%TZ) --end-
time $(date -u +%FT%TZ) --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: Datapoints show a low Average CPU (typically under 10%), establishing the idle baseline.

164. Connect to an instance via SSM Session Manager so you can drive CPU load from inside the group.

```
aws ssm start-session --target <instance-id> --profile devops
```

Expected: An interactive Session Manager shell opens on the instance without needing SSH or a key pair.

165. Generate sustained CPU load on the group using Run Command so every instance is stressed at once.

```
aws ssm send-command --document-name AWS-RunShellScript --targets
Key=tag:aws:autoscaling:groupName,Values=devops-ha-asg --
parameters commands='for c in 1 2 3 4; do (yes > /dev/null &) ;
done; sleep 600; pkill yes' --profile devops
```

Expected: A CommandId is returned; the `yes` loops peg CPU on the instances for ~10 minutes to force scale-out.

166. Watch the scale-out alarm move into ALARM as average CPU crosses 50%.

```
aws cloudwatch describe-alarms --alarm-name-prefix
TargetTracking-devops-ha-asg-AlarmHigh --query
'MetricAlarms[0].StateValue' --profile devops
```

Expected: Within a few minutes the AlarmHigh StateValue transitions from OK to "ALARM".

167. Confirm the Auto Scaling group increased DesiredCapacity in response.

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-
group-names devops-ha-asg --query 'AutoScalingGroups[0].
{Desired:DesiredCapacity,Instances:length(Instances)}' --profile
devops
```

Expected: DesiredCapacity has risen above 2 (e.g. 3 or 4) and additional instances are launching to bring CPU back toward 50%.

168. Review the scaling activity log to see the human-readable reason the group scaled out.

```
aws autoscaling describe-scaling-activities --auto-scaling-group-
name devops-ha-asg --max-items 5 --query 'Activities[].
{Cause:Cause,Status:StatusCode}' --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: A recent activity shows a successful launch with a Cause referencing the target-tracking policy monitor alarm in ALARM.

169. Stop the load and confirm CPU falls back below the target.

```
aws ssm send-command --document-name AWS-RunShellScript --targets
Key=tag:aws:autoscaling:groupName,Values=devops-ha-asg --
parameters commands='pkill yes || true' --profile devops
```

Expected: A CommandId is returned; average CPU begins dropping back toward the idle baseline over the next few minutes.

170. Watch the scale-in (low) alarm trip and the group begin removing instances.

```
aws cloudwatch describe-alarms --alarm-name-prefix
TargetTracking-devops-ha-asg-AlarmLow --query
'MetricAlarms[0].StateValue' --profile devops
```

Expected: The AlarmLow StateValue transitions to "ALARM", signalling the policy to reduce capacity.

171. Confirm the group scaled back in toward its minimum of two.

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-
group-names devops-ha-asg --query
'AutoScalingGroups[0].DesiredCapacity' --profile devops
```

Expected: DesiredCapacity returns to 2 (respecting MinSize) once CPU has stayed below the target through the cooldown.

172. Confirm the service stayed available throughout by curling the load balancer during the exercise.

```
curl -fsS http://<alb-dns-name>/
```

Expected: The ALB continues to return HTTP 200 'OK from ip-...' throughout scale-out and scale-in, proving no downtime.

Validation

describe-policies shows cpu-target-tracking-50 attached to devops-ha-asg, describe-alarms shows the auto-created AlarmHigh and AlarmLow, and describe-scaling-activities records a scale-out caused by the policy under load and a later scale-in when the load stopped. DesiredCapacity is observed rising above 2 and returning to 2.

Test it

Under generated CPU load the AlarmHigh trips, the group scales out above two instances, and when the load stops the AlarmLow trips and the group scales back in to

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

its minimum — all while the ALB keeps returning HTTP 200 — proving demand-driven, zero-downtime elasticity.

Note: Cost & safety: Scaling out temporarily adds t3.micro instances (billed per second) and creates two CloudWatch alarms (a few cents/month). Keeping MaxSize modest, running the load test only briefly and letting the group scale back in keeps this well under US$1. The alarms are removed with the policy at cleanup. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-12-target-tracking-load-validation/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 13 — Select a DR Strategy by RTO/RPO and Implement a Pilot-Light Skeleton

Learning outcome: LO3. Skills mapped: K2, K4, A2. Objective: Compare the four AWS disaster-recovery strategies against RTO/RPO targets, select an appropriate pattern, and implement a pilot-light skeleton in a second Region (LO3, K2, K4, A2).

Goal: Not every workload needs active-active — and not every business can accept a day-long restore. In this lab you complete an RTO/RPO worksheet, compare the four AWS DR strategies (backup & restore, pilot light, warm standby, active-active) against those targets, and then stand up a pilot-light skeleton in a second Region: a minimal network and a scaled-to-zero application tier kept warm by cross-Region data replication, with a Route 53 health check and failover record ready to promote it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

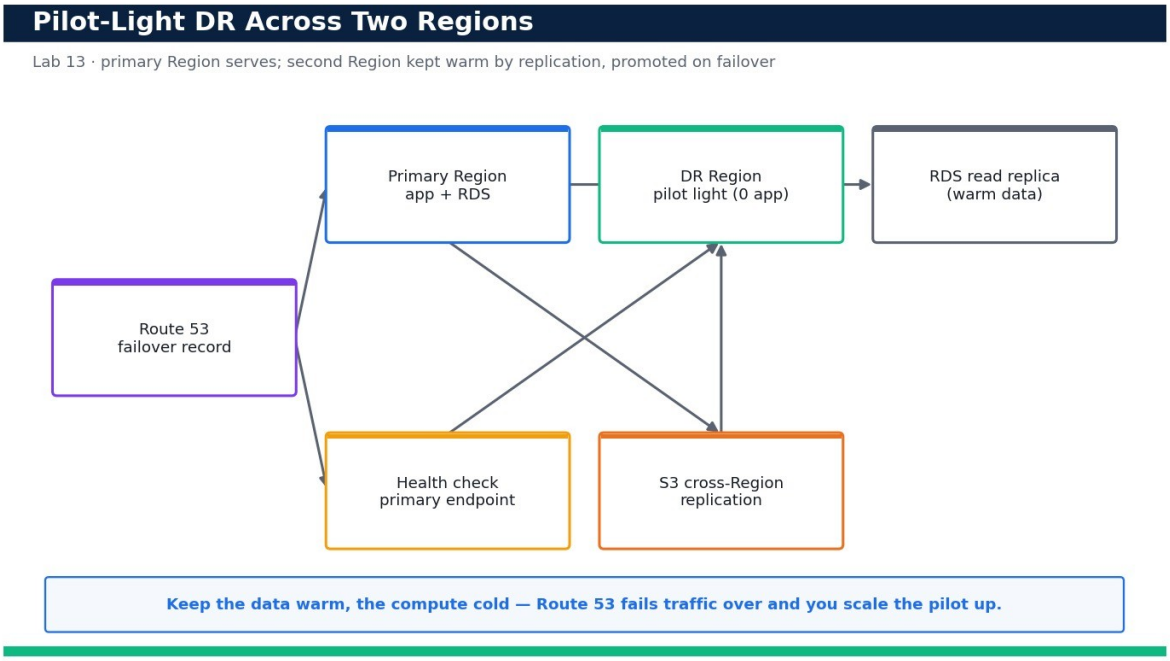


Figure — Lab 13 reference architecture: Select a DR Strategy by RTO/RPO and Implement a Pilot-Light Skeleton.

What you'll build

A completed DR selection worksheet plus a deployed pilot-light skeleton in a second Region (minimal VPC, a scaled-to-zero app tier and an S3 cross-Region replication path) fronted by a Route 53 failover record set. (Services: Amazon Route 53, Amazon RDS, AWS CloudFormation, Amazon S3 (Cross-Region Replication).)

Step-by-step

173. Open the DR worksheet and record the workload's target RTO and RPO before choosing anything.

```
cat dr-plan.md
```

Expected: The worksheet shows the four strategies with typical RTO/RPO ranges and blank fields for your workload's targets and chosen pattern.

174. Confirm your primary Region and pick a distinct DR Region for the second copy.

```
aws configure get region --profile devops
```

Expected: Your primary Region prints (e.g. ap-southeast-1); you choose a different DR Region such as ap-southeast-2.

175. Validate the pilot-light template that will run in the DR Region.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws cloudformation validate-template --template-body
file://pilot-light.yaml --profile devops
```

Expected: The command returns the template Description and Parameters (EnvName, DrVpcCidr) with no error.

176. Create an S3 source bucket in the primary Region with versioning enabled (required for replication).

```
aws s3api create-bucket --bucket devops-dr-src-<account-id> --
region ap-southeast-1 --create-bucket-configuration
LocationConstraint=ap-southeast-1 --profile devops && aws s3api
put-bucket-versioning --bucket devops-dr-src-<account-id> --
versioning-configuration Status=Enabled --profile devops
```

Expected: The source bucket is created and versioning reports Status=Enabled.

177. Create the destination bucket in the DR Region, also versioned, and block public access.

```
aws s3api create-bucket --bucket devops-dr-dst-<account-id> --
region ap-southeast-2 --create-bucket-configuration
LocationConstraint=ap-southeast-2 --profile devops && aws s3api
put-public-access-block --bucket devops-dr-dst-<account-id> --
public-access-block-configuration
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true
,RestrictPublicBuckets=true --profile devops
```

Expected: The destination bucket exists in the DR Region with Block Public Access fully enabled.

178. Apply the cross-Region replication configuration so new objects flow to the DR Region automatically.

```
aws s3api put-bucket-replication --bucket devops-dr-src-<account-
id> --replication-configuration file://s3-crr.json --profile
devops
```

Expected: The command returns with no error; the source bucket now has a replication rule targeting the DR bucket.

179. Prove replication works by uploading a test object and checking it appears in the DR bucket.

```
aws s3 cp dr-plan.md s3://devops-dr-src-<account-id>/dr-plan.md
--profile devops && sleep 30 && aws s3 ls s3://devops-dr-dst-
<account-id>/ --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: After a short delay dr-plan.md is listed in the destination bucket, confirming cross-Region replication (RPO for object data).

180. Deploy the pilot-light skeleton stack in the DR Region (minimal VPC + a scaled-to-zero Auto Scaling group).

```
aws cloudformation deploy --template-file pilot-light.yaml --
stack-name dr-pilot --region ap-southeast-2 --parameter-overrides
EnvName=dr --capabilities CAPABILITY_IAM --profile devops
```

Expected: CloudFormation reports stack dr-pilot reaching CREATE_COMPLETE in the DR Region.

181. Confirm the DR app tier is present but running zero instances (pilot light = warm data, cold compute).

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-
group-names dr-pilot-asg --region ap-southeast-2 --query
'AutoScalingGroups[0].{Min:MinSize,Desired:DesiredCapacity}' --
profile devops
```

Expected: The DR Auto Scaling group shows MinSize=0 and DesiredCapacity=0 — no compute cost while idle.

182. Create a Route 53 health check against the primary endpoint (an HTTP check on / every 30s) that will drive failover.

```
aws route53 create-health-check --caller-reference dr-$(date +%s)
--health-check-config
Type=HTTP,ResourcePath=/,FullyQualifiedDomainName=<primary-alb-
dns-name>,Port=80,RequestInterval=30,FailureThreshold=3 --profile
devops
```

Expected: A HealthCheck Id is returned; it monitors the primary endpoint over HTTP.

183. Create the primary and secondary failover record sets in your hosted zone.

```
aws route53 change-resource-record-sets --hosted-zone-id <hosted-
zone-id> --change-batch file://failover-records.json --profile
devops
```

Expected: A ChangeInfo with Status PENDING is returned; the zone now has a PRIMARY (health-checked) and a SECONDARY failover record.

184. Read failover-notes.md and rehearse the promotion runbook so the team knows exactly what to do on failover.

```
cat failover-notes.md
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: The runbook lists the ordered steps: promote/restore data, scale the DR Auto Scaling group up from 0, verify health, and confirm Route 53 has shifted traffic.

185. Rehearse promoting the pilot light: scale the DR Auto Scaling group up from zero to serve traffic.

```
aws autoscaling update-auto-scaling-group --auto-scaling-group-name dr-pilot-asg --min-size 2 --desired-capacity 2 --region ap-southeast-2 --profile devops
```

Expected: The DR group scales from 0 to 2 InService instances, demonstrating the pilot light can be brought to full capacity.

186. Scale the DR tier back to zero to return to the cheap pilot-light steady state after the rehearsal.

```
aws autoscaling update-auto-scaling-group --auto-scaling-group-name dr-pilot-asg --min-size 0 --desired-capacity 0 --region ap-southeast-2 --profile devops
```

Expected: The DR group returns to 0 instances, confirming the pilot light is warm-data / cold-compute again.

Validation

The dr-plan.md worksheet records the workload RTO/RPO and a justified strategy choice; the S3 source bucket replicates objects to the DR Region; the dr-pilot stack is CREATE_COMPLETE with a scaled-to-zero Auto Scaling group; and Route 53 has a PRIMARY (health-checked) plus SECONDARY failover record. The rehearsal shows the DR group can scale from 0 to 2 and back.

Test it

A test object uploaded to the primary bucket appears in the DR Region bucket (data is warm), and the DR Auto Scaling group can be promoted from 0 to 2 instances and returned to 0 — proving the pilot-light pattern meets the chosen RTO/RPO at minimal steady-state cost.

Note: Cost & safety: Pilot light is deliberately cheap: the DR Region runs only a minimal network, replicated S3 objects and a small RDS instance/replica — no running app fleet. Expect low single-digit US$ for the duration of the lab; the RDS instance and cross-Region replication are the only meaningful line items. Delete both stacks after. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-13-rto-rpo-dr-selection/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 14 — Automate Backup, Restore and Data Protection with AWS Backup

Learning outcome: LO3. Skills mapped: K2, K4, A2. Objective: Automate centralized backups with AWS Backup (plan + tag-based selection), test a restore, and add S3 versioning with cross-Region replication and RDS automated snapshots for defense in depth (LO3, K2, K4, A2).

Goal: Resilience is not only about staying up — it is about getting your data back. In this lab you build a centralized AWS Backup plan with a scheduled rule and a tag-based resource selection so every tagged EBS volume and RDS database is backed up automatically. You then run an on-demand backup, restore from a recovery point to prove the backup is usable, and layer in S3 versioning + cross-Region replication and RDS automated snapshots so data is protected at multiple levels.

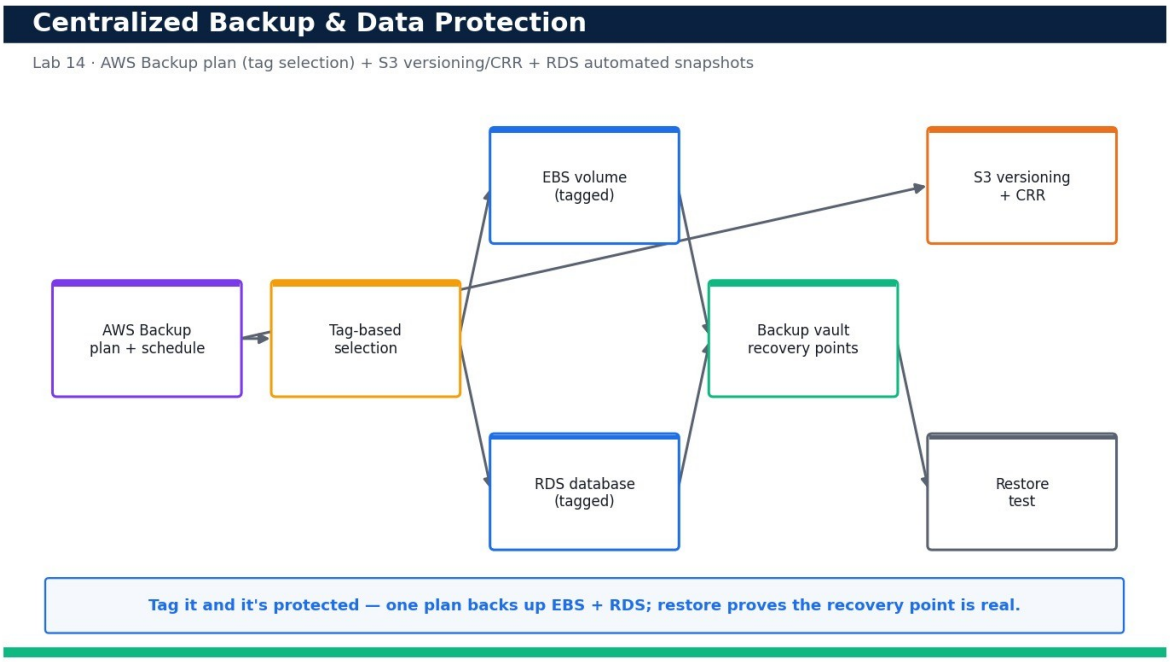


Figure — Lab 14 reference architecture: Automate Backup, Restore and Data Protection with AWS Backup.

What you'll build

An AWS Backup plan with a tag-based selection protecting EBS and RDS, a verified restore from a recovery point, plus S3 versioning + cross-Region replication and RDS automated snapshots enabled. (Services: AWS Backup, Amazon RDS, Amazon S3, Amazon EBS.)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

187. Tag the EBS volume and RDS instance you want protected so the plan's selection can find them.

```
aws ec2 create-tags --resources <volume-id> --tags
Key=backup,Value=daily --profile devops && aws rds add-tags-to-
resource --resource-name <rds-arn> --tags Key=backup,Value=daily
--profile devops
```

Expected: Both resources now carry the tag backup=daily; no error is returned.

188. Create a dedicated backup vault to hold the recovery points.

```
aws backup create-backup-vault --backup-vault-name devops-vault
--profile devops
```

Expected: The vault devops-vault is created and its BackupVaultArn is returned.

189. Review the backup-plan.yaml so you understand the schedule, lifecycle and tag selection before deploying.

```
cat backup-plan.yaml
```

Expected: The template defines a plan with a daily rule (35-day retention), a tag-based selection on backup=daily and the AWS Backup service role.

190. Deploy the backup plan and its tag-based selection with CloudFormation.

```
aws cloudformation deploy --template-file backup-plan.yaml --
stack-name devops-backup --capabilities CAPABILITY_NAMED_IAM --
profile devops
```

Expected: Stack devops-backup reaches CREATE_COMPLETE and outputs the BackupPlanId, SelectionId and BackupRoleArn.

191. Confirm the plan and its rule are in place.

```
aws backup get-backup-plan --backup-plan-id <backup-plan-id> --
query 'BackupPlan.{Name:BackupPlanName,Rules:Rules[].
{Rule:RuleName,Schedule:ScheduleExpression}}' --profile devops
```

Expected: The plan devops-daily-plan is returned with a rule whose ScheduleExpression is a daily cron.

192. Confirm the tag-based resource selection is attached to the plan.

```
aws backup list-backup-selections --backup-plan-id <backup-plan-
id> --query 'BackupSelectionsList[].SelectionName' --profile
devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: The selection tag-backup-daily is listed under the plan.

193. Start an on-demand backup of the EBS volume so you have a recovery point without waiting for the schedule.

```
aws backup start-backup-job --backup-vault-name devops-vault --
resource-arn <volume-arn> --iam-role-arn <backup-role-arn> --
profile devops
```

Expected: A BackupJobId is returned; describe-backup-job will show it moving from RUNNING to COMPLETED.

194. Wait for the backup job to complete and note the recovery point.

```
aws backup list-recovery-points-by-backup-vault --backup-vault-
name devops-vault --query 'RecoveryPoints[].
{Arn:RecoveryPointArn,Status:Status}' --profile devops
```

Expected: At least one recovery point is listed with Status=COMPLETED.

195. Restore the recovery point to prove the backup is actually usable (the real test of a backup).

```
aws backup start-restore-job --recovery-point-arn <recovery-
point-arn> --iam-role-arn <backup-role-arn> --metadata
file://restore-metadata.json --profile devops
```

Expected: A RestoreJobId is returned; list-restore-jobs shows it reaching COMPLETED and creating a new volume.

196. Verify the restore produced a usable resource (a new EBS volume in the correct AZ).

```
aws backup describe-restore-job --restore-job-id <restore-job-id>
--query '{Status:Status,Created:CreatedResourceArn}' --profile
devops
```

Expected: Status=COMPLETED and CreatedResourceArn points at a new EBS volume, proving the recovery point restores cleanly.

197. Enable versioning on the application S3 bucket so overwritten/deleted objects remain recoverable.

```
aws s3api put-bucket-versioning --bucket devops-app-data-
<account-id> --versioning-configuration Status=Enabled --profile
devops
```

Expected: Versioning reports Status=Enabled; overwriting an object now keeps prior versions.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

198. Add cross-Region replication to the versioned bucket for off-Region data protection.

```
aws s3api put-bucket-replication --bucket devops-app-data-
<account-id> --replication-configuration file://s3-
replication.json --profile devops
```

Expected: The command returns with no error; new objects now replicate to the destination bucket in the DR Region.

199. Confirm RDS automated backups (snapshots) are enabled with a sensible retention window.

```
aws rds modify-db-instance --db-instance-identifier devops-db --
backup-retention-period 7 --apply-immediately --profile devops
```

Expected: The instance shows BackupRetentionPeriod=7, so RDS takes daily automated snapshots kept for 7 days.

200. Verify RDS automated snapshots exist for the database.

```
aws rds describe-db-snapshots --db-instance-identifier devops-db
--snapshot-type automated --query 'DBSnapshots[].
{Id:DBSnapshotIdentifier,Status:Status}' --profile devops
```

Expected: One or more automated snapshots are listed with Status=available.

201. Confirm object-level protection end to end: overwrite an S3 object and list its versions.

```
aws s3 cp restore.sh s3://devops-app-data-<account-id>/restore.sh
--profile devops && aws s3api list-object-versions --bucket
devops-app-data-<account-id> --prefix restore.sh --query
'Versions[].{Id:VersionId,Latest:IsLatest}' --profile devops
```

Expected: Two or more versions of restore.sh are listed, confirming versioning retains the previous copy for recovery.

Validation

AWS Backup shows the devops-daily-plan with a tag-based selection, at least one COMPLETED recovery point in devops-vault, and a COMPLETED restore job that created a new resource. S3 versioning is Enabled with a replication rule, and RDS reports a 7-day backup retention with automated snapshots available.

Test it

An on-demand backup produces a COMPLETED recovery point that restores cleanly into a new EBS volume, S3 retains multiple object versions and replicates them cross-

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Region, and RDS holds available automated snapshots — proving data is protected and, crucially, recoverable at multiple layers.

Note: Cost & safety: Backup storage in the vault, RDS snapshot storage and S3 replication all bill on stored GB but a tiny test database/volume keeps this to a few cents. The main safety note: delete recovery points and snapshots at cleanup or they continue to accrue storage charges. Keep test data small. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-14-backup-restore-data-protection/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 15 — Inject Failure and Verify Self-Healing with AWS FIS

Learning outcome: LO3. Skills mapped: K2, K4, A2. Objective: Prove the multi-AZ tier self-heals by terminating an instance and watching the Auto Scaling group replace it, then run an AWS FIS experiment and confirm the ALB routes around the impaired instances (LO3, K2, K4, A2).

Goal: A resilient design is only proven when you break it on purpose. In this lab you first terminate an instance manually and watch the Auto Scaling group launch a replacement to restore capacity. You then run a controlled AWS Fault Injection Service (FIS) experiment that stops instances and confirm the Application Load Balancer keeps serving traffic by routing only to healthy targets — turning chaos engineering into evidence that the tier from Lab 11 genuinely self-heals.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

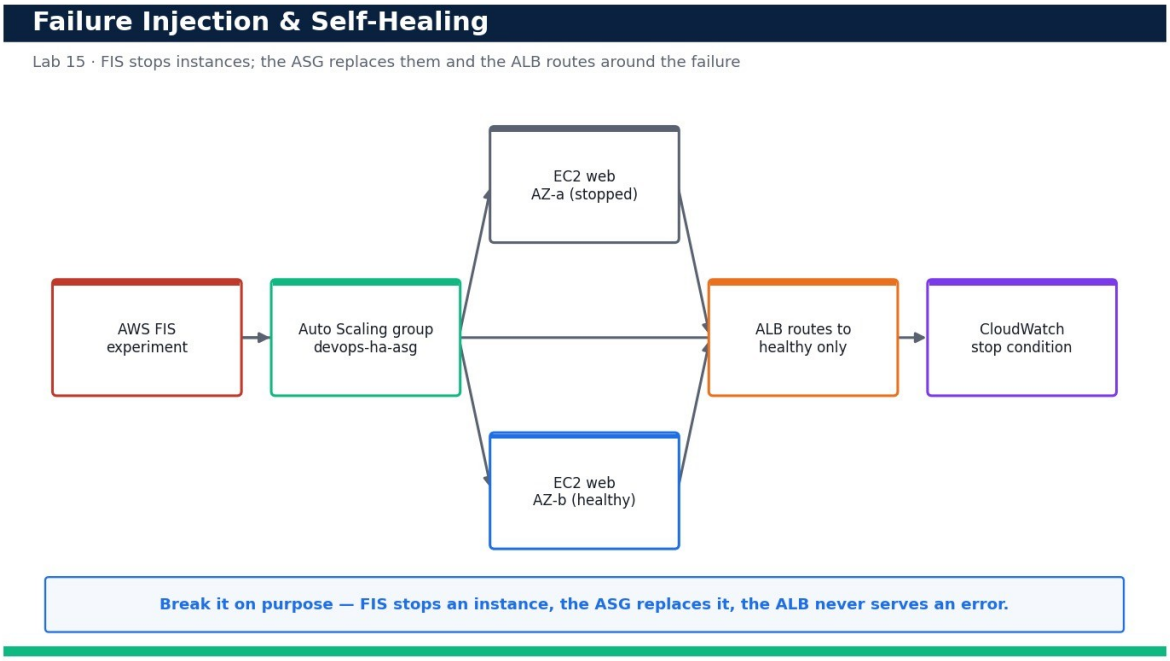


Figure — Lab 15 reference architecture: Inject Failure and Verify Self-Healing with AWS FIS.

What you'll build

A verified self-healing web tier: a manual instance termination is auto-replaced by the Auto Scaling group, and an AWS FIS experiment stops instances while the ALB continues to serve traffic with zero errors. (Services: AWS Fault Injection Service (FIS), AWS Auto Scaling, Amazon EC2, Amazon CloudWatch.)

Step-by-step

202. Confirm the Lab 11 tier is healthy with two InService instances before you break anything.

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-
group-names devops-ha-asg --query
'AutoScalingGroups[0].Instances[].
{Id:InstanceId,Az:AvailabilityZone,State:LifecycleState}' --
profile devops
```

Expected: Two instances are listed, one per AZ, both LifecycleState=InService.

203. Establish a steady-state baseline: the ALB returns HTTP 200 right now.

```
curl -fsS http://<alb-dns-name>/
```

Expected: The response is HTTP 200 with body 'OK from ip-...', confirming the tier is serving before the experiment.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

204. Manually terminate ONE instance to test the most basic self-healing behaviour of the group.

```
aws autoscaling terminate-instance-in-auto-scaling-group --
instance-id <instance-id> --no-should-decrement-desired-capacity
--profile devops
```

Expected: The instance enters Terminating and a scaling activity begins launching a replacement (desired capacity is preserved).

205. Watch the Auto Scaling group launch a replacement to restore capacity.

```
aws autoscaling describe-scaling-activities --auto-scaling-group-
name devops-ha-asg --max-items 3 --query 'Activities[].
{Desc:Description,Status:StatusCode}' --profile devops
```

Expected: A recent activity shows 'Launching a new EC2 instance' with StatusCode Successful, replacing the terminated one.

206. Confirm the group is back to two InService instances across both AZs.

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-
group-names devops-ha-asg --query 'AutoScalingGroups[0].
{Desired:DesiredCapacity,InService:length(Instances[?
LifecycleState==`InService`])}' --profile devops
```

Expected: DesiredCapacity=2 and two instances are InService again, proving the group self-heals after a termination.

207. Create an IAM role for FIS to assume so it can act on your instances safely.

```
aws iam create-role --role-name devops-fis-role --assume-role-
policy-document file://fis-trust-policy.json --profile devops
```

Expected: The role devops-fis-role is created and its Arn is returned.

208. Attach the FIS-required permissions so the experiment can stop EC2 instances.

```
aws iam attach-role-policy --role-name devops-fis-role --policy-
arn
arn:aws:iam::aws:policy/service-role/AWSFaultInjectionSimulatorEC
2Access --profile devops
```

Expected: The command returns with no error; the FIS role can now perform EC2 stop/start actions.

209. Review the FIS experiment template so you understand the target, action and stop condition before running it.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cat fis-experiment.json
```

Expected: The template targets one instance in the group, uses the aws:ec2:stop-instances action, and has a CloudWatch alarm stop condition to abort automatically.

210. Create the FIS experiment template.

```
aws fis create-experiment-template --cli-input-json file://fis-experiment.json --profile devops
```

Expected: An experiment template id (EXT...) is returned.

211. Start the experiment to stop an instance and begin observing how the tier responds.

```
aws fis start-experiment --experiment-template-id <experiment-template-id> --profile devops
```

Expected: An experiment id (EXP...) is returned with state initiating, then running.

212. While the experiment runs, prove the ALB routes AROUND the impaired instance with no user-visible errors.

```
for i in $(seq 30); do curl -s -o /dev/null -w '%{http_code}\n' http://<alb-dns-name>/; sleep 1; done | sort | uniq -c
```

Expected: Every (or nearly every) request returns 200 — the ALB stopped sending traffic to the stopped/unhealthy target and served from the healthy AZ.

213. Watch the target group react to the stopped instance.

```
aws elbv2 describe-target-health --target-group-arn <target-group-arn> --query 'TargetHealthDescriptions[].{Id:Target.Id,State:TargetHealth.State}' --profile devops
```

Expected: The stopped instance shows unhealthy/unused while the remaining/replacement target shows healthy, confirming traffic is only routed to healthy instances.

214. Confirm the FIS experiment reached a terminal state (completed, or stopped by its safety alarm).

```
aws fis get-experiment --id <experiment-id> --query '{State:state.status,Reason:state.reason}' --profile devops
```

Expected: The experiment State is completed (or stopped if the CloudWatch stop condition fired), demonstrating a controlled blast radius.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

215. Run the verify-recovery.sh script to confirm the tier is fully recovered after the experiment.

```
ALB_DNS=<alb-dns-name> ASG_NAME=devops-ha-asg bash verify-recovery.sh
```

Expected: The script reports two InService instances, all targets healthy, and 30/30 HTTP 200 responses — a clean recovery.

216. Confirm the whole exercise caused no sustained availability loss by checking CloudWatch for ALB 5XX errors during the window.

```
aws cloudwatch get-metric-statistics --namespace AWS/ApplicationELB --metric-name HTTPCode_ELB_5XX_Count --dimensions Name=LoadBalancer,Value=<alb-arn-suffix> --statistics Sum --period 60 --start-time $(date -u -v-15M +%FT%TZ) --end-time $(date -u +%FT%TZ) --profile devops
```

Expected: The Sum of ELB 5XX errors is 0 (or negligible) across the experiment window, proving the failure was absorbed without downtime.

Validation

A manually terminated instance is automatically replaced (describe-scaling-activities shows a successful launch) and the group returns to two InService instances. The FIS experiment reaches a terminal state with a CloudWatch stop condition configured, target health shows only healthy instances receiving traffic, and ALB 5XX count stays at ~0 throughout.

Test it

Terminating an instance and running a FIS stop-instances experiment both leave the ALB serving HTTP 200 with zero sustained 5XX errors while the Auto Scaling group launches replacements and the load balancer routes around the impaired targets — proving the tier self-heals under real injected failure.

Note: Cost & safety: FIS bills per action-minute (cents) and the replacement instances are ordinary t3.micro time. A short experiment with a CloudWatch stop condition keeps this to well under US$1. The stop condition is the key safety control — it aborts the experiment automatically if things go wrong. Clean up the templates after. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-15-failure-injection-self-healing/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Domain 04 — Monitoring and Logging (15%)

Metrics · logs · dashboards · alarms · automated event management

Key concepts

- CloudWatch metrics — Namespaces, dimensions and custom metrics measure system and application health over time.
- CloudWatch Logs — Log groups, streams, subscription filters and metric filters turn raw logs into searchable, alertable signals.
- Dashboards — A single pane of glass correlates metrics, logs and alarms for fast troubleshooting.
- Alarms & composite alarms — Thresholds and anomaly detection notify SNS or trigger auto-remediation before users are affected.
- Tracing & insights — X-Ray traces distributed requests; CloudWatch Logs Insights and Container/Lambda Insights find the root cause.
- Aggregation at scale — Cross-account observability, the CloudWatch agent and OpenTelemetry centralise telemetry from many sources.

Lab 16 — Collect Custom Metrics with the CloudWatch Agent and Build an Operations Dashboard

Learning outcome: LO4. Skills mapped: K5, A6. Objective: Instrument an EC2 workload with memory/disk and application metrics, then visualise CPU, latency, errors and request count on a CloudWatch dashboard (LO4, K5, A6).

Goal: Your team runs a web app on EC2 but only has the default hypervisor CPU metric — you are blind to memory pressure, disk usage and application latency. In this lab you install and configure the CloudWatch agent to publish memory and disk custom metrics, push an application latency/request metric with put-metric-data, then assemble a single operations dashboard that shows CPU, p90 latency, error count and request count side by side.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

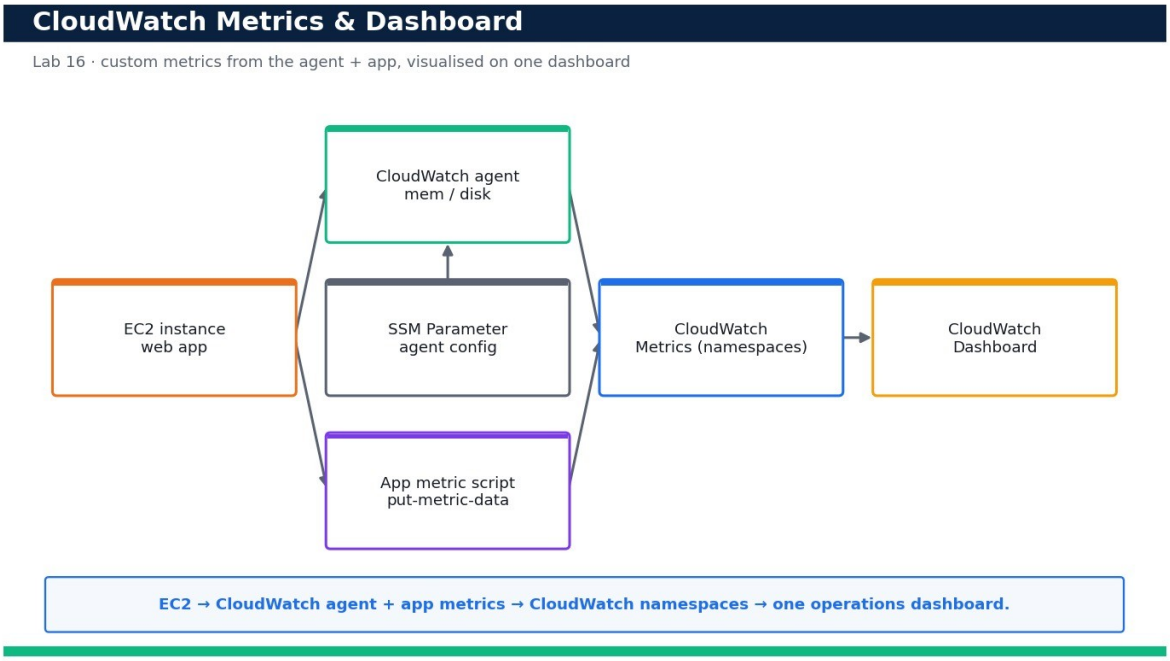


Figure — Lab 16 reference architecture: Collect Custom Metrics with the CloudWatch Agent and Build an Operations Dashboard.

What you'll build

A running CloudWatch agent emitting memory/disk custom metrics, a script publishing app latency/request/error metrics, and a CloudWatch dashboard visualising CPU, p90 latency, errors and request count. (Services: Amazon CloudWatch, CloudWatch agent, Amazon EC2, AWS Systems Manager (SSM Parameter Store), IAM.)

Step-by-step

217. Confirm your identity and note the instance you will instrument (reuse a tagged Day 3 instance or launch one t3.micro).

```
aws ec2 describe-instances --filters Name=tag:Name,Values=devops-web --query 'Reservations[].Instances[].InstanceId' --profile devops
```

Expected: One or more running InstanceId values are returned for the devops-web tag.

218. Create an IAM role for the instance that allows the agent to publish metrics and read its config, then attach it via an instance profile.

```
aws iam create-role --role-name devops-cwagent-role --assume-role-policy-document file://ec2-trust.json --profile devops
```

Expected: A Role with RoleName devops-cwagent-role is created and its Arn is returned.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

219. Attach the AWS-managed CloudWatchAgentServerPolicy so the agent can PutMetricData and read the SSM config parameter.

```
aws iam attach-role-policy --role-name devops-cwagent-role --
policy-arn arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy --
profile devops
```

Expected: The command returns with no error; list-attached-role-policies now shows CloudWatchAgentServerPolicy.

220. Store the agent configuration in SSM Parameter Store so the agent can be fetched consistently across instances.

```
aws ssm put-parameter --name /devops/cwagent/config --type String
--value file://cw-agent-config.json --profile devops
```

Expected: A Version number is returned, confirming the parameter /devops/cwagent/config was stored.

221. Install the CloudWatch agent on the instance (via SSM Run Command / package manager on Amazon Linux 2023).

```
sudo dnf install -y amazon-cloudwatch-agent
```

Expected: The amazon-cloudwatch-agent package installs successfully and its files appear under /opt/aws/amazon-cloudwatch-agent.

222. Start the agent using the configuration you stored in SSM Parameter Store.

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-
agent-ctl -a fetch-config -m ec2 -c ssm:/devops/cwagent/config -s
```

Expected: The agent reports 'Configuration validation first phase succeeded' and the status becomes running.

223. Verify the agent is running and healthy on the instance.

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-
agent-ctl -a status
```

Expected: Status shows "status": "running" for the amazon-cloudwatch-agent.

224. Confirm the custom memory metric is now flowing into CloudWatch under your namespace.

```
aws cloudwatch list-metrics --namespace CWAgent --metric-name
mem_used_percent --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: The response lists a mem_used_percent metric in the CWAgent namespace with an InstanceId dimension.

225. Confirm the custom disk-used metric is also present.

```
aws cloudwatch list-metrics --namespace CWAgent --metric-name
disk_used_percent --profile devops
```

Expected: A disk_used_percent metric is listed with device/path dimensions for the instance.

226. Publish an application latency metric with put-metric-data to model your app emitting a p90 latency observation.

```
aws cloudwatch put-metric-data --namespace DevOps/App --metric-
name LatencyMs --value 142 --unit Milliseconds --profile devops
```

Expected: The command returns with no output (exit code 0); the LatencyMs metric appears in the DevOps/App namespace.

227. Publish request-count and error-count application metrics (the provided script sends a small batch so the dashboard has data).

```
bash put-metric.sh
```

Expected: The script prints 'published N samples' and RequestCount / AppErrorCount now appear under DevOps/App.

228. Query the p90 statistic back from CloudWatch to prove the latency metric is queryable at the percentile you will chart.

```
aws cloudwatch get-metric-statistics --namespace DevOps/App --
metric-name LatencyMs --start-time $(date -u -d '-15 min' +
%FT%TZ) --end-time $(date -u +%FT%TZ) --period 60 --extended-
statistics p90 --profile devops
```

Expected: Datapoints are returned with an ExtendedStatistics p90 value for the recent window.

229. Create the operations dashboard from the provided dashboard.json (CPU, p90 latency, errors, request count).

```
aws cloudwatch put-dashboard --dashboard-name DevOpsOps --
dashboard-body file://dashboard.json --profile devops
```

Expected: DashboardValidationMessages is empty ([]), confirming the four widgets validated successfully.

230. List the dashboard to confirm it was created and is retrievable.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws cloudwatch list-dashboards --query 'DashboardEntries[?
DashboardName==`DevOpsOps`]' --profile devops
```

Expected: One entry named DevOpsOps is returned with a DashboardArn and LastModified timestamp.

231. Open the dashboard in the console and confirm all four widgets render live data.

```
# Console: CloudWatch > Dashboards > DevOpsOps
```

Expected: The dashboard shows CPUUtilization, LatencyMs (p90), AppErrorCount (Sum) and RequestCount (Sum) plotting recent datapoints.

232. Set the dashboard auto-refresh and time range, then re-run put-metric.sh and watch the widgets update.

```
bash put-metric.sh
```

Expected: Within the refresh interval the latency, request and error widgets step up to reflect the newly published samples.

Validation

`aws cloudwatch list-metrics` shows mem_used_percent and disk_used_percent under CWAgent and LatencyMs/RequestCount/AppErrorCount under DevOps/App; put-dashboard returns an empty DashboardValidationMessages array; and the DevOpsOps dashboard renders all four widgets with live data.

Test it

Publishing a fresh batch of samples with put-metric.sh causes the latency (p90), request-count and error-count widgets to move within one refresh cycle, and the memory/disk widgets track the instance — proving end-to-end custom-metric collection and visualisation.

Note: Cost & safety: Uses one t3.micro EC2 instance plus a handful of CloudWatch custom metrics, dashboards and API calls. Custom metrics and dashboards past the free allowance are a few US cents/month; delete the dashboard and custom metrics stop being written when you tear the instance down. Stay near-free by reusing a Day 3 instance. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-16-cloudwatch-metrics-dashboard/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 17 — Centralise Logs in CloudWatch Logs and Analyse Them with Logs Insights

Learning outcome: LO4. Skills mapped: K5, A6. Objective: Ship application and system logs into CloudWatch Logs with retention, then use Logs Insights queries to find the slowest and most error-prone requests (LO4, K5, A6).

Goal: When an incident hits, grepping across scattered instances wastes precious minutes. In this lab you centralise your application and system logs into CloudWatch Logs — creating log groups with a sensible retention policy and extending the CloudWatch agent to ship log files — then use CloudWatch Logs Insights queries to rank requests by latency, count errors by type and pinpoint the slowest and most error-prone endpoints.

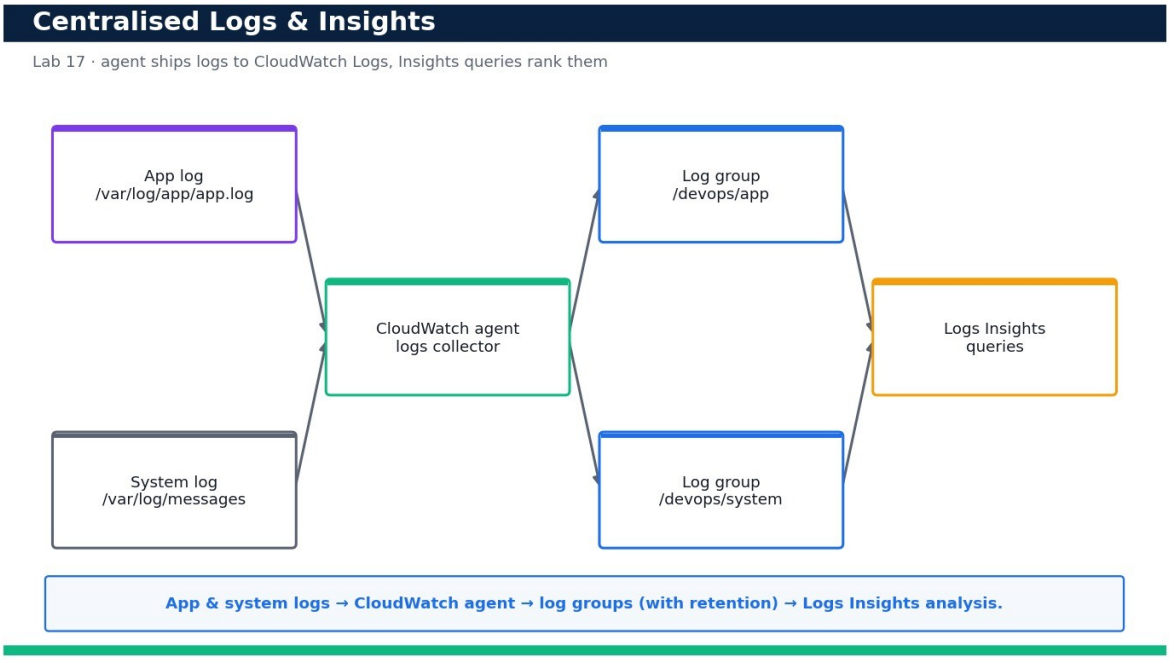


Figure — Lab 17 reference architecture: Centralise Logs in CloudWatch Logs and Analyse Them with Logs Insights.

What you'll build

Centralised app and system log groups (with retention) fed by the CloudWatch agent, plus a set of saved Logs Insights queries that surface the slowest and most error-prone requests. (Services: Amazon CloudWatch Logs, CloudWatch Logs Insights, CloudWatch agent, Amazon EC2, IAM.)

Step-by-step

233. Confirm your identity and the instance whose logs you will centralise.

```
aws sts get-caller-identity --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: Your own Account, UserId and Arn are printed (no error).

234. Create the application log group with the provided helper script.

```
bash create-log-group.sh /devops/app
```

Expected: The script reports the log group /devops/app was created (or already exists).

235. Create the system log group the same way.

```
bash create-log-group.sh /devops/system
```

Expected: The script reports the log group /devops/system was created.

236. Set a 14-day retention policy on the application log group so logs do not accumulate cost forever.

```
aws logs put-retention-policy --log-group-name /devops/app --
retention-in-days 14 --profile devops
```

Expected: The command returns with no output; describe-log-groups now shows retentionInDays 14 for /devops/app.

237. Set retention on the system log group as well.

```
aws logs put-retention-policy --log-group-name /devops/system --
retention-in-days 14 --profile devops
```

Expected: retentionInDays 14 is applied to /devops/system.

238. Extend the CloudWatch agent configuration to collect the app and system log files and start it.

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-
agent-ctl -a fetch-config -m ec2 -c file:/opt/cw-agent-logs.json
-s
```

Expected: The agent validates the config and restarts; it begins tailing /var/log/app/app.log and /var/log/messages.

239. Generate some sample traffic so the app writes structured request log lines (some slow, some errors).

```
for i in $(seq 1 50); do curl -s http://localhost/ >/dev/null;
done
```

Expected: The app writes 50 request lines to /var/log/app/app.log including a mix of latencies and a few ERROR lines.

240. Confirm a log stream now exists in the application log group.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws logs describe-log-streams --log-group-name /devops/app --
order-by LastEventTime --descending --profile devops
```

Expected: At least one logStream (named for the instance) is listed with a recent lastEventTimestamp.

241. Tail the most recent events to confirm the request log format is arriving.

```
aws logs tail /devops/app --since 10m --profile devops
```

Expected: Recent lines print showing timestamp, request path, status and latency_ms fields.

242. Run a Logs Insights query to rank the slowest requests by parsing latency_ms (query text is in logs-insights-queries.txt).

```
aws logs start-query --log-group-name /devops/app --start-time $
(date -u -d '-30 min' +%s) --end-time $(date -u +%s) --query-
string 'fields @timestamp, path, latency_ms | sort latency_ms
desc | limit 20' --profile devops
```

Expected: A queryId is returned; the query begins running against /devops/app.

243. Fetch the query results and read the slowest endpoints.

```
aws logs get-query-results --query-id <query-id> --profile devops
```

Expected: Status is Complete and up to 20 rows list paths ordered by descending latency_ms (the slowest requests first).

244. Run the error-breakdown query that counts ERROR lines grouped by request path.

```
aws logs start-query --log-group-name /devops/app --start-time $
(date -u -d '-30 min' +%s) --end-time $(date -u +%s) --query-
string 'filter level="ERROR" | stats count(*) as errors by path |
sort errors desc' --profile devops
```

Expected: A queryId is returned for the error-breakdown query.

245. Fetch the error-breakdown results to find the most error-prone endpoint.

```
aws logs get-query-results --query-id <query-id> --profile devops
```

Expected: Rows list each path with its error count in descending order, revealing the most error-prone endpoint.

246. Compute the p90 latency across all requests with an Insights aggregation query.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws logs start-query --log-group-name /devops/app --start-time $
(date -u -d '-30 min' +%s) --end-time $(date -u +%s) --query-
string 'stats pct(latency_ms, 90) as p90, count(*) as reqs by
bin(5m)' --profile devops
```

Expected: A queryId is returned; results show p90 latency and request count per 5-minute bin.

247. Save the useful queries in the console so the team can reuse them during incidents.

```
# Console: CloudWatch > Logs Insights > Actions > Save query
(paste from logs-insights-queries.txt)
```

Expected: The 'Slowest requests' and 'Errors by path' queries appear in your saved-queries list.

Validation

`describe-log-groups` shows /devops/app and /devops/system with retentionInDays 14; the agent is tailing both files (log streams exist with recent timestamps); and the three Logs Insights queries return Complete with rows ranking slowest requests, errors by path, and p90 latency per bin.

Test it

Running the 'slowest requests' and 'errors by path' Insights queries over the last 30 minutes returns concrete endpoints ordered by latency and error count — you can name the slowest and most error-prone request from the centralised logs alone, without touching the instance.

Note: Cost & safety: CloudWatch Logs charges for ingestion, storage and Insights scans. A single instance emitting light logs with a short retention (7–14 days) costs a few US cents; Insights bills per GB scanned so keep query time ranges tight. Setting retention (not 'Never expire') and deleting the log groups on teardown keeps this near-free. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-17-centralized-logs-insights/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 18 — Turn Logs into Signals: Metric Filters, CloudWatch Alarms and SNS Notifications

Learning outcome: LO4. Skills mapped: K5, A6. Objective: Create a metric filter that counts ERROR log lines, alarm on that metric and on high CPU, and notify an on-call engineer by email via SNS (LO4, K5, A6).

Goal: Logs are only useful if they page someone when things go wrong. In this lab you turn the centralised app logs from Lab 17 into a signal: a CloudWatch Logs metric filter counts ERROR lines into a metric, then two alarms — one on the error metric and one on high CPU — publish to an SNS topic that emails your on-call address. You then drive the error alarm into ALARM state and confirm the notification actually arrives.

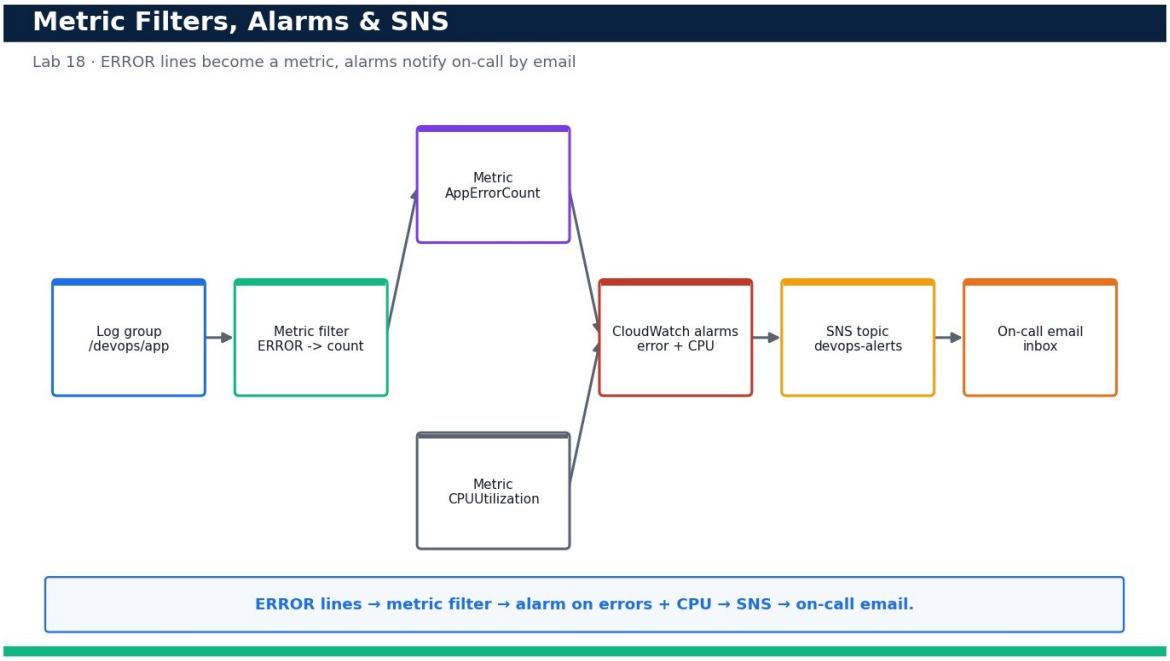


Figure — Lab 18 reference architecture: Turn Logs into Signals: Metric Filters, CloudWatch Alarms and SNS Notifications.

What you'll build

A metric filter converting ERROR log lines into a CloudWatch metric, two alarms (errors and high CPU) wired to an SNS topic with a confirmed email subscription, and a demonstrated ALARM-state email notification. (Services: Amazon CloudWatch (metric filters + alarms), Amazon CloudWatch Logs, Amazon SNS, AWS CloudFormation, IAM.)

Step-by-step

248. Confirm the /devops/app log group from Lab 17 exists (the metric filter attaches to it).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws logs describe-log-groups --log-group-name-prefix /devops/app
--query 'logGroups[].logGroupName' --profile devops
```

Expected: The list contains /devops/app.

249. Deploy the SNS topic and the two alarms from the provided CloudFormation template.

```
aws cloudformation deploy --template-file alarm.yaml --stack-name
devops-alerts --capabilities CAPABILITY_IAM --profile devops
```

Expected: Stack devops-alerts reaches CREATE_COMPLETE and outputs a TopicArn.

250. Capture the SNS topic ARN from the stack outputs for the subscription step.

```
aws cloudformation describe-stacks --stack-name devops-alerts --
query 'Stacks[0].Outputs[?OutputKey==`TopicArn`].OutputValue' --
output text --profile devops
```

Expected: A topic ARN like arn:aws:sns:ap-southeast-1:<your-account-id>:devops-alerts is printed.

251. Subscribe your on-call email address to the topic with the provided helper script.

```
bash subscribe.sh you@example.com <topic-arn>
```

Expected: The script requests a subscription and prints 'confirmation email sent to you@example.com'.

252. Open the confirmation email and click 'Confirm subscription', then verify the subscription is confirmed.

```
aws sns list-subscriptions-by-topic --topic-arn <topic-arn> --
query 'Subscriptions[].SubscriptionArn' --profile devops
```

Expected: The SubscriptionArn is a real ARN (not the string 'PendingConfirmation'), confirming the email is subscribed.

253. Create the metric filter that counts ERROR lines from the log group into the DevOps/App AppErrorCount metric.

```
aws logs put-metric-filter --cli-input-json file://metric-
filter.json --profile devops
```

Expected: The command returns with no output; describe-metric-filters now lists app-error-count on /devops/app.

254. Confirm the metric filter is attached to the log group.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws logs describe-metric-filters --log-group-name /devops/app --
query 'metricFilters[].filterName' --profile devops
```

Expected: The list contains app-error-count.

255. Verify the error alarm was created and note its current state.

```
aws cloudwatch describe-alarms --alarm-names devops-app-errors --
query 'MetricAlarms[0].StateValue' --profile devops
```

Expected: The state prints as OK or INSUFFICIENT_DATA (no errors have breached the threshold yet).

256. Verify the high-CPU alarm exists too.

```
aws cloudwatch describe-alarms --alarm-names devops-high-cpu --
query 'MetricAlarms[0].StateValue' --profile devops
```

Expected: The high-CPU alarm is listed with state OK or INSUFFICIENT_DATA.

257. Generate ERROR log lines so the metric filter increments the AppErrorCount metric above the alarm threshold.

```
for i in $(seq 1 10); do aws logs put-log-events --log-group-name
/devops/app --log-stream-name test-errors --log-events
timestamp=$(date +%s000),message='ERROR simulated failure' --
profile devops; done
```

Expected: Ten ERROR events are written; within a minute the AppErrorCount metric shows a Sum above the threshold of 5.

258. Confirm the metric filter is publishing the error count metric.

```
aws cloudwatch get-metric-statistics --namespace DevOps/App --
metric-name AppErrorCount --start-time $(date -u -d '-10 min' +
%FT%TZ) --end-time $(date -u +%FT%TZ) --period 60 --statistics
Sum --profile devops
```

Expected: Datapoints are returned with a Sum reflecting the ERROR lines you generated.

259. Wait for the alarm evaluation period and confirm the error alarm transitions to ALARM.

```
aws cloudwatch describe-alarms --alarm-names devops-app-errors --
query 'MetricAlarms[0].StateValue' --profile devops
```

Expected: The StateValue is now ALARM.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

260. Confirm the on-call email received the ALARM notification (check your inbox).

```
# Inbox: subject 'ALARM: devops-app-errors in ...'
```

Expected: An email arrives from AWS Notifications describing the alarm transition to ALARM with the reason and metric.

261. Review the alarm's state-change history to see the transition and the SNS action firing.

```
aws cloudwatch describe-alarm-history --alarm-name devops-app-errors --history-item-type StateUpdate --max-records 5 --profile devops
```

Expected: The history shows a StateUpdate from OK/INSUFFICIENT_DATA to ALARM with the SNS action executed.

262. Stop generating errors and confirm the alarm returns to OK once the metric falls back below the threshold.

```
aws cloudwatch describe-alarms --alarm-names devops-app-errors --query 'MetricAlarms[0].StateValue' --profile devops
```

Expected: After the next evaluation period with no new errors the StateValue returns to OK (and an 'OK:' email arrives).

263. (Optional) Manually drive the alarm with set-alarm-state to demonstrate the notification path without waiting for real data.

```
aws cloudwatch set-alarm-state --alarm-name devops-app-errors --state-value ALARM --state-reason 'manual test' --profile devops
```

Expected: The alarm flips to ALARM immediately and a test notification email is delivered via SNS.

Validation

`describe-metric-filters` lists app-error-count on /devops/app; both devops-app-errors and devops-high-cpu alarms exist; the email subscription shows a real (confirmed) SubscriptionArn; and generating ERROR lines drives devops-app-errors into ALARM with a matching state-change history entry.

Test it

After you generate ERROR log lines, the metric filter increments AppErrorCount, the devops-app-errors alarm transitions to ALARM, and an 'ALARM:' email arrives at the confirmed on-call address — a full log-to-notification signal path. Stopping the errors returns the alarm to OK with an 'OK:' email.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: Cost & safety: Metric filters, alarms and SNS email are effectively free at this scale (standard alarms and a handful of email notifications sit within the free allowance). The only care needed is deleting the alarms, filter and topic on teardown so no stray metric-based alarms linger. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-18-metric-filters-alarms-sns/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Domain 05 — Incident and Event Response (14%)

Event sources · automated response · configuration change · troubleshooting

Key concepts

- Event-driven operations — EventBridge routes events from AWS services and applications to targets that act automatically.
- Notify & act — SNS notifies people; Lambda, Systems Manager Automation and Step Functions take the remediating action.
- Auto-remediation — AWS Config rules and EventBridge patterns detect drift and fix it — tag, isolate, restart or roll back.
- Runbooks & playbooks — Systems Manager documents codify the response so incidents are handled consistently and fast.
- Deployment failure response — CloudWatch alarms roll back CodeDeploy; pipeline stages stop and alert on failure.
- Troubleshooting — Correlate metrics, logs, traces and CloudTrail events to find and fix system and application failures.

Lab 19 — Route Incidents with Amazon EventBridge

Learning outcome: LO5. Skills mapped: K7, A5. Objective: Build an EventBridge rule that matches incident signals and fans them out to notify and log targets (LO5, K7, A5).

Goal: Your team wants a single 'routing brain' for operational incidents so that a CloudWatch alarm firing, an EC2 instance changing state, or an AWS Config compliance change all land in the same place. In this lab you author and validate a precise EventBridge event pattern, create a rule on the default event bus, and fan the matched events out to two targets — an SNS topic for human notification and a CloudWatch Logs group for an audit trail. You then prove the pattern with `aws events test-event-pattern` and simulate a real event.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

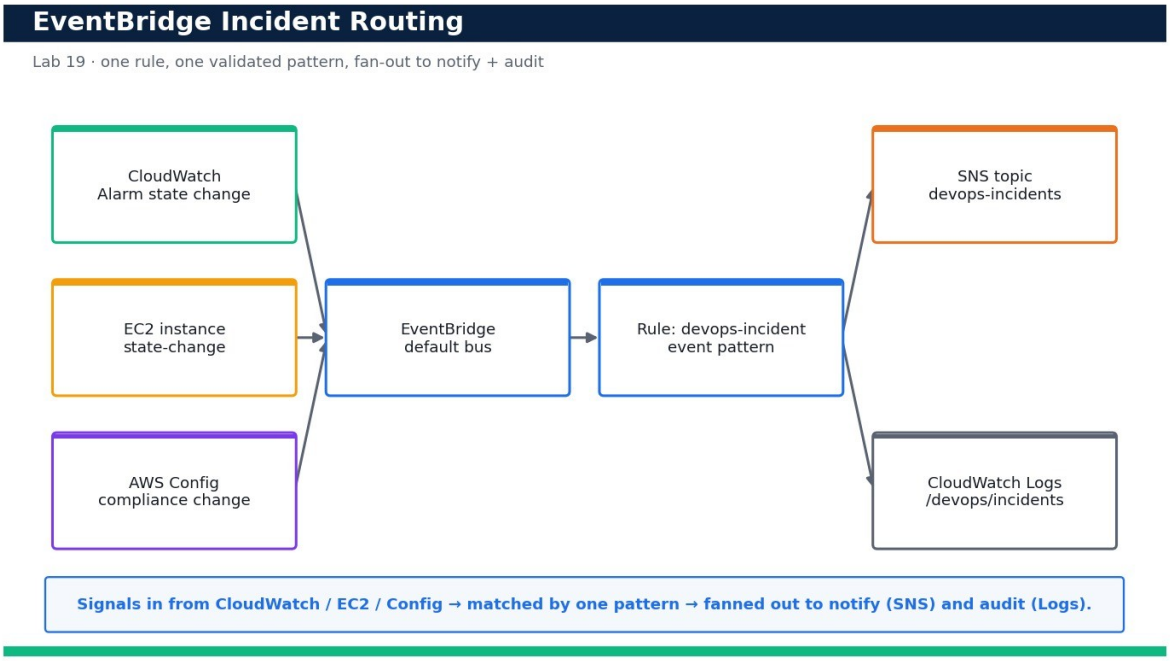


Figure — Lab 19 reference architecture: Route Incidents with Amazon EventBridge.

What you'll build

An enabled EventBridge rule on the default bus whose validated event pattern matches CloudWatch Alarm state changes, EC2 state-change notifications and AWS Config compliance-change events, fanning out to an SNS topic (notify) and a CloudWatch Logs group (audit) — proven with a simulated event. (Services: Amazon EventBridge, Amazon SNS, Amazon CloudWatch, CloudWatch Logs, IAM.)

Step-by-step

264. Confirm your identity and Region before you start so every resource lands in the right account.

```
aws sts get-caller-identity --profile devops
```

Expected: Your own Account, UserId and ARN are printed (no error).

265. Reuse the incident SNS topic from the Day-4 monitoring labs, or create it if it does not exist, and capture its ARN.

```
aws sns create-topic --name devops-incidents --profile devops --
query TopicArn --output text
```

Expected: An ARN like arn:aws:sns:ap-southeast-1:<your-account-id>:devops-incidents is returned.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

266. Subscribe your email to the topic so you receive incident notifications, then confirm the subscription from your inbox.

```
aws sns subscribe --topic-arn <topic-arn> --protocol email --
notification-endpoint you@example.com --profile devops
```

Expected: SubscriptionArn is 'pending confirmation'; after you click the link in the email it becomes a real ARN.

267. Create the CloudWatch Logs group that will hold the incident audit trail.

```
aws logs create-log-group --log-group-name /devops/incidents --
profile devops
```

Expected: The command returns no output; the log group now exists.

268. Validate your event pattern BEFORE creating any rule, using a sample CloudWatch Alarm state-change event.

```
aws events test-event-pattern --event-pattern file://event-
pattern.json --event file://sample-alarm-event.json --profile
devops
```

Expected: The command prints { "Result": true } — the pattern matches the sample alarm event.

269. Confirm the same pattern also matches a sample EC2 state-change event (multi-source pattern works).

```
aws events test-event-pattern --event-pattern file://event-
pattern.json --event file://sample-ec2-event.json --profile
devops
```

Expected: { "Result": true } is printed for the EC2 sample event too.

270. Confirm the pattern REJECTS an unrelated event so you know it is precise, not a catch-all.

```
aws events test-event-pattern --event-pattern file://event-
pattern.json --event file://sample-unrelated-event.json --profile
devops
```

Expected: { "Result": false } is printed — an S3 object-created event does NOT match.

271. Create the EventBridge rule on the default bus from the reviewed rule definition.

```
aws events put-rule --name devops-incident --event-pattern
file://event-pattern.json --state ENABLED --description 'Route
CloudWatch/EC2/Config incidents' --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: A RuleArn like arn:aws:events:ap-southeast-1:<your-account-id>:rule/devops-incident is returned.

272. Grant EventBridge permission to publish to your SNS topic by setting a topic policy allowing the events service.

```
aws sns set-topic-attributes --topic-arn <topic-arn> --attribute-name Policy --attribute-value file://sns-topic-policy.json --profile devops
```

Expected: The command returns no output; the topic now allows events.amazonaws.com to Publish.

273. Attach BOTH targets to the rule in one call — the SNS topic (notify) and the CloudWatch Logs group (audit).

```
aws events put-targets --rule devops-incident --targets Id=notify,Arn=<topic-arn> Id=audit,Arn=arn:aws:logs:ap-southeast-1:<your-account-id>:log-group:/devops/incidents --profile devops
```

Expected: FailedEntryCount is 0 — both targets were registered on the rule.

274. List the rule's targets and confirm the fan-out is in place.

```
aws events list-targets-by-rule --rule devops-incident --profile devops --query 'Targets[].Id'
```

Expected: The output lists both target Ids: ["notify", "audit"].

275. Simulate a live incident by putting a synthetic EC2 state-change event onto the default bus.

```
aws events put-events --entries file://put-events-ec2.json --profile devops
```

Expected: FailedEntryCount is 0 and an EventId is returned — the event was accepted by the bus.

276. Confirm the rule actually matched and invoked by inspecting its Invocations metric in CloudWatch.

```
aws cloudwatch get-metric-statistics --namespace AWS/Events --metric-name Invocations --dimensions Name=RuleName,Value=devops-incident --start-time $(date -u -v-15M +%Y-%m-%dT%H:%M:%SZ) --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) --period 300 --statistics Sum --profile devops
```

Expected: At least one datapoint with Sum >= 1 is returned, proving the rule fired.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

277. Verify the audit target received the event by checking that the log group now has a log stream with content.

```
aws logs describe-log-streams --log-group-name /devops/incidents
--order-by LastEventTime --descending --profile devops --query
'logStreams[0].logStreamName'
```

Expected: A log stream name is returned; opening it shows the routed EC2 state-change event JSON.

278. Confirm the notify path worked by checking your email for the SNS message about the simulated incident.

Expected: Your inbox shows an 'AWS Notification' email containing the EC2 state-change event details.

Validation

`aws events list-targets-by-rule` shows both the notify and audit targets; the three `test-event-pattern` runs return true/true/false; after `put-events` the AWS/Events Invocations metric shows a datapoint, the /devops/incidents log group has a stream containing the event, and an SNS email arrives — proving one validated pattern fans a matched incident out to both targets.

Test it

You author a precise multi-source event pattern, prove it matches CloudWatch Alarm and EC2 events but rejects an unrelated S3 event, then simulate an incident and observe it routed to SNS (email received) and CloudWatch Logs (stream written) with the rule's Invocations metric incrementing.

Note: Cost & safety: EventBridge default-bus rules and AWS-service events are effectively free at lab scale; you pay only trivial amounts for SNS email notifications and CloudWatch Logs ingestion/storage. Delete the log group and rule the same day to keep this to a few cents. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-19-eventbridge-incident-routing/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 20 — Automated Remediation with Lambda and Systems Manager

Learning outcome: LO5. Skills mapped: K7, A5. Objective: Wire an EventBridge target to a least-privilege Lambda that triggers an SSM Automation runbook so an unhealthy resource self-heals (LO5, K7, A5).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: Notification alone does not resolve an incident — the on-call engineer still has to act. In this lab you close the loop: an EventBridge rule detects an unhealthy/stopped EC2 instance and invokes a Lambda function running under a tightly-scoped IAM role. The Lambda validates the event and starts an AWS Systems Manager Automation runbook that remediates the instance (restarts it), while also notifying the team over SNS. You deploy the whole stack with CloudFormation, simulate an incident, and confirm the resource returns to a healthy state automatically with no human intervention.

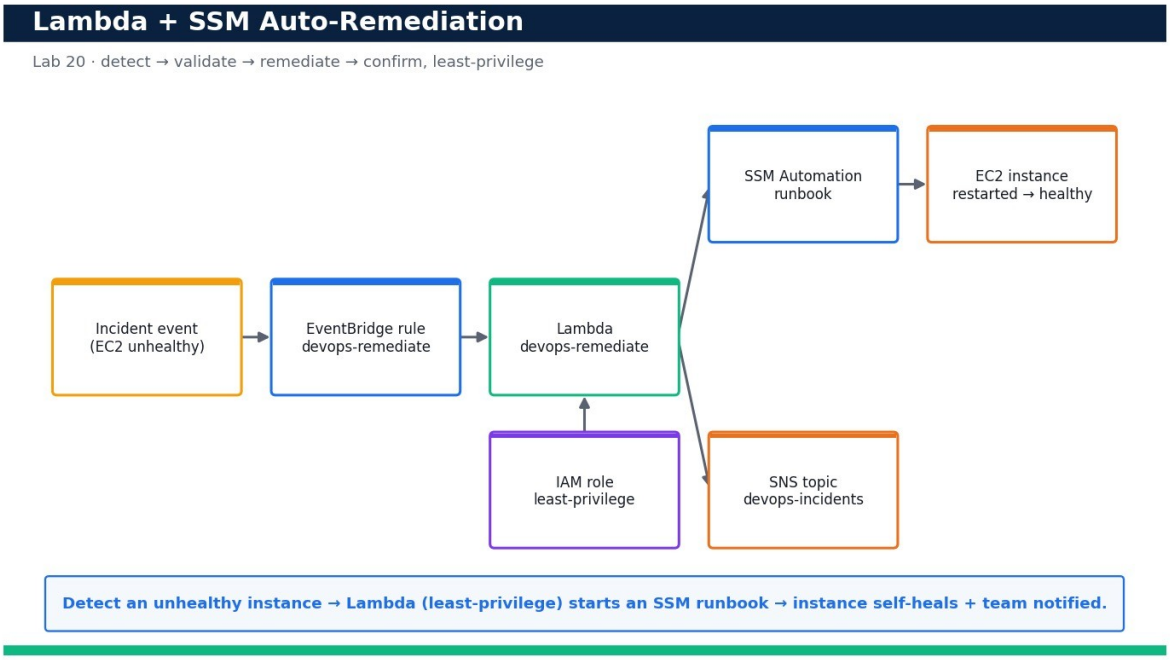


Figure — Lab 20 reference architecture: Automated Remediation with Lambda and Systems Manager.

What you'll build

A CloudFormation-deployed auto-remediation stack (EventBridge rule + least-privilege Lambda + SSM Automation runbook + SNS notify) that, on an unhealthy-instance event, restarts the instance automatically and returns it to a running/healthy state — proven end to end from a simulated incident. (Services: AWS Lambda, AWS Systems Manager Automation, Amazon EventBridge, AWS IAM, Amazon CloudWatch Logs, Amazon SNS, Amazon EC2.)

Step-by-step

279. Confirm your identity and Region so the stack deploys into the right account.

```
aws sts get-caller-identity --profile devops
```

Expected: Your own Account, UserId and ARN are printed.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

280. Launch a disposable t3.micro test instance tagged so the remediation logic can identify it (or reuse a tagged instance from Lab 3).

```
aws ec2 run-instances --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 --instance-type t3.micro --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=devops-remediation-target},{Key=devops:remediate,Value=true}]' --profile devops --query 'Instances[0].InstanceId' --output text
```

Expected: An InstanceId like i-0abcd1234efgh5678 is returned.

281. Package the Lambda handler into a deployment zip that CloudFormation will reference.

```
zip remediate.zip remediate.py
```

Expected: adding: remediate.py — a remediate.zip archive is created.

282. Upload the Lambda package to an S3 bucket you control (create one with Block Public Access if needed).

```
aws s3 cp remediate.zip s3://devops-lambda-<your-account-id>/remediate.zip --profile devops
```

Expected: upload: ./remediate.zip to s3://devops-lambda-<your-account-id>/remediate.zip.

283. Register the SSM Automation runbook document before wiring the stack so a syntax error does not surface at runtime.

```
aws ssm create-document --name devops-restart-instance --document-type Automation --document-format YAML --content file://ssm-runbook.yaml --profile devops
```

Expected: The document is created with Status Creating then Active; DocumentType is Automation.

284. Validate the CloudFormation template locally before deploying.

```
aws cloudformation validate-template --template-body file://response.yaml --profile devops
```

Expected: The template is valid; its Parameters and Capabilities (CAPABILITY_NAMED_IAM) are listed.

285. Deploy the auto-remediation stack (least-privilege role, Lambda, EventBridge rule and SNS wiring).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws cloudformation deploy --template-file response.yaml --stack-name devops-remediation --capabilities CAPABILITY_NAMED_IAM --parameter-overrides LambdaBucket=devops-lambda-<your-account-id> RunbookName=devops-restart-instance --profile devops
```

Expected: Stack devops-remediation reaches CREATE_COMPLETE.

286. Confirm the Lambda's execution role is least-privilege — it can start ONLY the intended automation and EC2 actions.

```
aws iam get-role-policy --role-name devops-remediate-role --policy-name devops-remediate-inline --profile devops --query 'PolicyDocument.Statement[].Action'
```

Expected: The actions list is scoped to ssm:StartAutomationExecution, ec2:StartInstances/DescribeInstances, sns:Publish and CloudWatch Logs only — no Resource wildcard on the runbook.

287. Verify the EventBridge rule that triggers remediation is enabled and targets the Lambda.

```
aws events list-targets-by-rule --rule devops-remediate --profile devops --query 'Targets[].Arn'
```

Expected: The output shows the ARN of the devops-remediate Lambda function.

288. Create the incident: stop the test instance so it becomes 'unhealthy' from the app's point of view.

```
aws ec2 stop-instances --instance-ids <instance-id> --profile devops
```

Expected: The instance transitions to stopping then stopped.

289. Simulate the detection event that EventBridge would receive for the stopped instance (drives the remediation path deterministically).

```
aws events put-events --entries file://put-events-stopped.json --profile devops
```

Expected: FailedEntryCount is 0 and an EventId is returned.

290. Confirm the Lambda was invoked by tailing its CloudWatch Logs and seeing the remediation decision.

```
aws logs tail /aws/lambda/devops-remediate --since 5m --profile devops
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: The log shows 'Received event', the extracted instance-id, and 'Started SSM automation execution <id>'.

291. Confirm the SSM Automation runbook actually ran and succeeded.

```
aws ssm describe-automation-executions --filters
Key=DocumentNamePrefix,Values=devops-restart-instance --profile
devops --query
'AutomationExecutionMetadataList[0].AutomationExecutionStatus'
```

Expected: The status is Success (the runbook restarted the instance).

292. Confirm the resource returned to a healthy state automatically — the instance is running again.

```
aws ec2 describe-instances --instance-ids <instance-id> --profile
devops --query 'Reservations[0].Instances[0].State.Name'
```

Expected: The state is 'running' — the instance self-healed with no human action.

293. Confirm the team was notified in parallel by checking your inbox for the SNS remediation message.

Expected: An SNS email arrives describing the auto-remediation that was performed on the instance.

Validation

`aws cloudformation describe-stacks` shows devops-remediation CREATE_COMPLETE; the Lambda log shows it received the event and started an SSM automation; `describe-automation-executions` reports Success; `describe-instances` shows the previously-stopped instance back in 'running'; and the IAM inline policy contains only the specific remediation actions (no broad wildcards).

Test it

You stop the test instance to create an incident, EventBridge invokes the least-privilege Lambda, the Lambda starts the SSM Automation runbook, and within a couple of minutes the instance is running again and an SNS notification is received — proving the incident self-heals automatically end to end.

Note: Cost & safety: Lambda and SSM Automation are pay-per-use and effectively free at lab scale; one t3.micro test instance is the only real line item. Stop or terminate the test instance and delete the stack the same day to stay near-zero. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-20-lambda-ssm-auto-remediation/README.md

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

(mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Domain 06 — Security and Compliance (17%)

Identity & access at scale · data protection · security monitoring & audit

Key concepts

- Identity & access management — IAM users, roles, policies and permission boundaries enforce least privilege; roles beat long-lived keys.
- IAM at scale — Organizations SCPs, IAM Identity Center and federation manage access across many accounts.
- Data protection — KMS encryption, Secrets Manager, S3 Block Public Access and TLS protect data at rest and in transit.
- Detective controls — CloudTrail, AWS Config, GuardDuty and Security Hub record, assess and score the account's posture.
- Automated compliance — Config rules and conformance packs continuously check resources against a technical blueprint.
- Governance blueprint — A landing zone plus guardrails give a documented, auditable security and compliance baseline.

Lab 21 — Enforce IAM Least Privilege and Analyse Policies for Over-Permissive Access

Learning outcome: LO6. Skills mapped: K6, A4. Objective: Write a scoped IAM policy, cap it with a permission boundary, and prove least privilege with IAM Access Analyzer.

Goal: Broad permissions are the single most common cloud security failure. In this lab you replace a wildcard grant with a tightly scoped policy for the sample app's role, attach a permission boundary so the role can never exceed its remit, then use IAM Access Analyzer policy validation, `iam simulate-custom-policy` and an Access Analyzer external-access analyzer to find over-permissive grants and unused access. You tighten the policy until it validates cleanly and only the actions the app truly needs remain.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

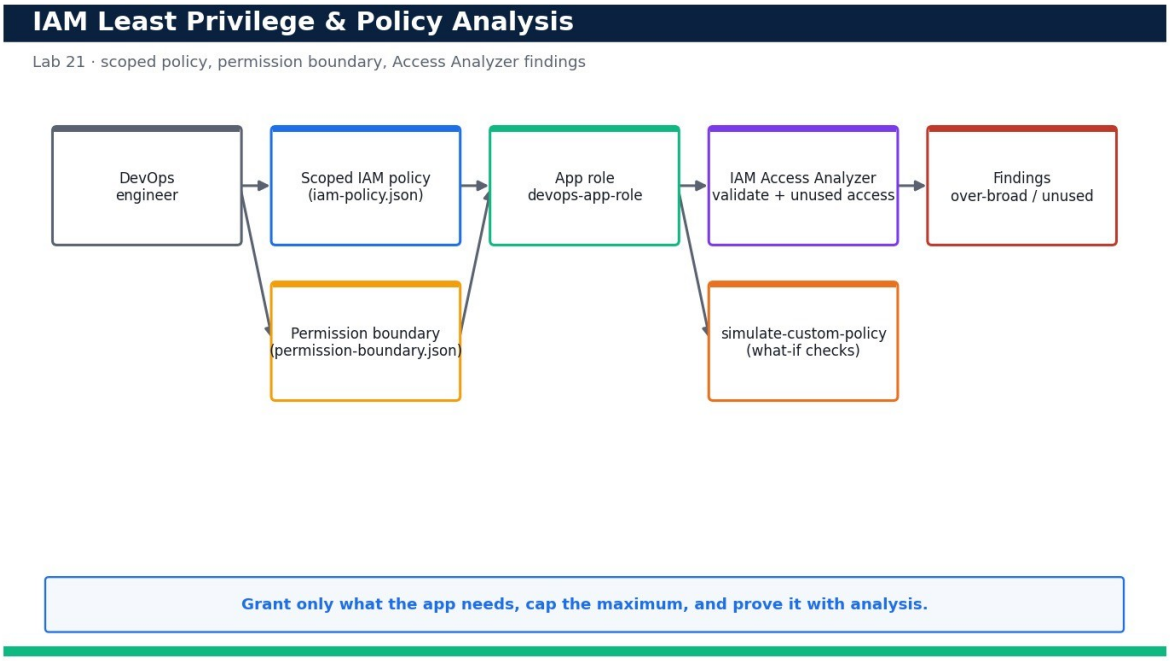


Figure — Lab 21 reference architecture: Enforce IAM Least Privilege and Analyse Policies for Over-Permissive Access.

What you'll build

A least-privilege IAM policy for the app role, capped by a permission boundary, that passes Access Analyzer validation with no security warnings and grants nothing beyond the app's real needs. (Services: AWS IAM, IAM Access Analyzer, AWS STS, AWS CloudFormation.)

Step-by-step

294. Review the deliberately over-broad starting policy so you understand what 'too much' looks like before you scope it.

```
cat iam-policy.json
```

Expected: The file shows the intended least-privilege policy; compare it against the broad `s3:*`/`logs:*` grant it replaces.

295. Create an app role that trusts EC2 (the sample app runs on EC2) so you have a real principal to scope.

```
aws iam create-role --role-name devops-app-role --assume-role-policy-document file://trust-policy.json --profile devops
```

Expected: Returns the role Arn arn:aws:iam::<your-account-id>:role/devops-app-role.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

296. Create the permission-boundary policy that caps the maximum permissions the app role can ever hold.

```
aws iam create-policy --policy-name devops-app-boundary --policy-document file://permission-boundary.json --profile devops
```

Expected: Returns the boundary policy Arn arn:aws:iam::<your-account-id>:policy/devops-app-boundary.

297. Attach the permission boundary to the app role so later policy attachments can never escalate beyond it.

```
aws iam put-role-permissions-boundary --role-name devops-app-role --permissions-boundary arn:aws:iam::<your-account-id>:policy/devops-app-boundary --profile devops
```

Expected: The command succeeds; `get-role` now shows PermissionsBoundary set on the role.

298. Create the tightly scoped least-privilege policy from the reviewed JSON.

```
aws iam create-policy --policy-name devops-app-least-priv --policy-document file://iam-policy.json --profile devops
```

Expected: Returns the policy Arn arn:aws:iam::<your-account-id>:policy/devops-app-least-priv.

299. Attach the least-privilege policy to the app role.

```
aws iam attach-role-policy --role-name devops-app-role --policy-arn arn:aws:iam::<your-account-id>:policy/devops-app-least-priv --profile devops
```

Expected: The command succeeds; the role now grants only the scoped S3 and logs actions.

300. Run IAM Access Analyzer policy validation to catch over-permissive constructs (wildcards, missing conditions).

```
aws accessanalyzer validate-policy --policy-document file://iam-policy.json --policy-type IDENTITY_POLICY --query 'findings[].{type:findingType,detail:findingDetails}'
```

Expected: Returns an empty list (or only SUGGESTION/WARNING items you resolve) — no SECURITY_WARNING findings remain.

301. Simulate a WHAT-IF: prove the app role is allowed the one write it needs (put its data object).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws iam simulate-custom-policy --policy-input-list file://iam-policy.json --action-names s3:PutObject --resource-arns arn:aws:s3:::devops-sample-app-data/report.json --profile devops
```

Expected: The EvalDecision for s3:PutObject on the app data bucket is 'allowed'.

302. Simulate a WHAT-IF: prove the role is DENIED an over-broad action it must never have (deleting any bucket).

```
aws iam simulate-custom-policy --policy-input-list file://iam-policy.json --action-names s3:DeleteBucket --resource-arns arn:aws:s3:::any-other-bucket --profile devops
```

Expected: The EvalDecision for s3:DeleteBucket is 'implicitDeny' — the scoped policy does not grant it.

303. Create an IAM Access Analyzer ACCOUNT analyzer to detect resources shared outside your account (external access).

```
aws accessanalyzer create-analyzer --analyzer-name devops-account-analyzer --type ACCOUNT --profile devops
```

Expected: Returns the analyzer Arn; the analyzer begins scanning IAM roles, S3 buckets and KMS keys for external access.

304. List any external-access findings so you can spot resources unintentionally shared with other principals.

```
aws accessanalyzer list-findings --analyzer-arn <analyzer-arn> --query 'findings[?status==`ACTIVE`].{resource:resource,principal:principal}' --profile devops
```

Expected: Returns ACTIVE findings (ideally an empty list once the app role and buckets are private) with the offending principal shown.

305. Create an UNUSED-ACCESS analyzer to surface permissions the app role has but has never used.

```
aws accessanalyzer create-analyzer --analyzer-name devops-unused-access --type ACCOUNT_UNUSED_ACCESS --profile devops
```

Expected: Returns the unused-access analyzer Arn; after its first evaluation it reports unused actions/roles you can remove.

306. Run the analyze-access.sh helper to bundle validation, both what-if simulations and the findings pull into one signal.

```
bash analyze-access.sh
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: Prints 'VALIDATION: clean', 'ALLOW put: allowed', 'DENY delete: implicitDeny' and 'EXTERNAL FINDINGS: 0' — a single green summary.

307. Tighten further: if the unused-access analyzer flags an action, remove it from iam-policy.json, update the policy and re-validate.

```
aws iam create-policy-version --policy-arn arn:aws:iam::<your-account-id>:policy/devops-app-least-priv --policy-document file://iam-policy.json --set-as-default --profile devops
```

Expected: A new default policy version is created; re-running validate-policy still returns no security warnings and the removed action no longer appears.

308. Confirm the boundary blocks privilege escalation even if a broad policy were attached later.

```
aws iam simulate-custom-policy --policy-input-list file://permission-boundary.json --action-names iam:CreateUser --profile devops
```

Expected: The EvalDecision for iam:CreateUser is 'implicitDeny' — the boundary caps escalation regardless of attached policies.

Validation

The app role carries a permission boundary and a scoped least-privilege policy; `accessanalyzer validate-policy` returns no security warnings; `simulate-custom-policy` allows the one required write and implicit-denies over-broad actions; the account analyzer reports zero unintended external-access findings; and analyze-access.sh prints a fully green summary.

Test it

You can prove, with Access Analyzer and policy simulation, that the app role grants exactly the actions the app needs and nothing more, that a permission boundary caps escalation, and that no resource is shared externally — then tighten the policy further based on unused-access findings and re-validate cleanly.

Note: Cost & safety: IAM policies, roles, permission boundaries and `simulate-custom-policy` are free. IAM Access Analyzer's external-access analyzer is free; the unused-access analyzer is billed per IAM role/user analysed per month, so delete the unused-access analyzer at the end to avoid a small recurring charge. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-21-iam-least-privilege-analysis/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 22 — Protect Data with Secrets Manager and Customer-Managed KMS Encryption

Learning outcome: LO6. Skills mapped: K6, A4. Objective: Create a customer-managed KMS key with a key policy, store a secret with rotation, and enforce encryption on S3.

Goal: Sensitive data must be encrypted with keys you control and secrets must never live in code. In this lab you create a customer-managed KMS key with a least-privilege key policy, store an application credential in AWS Secrets Manager (wiring up automatic rotation), then encrypt an S3 bucket with your key and add a bucket policy that DENIES any unencrypted upload. You demonstrate encrypt/decrypt with the CLI and prove the deny-unencrypted guardrail actually blocks a plaintext put.

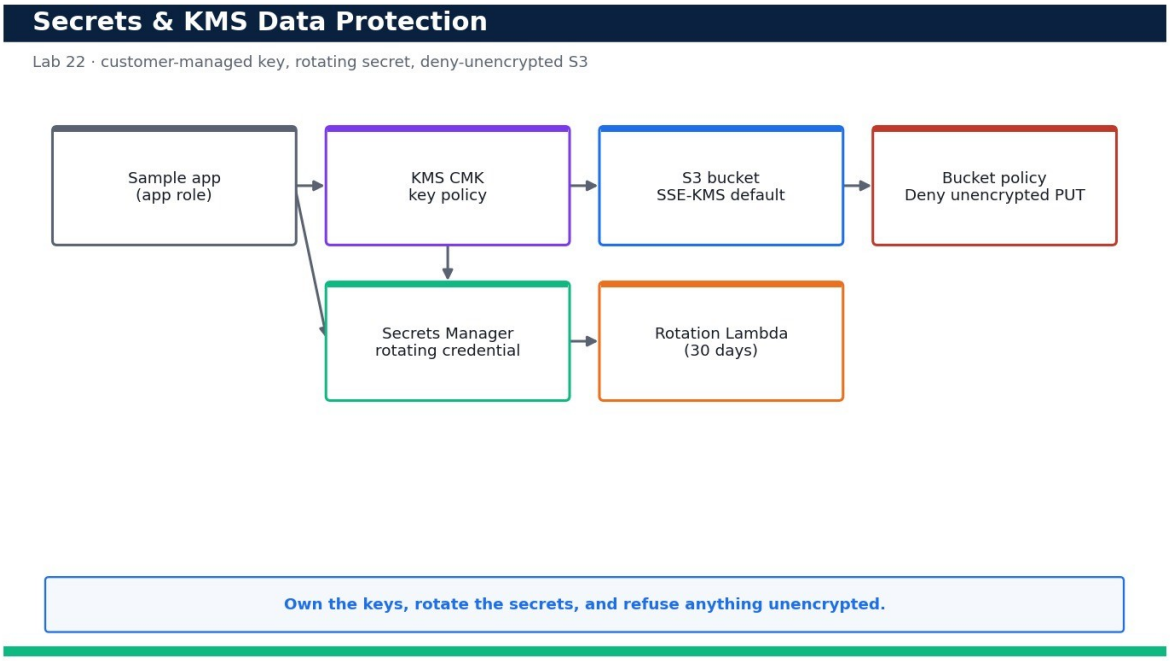


Figure — Lab 22 reference architecture: Protect Data with Secrets Manager and Customer-Managed KMS Encryption.

What you'll build

A customer-managed KMS key with a scoped key policy, a rotating Secrets Manager secret, and an S3 bucket that encrypts by default with your key and rejects any unencrypted upload. (Services: AWS KMS, AWS Secrets Manager, Amazon S3, AWS Lambda (rotation), AWS CloudFormation.)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

309. Deploy the KMS key and its scoped key policy from the template (the policy grants the app role decrypt, not admin).

```
aws cloudformation deploy --template-file kms-key.yaml --stack-name devops-kms --capabilities CAPABILITY_NAMED_IAM --profile devops
```

Expected: Stack devops-kms reaches CREATE_COMPLETE and outputs the KeyId and KeyArn of your customer-managed key.

310. Confirm the key exists and is customer-managed (KeyManager = CUSTOMER), not an AWS-managed key.

```
aws kms describe-key --key-id alias/devops-app --query 'KeyMetadata.{Id:KeyId,Manager:KeyManager,State:KeyState}' --profile devops
```

Expected: Returns KeyManager 'CUSTOMER' and KeyState 'Enabled' for the alias/devops-app key.

311. Confirm automatic key rotation is enabled so the key material rotates yearly without changing the key ARN.

```
aws kms get-key-rotation-status --key-id alias/devops-app --profile devops
```

Expected: Returns KeyRotationEnabled true.

312. Store the application credential in Secrets Manager encrypted with your customer-managed key (value from a placeholder file, never inline).

```
aws secretsmanager create-secret --name devops/app/db --kms-key-id alias/devops-app --secret-string file://secret-value.json --profile devops
```

Expected: Returns the secret ARN; the secret is encrypted with alias/devops-app, not the default aws/secretsmanager key.

313. Deploy the rotation configuration (Lambda + schedule) so the secret rotates automatically every 30 days.

```
aws cloudformation deploy --template-file secret.yaml --stack-name devops-secret --capabilities CAPABILITY_NAMED_IAM --profile devops
```

Expected: Stack devops-secret reaches CREATE_COMPLETE; the secret now has RotationEnabled true with a 30-day schedule.

314. Verify rotation is configured on the secret.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws secretsmanager describe-secret --secret-id devops/app/db --
query
'{Rotation:RotationEnabled,Days:RotationRules.AutomaticallyAfterD
ays}' --profile devops
```

Expected: Returns Rotation true and Days 30.

315. Retrieve the secret at runtime the way the app would (decrypts transparently via KMS).

```
aws secretsmanager get-secret-value --secret-id devops/app/db --
query SecretString --output text --profile devops
```

Expected: Returns the JSON secret value — proving the app role can decrypt it through the KMS key policy.

316. Create the encrypted data bucket and apply the deny-unencrypted bucket policy.

```
aws s3api create-bucket --bucket devops-sample-app-data-<your-
account-id> --region ap-southeast-1 --create-bucket-configuration
LocationConstraint=ap-southeast-1 --profile devops
```

Expected: The bucket is created; you will now enforce SSE-KMS and Block Public Access on it.

317. Set default SSE-KMS encryption on the bucket using your customer-managed key.

```
aws s3api put-bucket-encryption --bucket devops-sample-app-data-
<your-account-id> --server-side-encryption-configuration
file://bucket-encryption.json --profile devops
```

Expected: The command succeeds; get-bucket-encryption shows SSEAlgorithm aws:kms with your KeyId.

318. Attach the bucket policy that DENIES any PutObject that is not encrypted with your KMS key.

```
aws s3api put-bucket-policy --bucket devops-sample-app-data-
<your-account-id> --policy file://deny-unencrypted.json --profile
devops
```

Expected: The command succeeds; the bucket now rejects any upload without the correct SSE-KMS header.

319. Demonstrate the guardrail works: attempt an UNENCRYPTED upload and confirm it is denied.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws s3api put-object --bucket devops-sample-app-data-<your-
account-id> --key plain.txt --body ./plain.txt --profile devops
```

Expected: The upload fails with AccessDenied (An error occurred ... explicit deny in a resource-based policy).

320. Now upload correctly WITH SSE-KMS and confirm it succeeds.

```
aws s3api put-object --bucket devops-sample-app-data-<your-
account-id> --key report.json --body ./report.json --server-side-
encryption aws:kms --ssekms-key-id alias/devops-app --profile
devops
```

Expected: The upload succeeds and returns the ETag plus ServerSideEncryption aws:kms and the SSEKMSKeyId.

321. Verify the stored object is encrypted with YOUR key, not an AWS-managed default.

```
aws s3api head-object --bucket devops-sample-app-data-<your-
account-id> --key report.json --query
'{SSE:ServerSideEncryption,Key:SSEKMSKeyId}' --profile devops
```

Expected: Returns ServerSideEncryption aws:kms and the SSEKMSKeyId of alias/devops-app.

322. Demonstrate raw encrypt/decrypt with the CLI to show the app can round-trip data through the key.

```
aws kms encrypt --key-id alias/devops-app --plaintext
fileb://report.json --query CiphertextBlob --output text --
profile devops | base64 --decode > report.enc && aws kms decrypt
--ciphertext-blob fileb://report.enc --query Plaintext --output
text --profile devops | base64 --decode | head -c 40
```

Expected: The encrypt returns ciphertext and the decrypt returns the original first bytes of report.json — a clean round-trip.

Validation

describe-key shows a CUSTOMER-managed, Enabled key with rotation on; the secret is stored under your key with rotation enabled (30 days); the S3 bucket defaults to SSE-KMS with your KeyId; an unencrypted PutObject is denied while an SSE-KMS PutObject succeeds; and head-object confirms the object is encrypted with alias/devops-app.

Test it

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

You can store and rotate a secret in Secrets Manager under a customer-managed KMS key, retrieve it via KMS decrypt, and prove the S3 bucket refuses unencrypted uploads while accepting SSE-KMS uploads encrypted with your own key — an end-to-end data-protection guardrail.

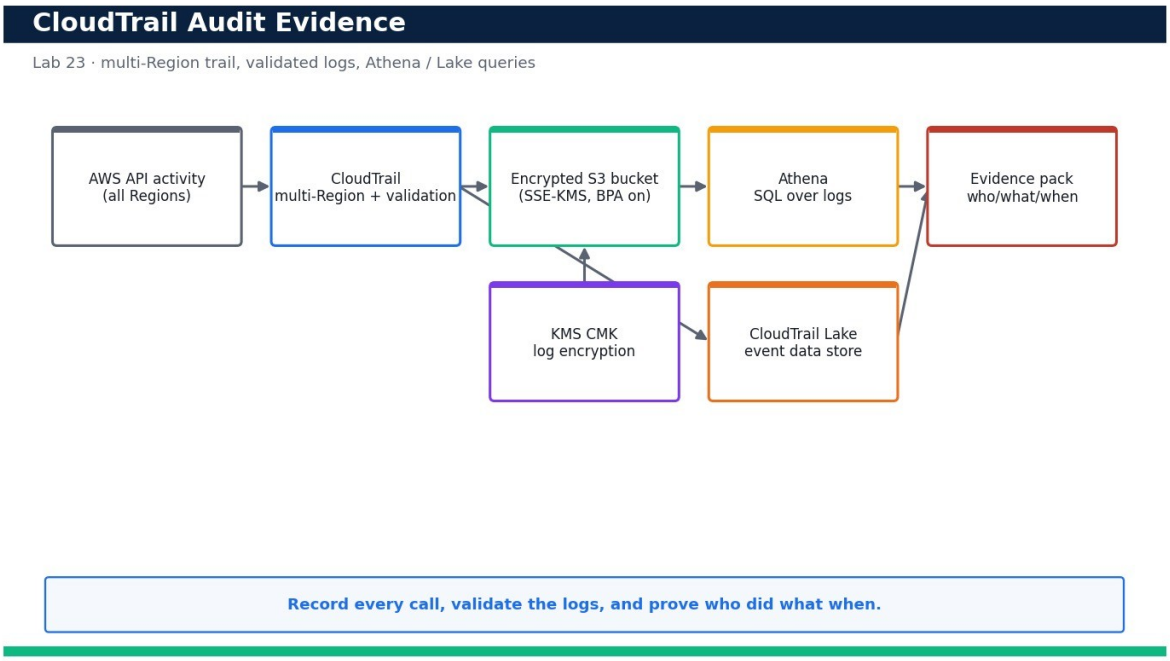
Note: Cost & safety: A customer-managed KMS key costs about US$1/month plus a tiny per-request charge; Secrets Manager is about US$0.40 per secret per month plus per-10k API calls; S3 storage at lab scale is cents. Schedule the KMS key for deletion (7-day window) and delete the secret with a short recovery window at the end to stop the monthly charges. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-22-secrets-kms-encryption/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 23 — Produce CloudTrail Audit Evidence and Answer Who-Did-What-When

Learning outcome: LO6. Skills mapped: K6, A4. Objective: Stand up a multi-Region CloudTrail with log-file validation and query it to export audit evidence.

Goal: Auditors and incident responders need a tamper-evident record of every API call. In this lab you create a multi-Region CloudTrail with log-file validation that delivers to an encrypted, access-blocked S3 bucket, then query the events two ways: with `cloudtrail lookup-events` for quick answers and with Athena over the S3 logs (plus a CloudTrail Lake event data store as the modern alternative) to answer precise 'who did what when' questions. You export the result as a signed evidence artifact.



© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Figure — Lab 23 reference architecture: Produce CloudTrail Audit Evidence and Answer Who-Did-What-When.

What you'll build

A validated, multi-Region CloudTrail delivering encrypted logs to S3, plus Athena/CloudTrail Lake queries that answer who-did-what-when and an exported evidence pack. (Services: AWS CloudTrail, Amazon S3, Amazon Athena, CloudTrail Lake, AWS KMS.)

Step-by-step

323. Deploy the trail stack: an encrypted, access-blocked S3 bucket, a KMS key for log encryption and a multi-Region trail with log-file validation.

```
aws cloudformation deploy --template-file cloudtrail.yaml --
stack-name devops-trail --capabilities CAPABILITY_NAMED_IAM --
profile devops
```

Expected: Stack devops-trail reaches CREATE_COMPLETE and outputs the TrailArn and the log bucket name.

324. Confirm the trail is multi-Region with log-file validation enabled.

```
aws cloudtrail describe-trails --trail-name-list devops-trail --
query 'trailList[0].
{Multi:IsMultiRegionTrail,Validation:LogFileValidationEnabled,Kms
:KmsKeyId}' --profile devops
```

Expected: Returns IsMultiRegionTrail true, LogFileValidationEnabled true and a KmsKeyId (SSE-KMS on the delivered logs).

325. Confirm the trail is actively logging.

```
aws cloudtrail get-trail-status --name devops-trail --query
'{Logging:IsLogging,LastDelivery:LatestDeliveryTime}' --profile
devops
```

Expected: Returns IsLogging true and a recent LatestDeliveryTime once the first log file is delivered.

326. Generate some auditable activity so there is something to find (a harmless describe call attributed to you).

```
aws ec2 describe-instances --profile devops >/dev/null && aws
s3api list-buckets --profile devops >/dev/null
```

Expected: The calls succeed; within a few minutes they will appear as CloudTrail events attributed to your identity.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

327. Quick answer with lookup-events: find recent console/CLI sign-ins or a specific event name.

```
aws cloudtrail lookup-events --lookup-attributes
AttributeKey=EventName,AttributeValue=RunInstances --max-results
5 --query 'Events[].
{Time:EventTime,User:Username,Event:EventName}' --profile devops
```

Expected: Returns a table of matching events with EventTime, the Username who called it and the EventName.

328. Validate log-file integrity to prove the logs have not been tampered with.

```
aws cloudtrail validate-logs --trail-arn <trail-arn> --start-time
2026-08-23T00:00:00Z --profile devops
```

Expected: Reports that all log files in the window pass digest validation (no gaps, no modified files).

329. Verify the delivered log objects are encrypted with your KMS key.

```
aws s3api head-object --bucket <trail-bucket> --key
AWSLogs/<your-account-id>/CloudTrail/ap-southeast-1/2026/08/23/<f
ile>.json.gz --query ServerSideEncryption --profile devops
```

Expected: Returns aws:kms — confirming the logs are encrypted at rest with the CMK.

330. Create the Athena table over the CloudTrail S3 logs (run the DDL from athena-query.sql).

```
aws athena start-query-execution --query-string file://athena-
create-table.sql --result-configuration
OutputLocation=s3://<athena-results-bucket>/ --profile devops
```

Expected: The query succeeds and creates the external table cloudtrail_logs pointing at the trail's S3 prefix.

331. Run the who-did-what-when Athena query to list actions by a given principal in a time window.

```
aws athena start-query-execution --query-string file://athena-
query.sql --result-configuration OutputLocation=s3://<athena-
results-bucket>/ --profile devops
```

Expected: Returns a QueryExecutionId; the results list eventtime, useridentity, eventname and sourceipaddress for the target principal.

332. Fetch the Athena results and confirm they answer the audit question.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws athena get-query-results --query-execution-id <qid> --query
'ResultSet.Rows[0:5]' --profile devops
```

Expected: Returns rows showing each API call the principal made, when, from which IP and against which resource.

333. Create a CloudTrail Lake event data store as the modern, SQL-native alternative to Athena.

```
aws cloudtrail create-event-data-store --name devops-lake --
retention-period 7 --multi-region-enabled --profile devops
```

Expected: Returns the EventDataStore ARN; management events begin flowing into the Lake store.

334. Query CloudTrail Lake directly with SQL to answer the same question without managing tables.

```
aws cloudtrail start-query --query-statement "SELECT eventTime,
userIdentity.arn, eventName FROM <eds-id> WHERE eventName =
'RunInstances' ORDER BY eventTime DESC LIMIT 20" --profile devops
```

Expected: Returns a QueryId; get-query-results returns the matching events directly from the Lake store.

335. Run the lookup-events.sh helper to bundle a targeted lookup + Athena question into one evidence run.

```
bash lookup-events.sh
```

Expected: Prints the matched events and writes evidence-<date>.json with the who/what/when rows for the pack.

336. Export the evidence: copy the query results and the validate-logs report into a dated evidence file for the audit pack.

```
aws s3 cp s3://<athena-results-bucket>/<qid>.csv ./evidence/who-
did-what-when.csv --profile devops
```

Expected: The CSV is saved locally under evidence/ and, together with the integrity report, forms the exportable audit evidence.

Validation

describe-trails shows a multi-Region trail with log-file validation and KMS encryption; get-trail-status shows IsLogging true with recent delivery; validate-logs reports no tampering; head-object shows aws:kms on the log objects; and both an Athena query and a CloudTrail Lake query return the who/what/when rows for a chosen principal.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Test it

You can answer 'who did what when' from CloudTrail two independent ways (Athena over S3 and CloudTrail Lake SQL), prove the logs are integrity-validated and encrypted, and export a dated evidence artifact suitable for an audit.

Note: Cost & safety: CloudTrail's first copy of management events is free; extra trails and data events bill per event delivered. S3 storage and Athena scanning are cents at lab scale (Athena bills per TB scanned — partition and LIMIT your queries). CloudTrail Lake bills per GB ingested/scanned, so delete the event data store at the end. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-23-cloudtrail-audit-evidence/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 24 — Run Continuous Compliance with AWS Config and Security Hub

Learning outcome: LO6. Skills mapped: K6, A4. Objective: Enable AWS Config managed rules and Security Hub standards, inject a violation and remediate to COMPLIANT.

Goal: Compliance is not a one-off audit — it must run continuously. In this lab you turn on the AWS Config recorder with managed rules (s3-bucket-public-read-prohibited, encrypted-volumes and more) delivered as a conformance pack, enable AWS Security Hub with the AWS Foundational Security Best Practices standard, then inject a real violation and watch it flip to NON_COMPLIANT. You remediate it back to COMPLIANT (by hand and via an auto-remediation) and read the Security Hub posture score before and after.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

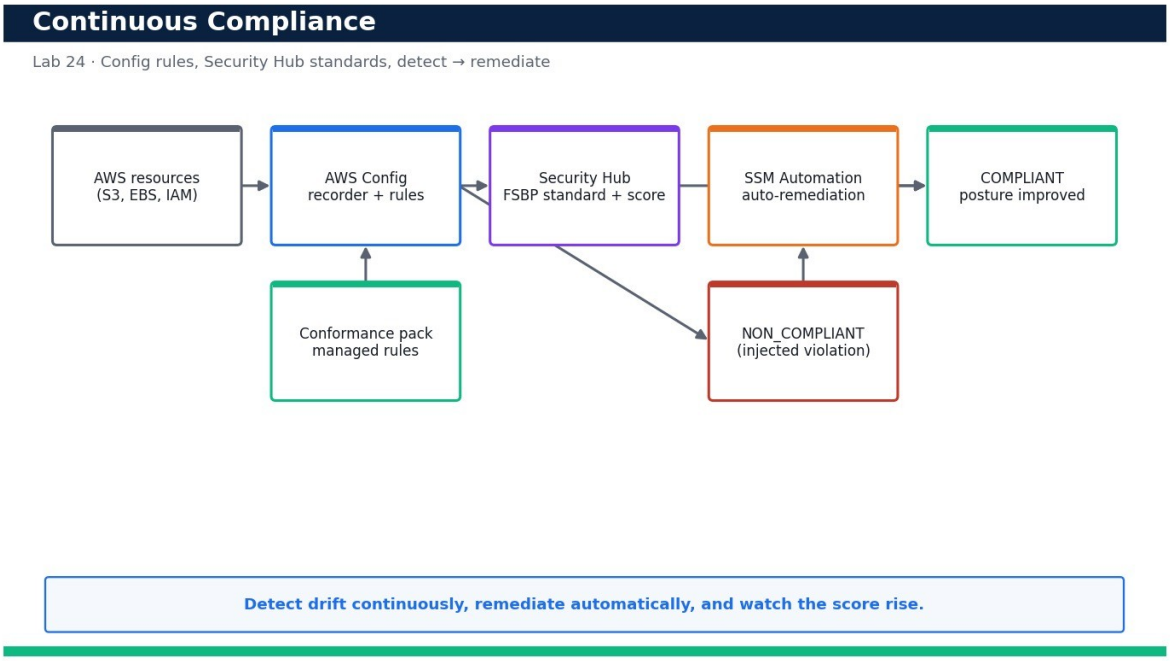


Figure — Lab 24 reference architecture: Run Continuous Compliance with AWS Config and Security Hub.

What you'll build

A continuously evaluating AWS Config + Security Hub setup that detects an injected violation as NON_COMPLIANT, auto-remediates it to COMPLIANT, and shows the posture score improve. (Services: AWS Config, AWS Security Hub, AWS Systems Manager Automation, Amazon S3, AWS CloudFormation, conformance packs.)

Step-by-step

337. Create the Config delivery bucket and turn on the configuration recorder + delivery channel so resource history is captured.

```
aws cloudformation deploy --template-file config-setup.yaml --
stack-name devops-config-base --capabilities CAPABILITY_NAMED_IAM
--profile devops
```

Expected: Stack devops-config-base reaches CREATE_COMPLETE with an encrypted delivery bucket and the recorder role.

338. Start the configuration recorder so evaluations can begin.

```
aws configservice start-configuration-recorder --configuration-
recorder-name default --profile devops
```

Expected: The recorder starts; describe-configuration-recorder-status shows recording true.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

339. Deploy the AWS Config managed rules directly (s3-bucket-public-read-prohibited, encrypted-volumes, iam-user-no-policies-check).

```
aws cloudformation deploy --template-file config-rules.yaml --
stack-name devops-config --capabilities CAPABILITY_IAM --profile
devops
```

Expected: Stack devops-config reaches CREATE_COMPLETE and the three Config rules are created.

340. Deploy a conformance pack so a whole set of best-practice rules is applied and tracked as a unit.

```
aws configservice put-conformance-pack --conformance-pack-name
devops-ops-best-practices --template-body file://conformance-
pack.yaml --delivery-s3-bucket <config-bucket> --profile devops
```

Expected: Returns the conformance pack ARN; describe-conformance-pack-status shows CREATE_COMPLETE.

341. Enable AWS Security Hub in the Region (this auto-enables the CIS/FSBP checks it manages).

```
aws securityhub enable-security-hub --enable-default-standards --
profile devops
```

Expected: Security Hub is enabled; a HubArn is returned and default standards begin subscribing.

342. Confirm the AWS Foundational Security Best Practices standard is enabled.

```
aws securityhub get-enabled-standards --query
'StandardsSubscriptions[].StandardsArn' --profile devops
```

Expected: The output lists the aws-foundational-security-best-practices standard subscription ARN.

343. Read the BASELINE compliance posture before you inject a violation.

```
aws securityhub get-findings --filters '{"ComplianceStatus":
[{"Value":"FAILED","Comparison":"EQUALS"}]}' --query
'length(Findings)' --profile devops
```

Expected: Returns a baseline count of failed checks — your starting posture number.

344. Inject a deliberate violation: make a disposable test bucket publicly readable to trip s3-bucket-public-read-prohibited.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws s3api put-public-access-block --bucket <test-bucket> --
public-access-block-configuration
BlockPublicAcls=false,IgnorePublicAcls=false,BlockPublicPolicy=fa
lse,RestrictPublicBuckets=false --profile devops
```

Expected: The bucket's Block Public Access is (deliberately) disabled — a compliance violation to be detected.

345. Force a re-evaluation and confirm AWS Config marks the bucket NON_COMPLIANT.

```
aws configservice start-config-rules-evaluation --config-rule-
names s3-bucket-public-read-prohibited --profile devops && aws
configservice get-compliance-details-by-config-rule --config-
rule-name s3-bucket-public-read-prohibited --query
'EvaluationResults[?ComplianceType==`NON_COMPLIANT`]' --profile
devops
```

Expected: The test bucket appears with ComplianceType NON_COMPLIANT.

346. Confirm the same violation surfaces in Security Hub as a FAILED finding.

```
aws securityhub get-findings --filters '{"ResourceId":
[{"Value":"<test-bucket>","Comparison":"PREFIX"}]}' --query
'Findings[].{Title:Title,Status:Compliance.Status}' --profile
devops
```

Expected: A finding with Compliance.Status FAILED for the public-access control appears for the test bucket.

347. Attach a Config auto-remediation (SSM Automation) so the rule self-heals public buckets going forward.

```
aws configservice put-remediation-configurations --remediation-
configurations file://remediation.json --profile devops
```

Expected: The remediation configuration is registered against s3-bucket-public-read-prohibited using AWS-DisableS3BucketPublicReadWrite (or re-enable BPA).

348. Remediate now: re-enable Block Public Access on the test bucket (or let the auto-remediation run).

```
aws s3api put-public-access-block --bucket <test-bucket> --
public-access-block-configuration
BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true
,RestrictPublicBuckets=true --profile devops
```

Expected: Block Public Access is restored on the test bucket.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

349. Re-evaluate and confirm the resource returns to COMPLIANT.

```
aws configservice start-config-rules-evaluation --config-rule-names s3-bucket-public-read-prohibited --profile devops && aws configservice get-compliance-details-by-config-rule --config-rule-name s3-bucket-public-read-prohibited --query 'EvaluationResults[?ComplianceType==`COMPLIANT`]' --profile devops
```

Expected: On re-evaluation the test bucket is reported COMPLIANT again.

350. Read the Security Hub posture AFTER remediation and compare against the baseline.

```
aws securityhub get-findings --filters '{"ComplianceStatus":[{"Value":"FAILED","Comparison":"EQUALS"}]}' --query 'length(Findings)' --profile devops
```

Expected: The failed-check count has dropped (the public-bucket finding is now RESOLVED/PASSED) — the posture score improved.

351. Run remediate-violation.sh to bundle inject → detect → remediate → confirm into one repeatable demonstration.

```
bash remediate-violation.sh <test-bucket>
```

Expected: Prints 'INJECTED', 'NON_COMPLIANT confirmed', 'REMEDIATED' and 'COMPLIANT confirmed' — a full closed-loop compliance run.

Validation

The Config recorder is recording, the three managed rules and the conformance pack are active, and Security Hub's FSBP standard is enabled; the injected public bucket is reported NON_COMPLIANT in both Config and Security Hub, then COMPLIANT after remediation; and the Security Hub failed-check count (posture score) improves between the baseline and the post-remediation read.

Test it

You can run a full continuous-compliance loop: inject a real violation, watch AWS Config and Security Hub flag it NON_COMPLIANT/FAILED, remediate it (manually and via auto-remediation), confirm it returns to COMPLIANT and show the posture score improving.

Note: Cost & safety: AWS Config bills per configuration item recorded and per rule evaluation; Security Hub bills per security check and per finding ingested. At lab scale this is a few US$ if left running, so stop the configuration recorder, disable the Security

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

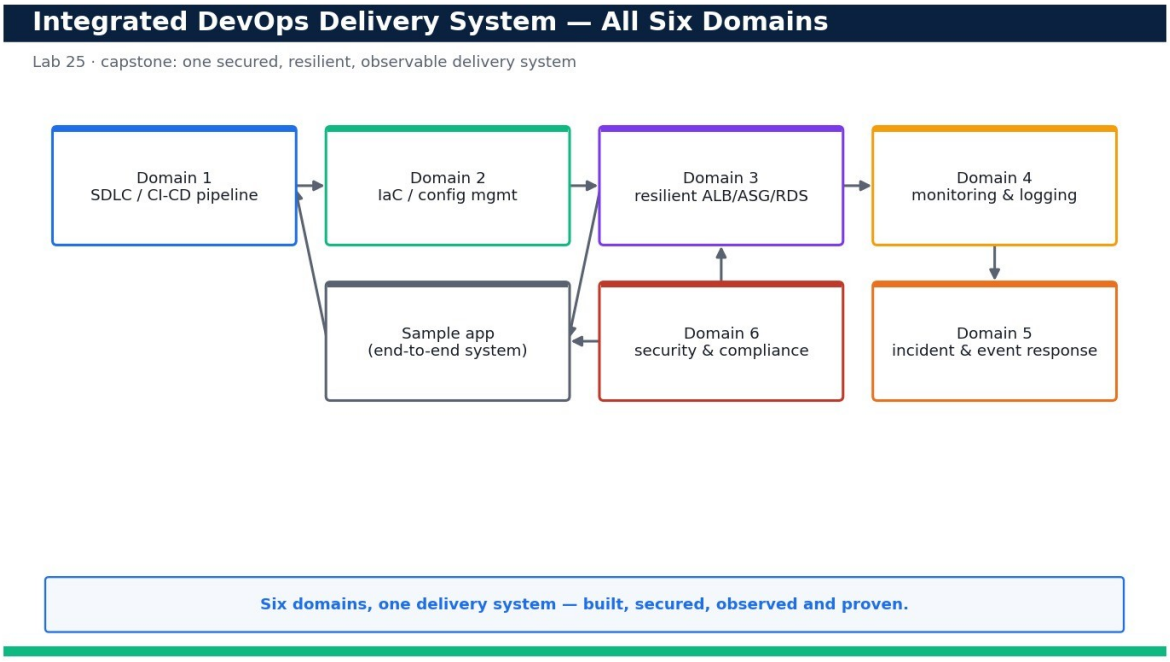
Hub standard and delete the conformance pack promptly at the end. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-24-config-securityhub-compliance/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

Lab 25 — Capstone — Blueprint, Well-Architected Review and Evidence Pack for the Whole Delivery System

Learning outcome: LO1–LO6. Skills mapped: K1, K2, K3, K4, K5, K6, K7, A1, A2, A3, A4, A5, A6. Objective: Synthesise Labs 1–24 into one secured, resilient, observable delivery system with a blueprint, an operational review and an evidence pack.

Goal: This capstone ties the entire course together. You produce a single technical blueprint that connects all six domains — SDLC automation (Domain 1), configuration management & IaC (Domain 2), resilient architecture (Domain 3), monitoring & logging (Domain 4), incident & event response (Domain 5) and security & compliance (Domain 6) — into one end-to-end delivery system for the sample app. You then run a Well-Architected-style operational review across all six pillars, assemble an evidence pack that maps every learning outcome to a concrete artifact from the prior 24 labs, and present the whole solution. A top-level CloudFormation template documents/nests the stacks built across the course.



© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Figure — Lab 25 reference architecture: Capstone — Blueprint, Well-Architected Review and Evidence Pack for the Whole Delivery System.

What you'll build

A completed technical blueprint, a Well-Architected pillar review with scores and actions, and an evidence pack that proves the whole secured/resilient/observable delivery system against LO1–LO6. (Services: AWS CodePipeline/CodeBuild/CodeDeploy, CloudFormation, EC2/ALB/Auto Scaling, RDS, CloudWatch, EventBridge, SNS, Lambda, IAM, KMS, Secrets Manager, CloudTrail, AWS Config, Security Hub.)

Step-by-step

352. Open blueprint.md and fill Section 1 (Domain 1 — SDLC automation) referencing your Lab 1 CI/CD pipeline: source, build, test, approval and rollback.

```
cat blueprint.md
```

Expected: The blueprint template shows one section per domain; Section 1 now describes your CodePipeline flow and links to the Lab 1 evidence.

353. Fill Section 2 (Domain 2 — IaC & configuration management) referencing your CloudFormation stacks and parameterised, reproducible environments.

```
aws cloudformation list-stacks --stack-status-filter
CREATE_COMPLETE UPDATE_COMPLETE --query
'StackSummaries[].StackName' --profile devops
```

Expected: The list shows your course stacks (devops-net, devops-app, devops-ha, devops-mon, devops-trail, devops-config...) which you cite as IaC evidence.

354. Fill Section 3 (Domain 3 — resilience) referencing your multi-AZ ALB + Auto Scaling + RDS design and stated RTO/RPO.

```
aws elbv2 describe-load-balancers --query 'LoadBalancers[].
{Name:LoadBalancerName,AZs:AvailabilityZones[].ZoneName}' --
profile devops
```

Expected: The ALB spans at least two AZs; you record the AZ list and your RTO/RPO in the blueprint.

355. Fill Section 4 (Domain 4 — observability) referencing your CloudWatch dashboard, metric filters, alarms and Logs Insights queries.

```
aws cloudwatch list-dashboards --query
'DashboardEntries[].DashboardName' --profile devops
```

Expected: Your monitoring dashboard is listed; you attach its screenshot and alarm ARNs as observability evidence.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

356. Fill Section 5 (Domain 5 — incident & event response) referencing your EventBridge rule, SNS topic and remediation Lambda.

```
aws events list-rules --query 'Rules[].Name' --profile devops
```

Expected: Your incident-response rule is listed; you document the detect → notify → auto-remediate flow in the blueprint.

357. Fill Section 6 (Domain 6 — security & compliance) referencing Labs 21–24: least-privilege IAM, KMS/Secrets, CloudTrail and Config/Security Hub.

```
aws securityhub get-findings --filters '{"ComplianceStatus":
[{"Value":"FAILED","Comparison":"EQUALS"}]}' --query
'length(Findings)' --profile devops
```

Expected: You record the current posture score and cite the least-privilege, encryption, audit and compliance controls.

358. Validate the top-level capstone-architecture.yaml that documents/nests the course stacks (validate-only, no deploy needed).

```
aws cloudformation validate-template --template-body
file://capstone-architecture.yaml --profile devops
```

Expected: The template validates; its Parameters/Resources reference the nested stack URLs for Domains 1–6 as one system.

359. Optionally deploy the capstone stack (or a change-set preview) to prove the pieces compose into one system.

```
aws cloudformation deploy --template-file capstone-
architecture.yaml --stack-name devops-capstone --capabilities
CAPABILITY_NAMED_IAM --no-execute-changeset --profile devops
```

Expected: A change-set is created showing the nested stacks (net, app, ha, mon, ir, security) that make up the whole system.

360. Open operational-review.md and score each Well-Architected pillar (Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability).

```
cat operational-review.md
```

Expected: Each pillar has a checklist; you mark each item Met/Partly/Gap and record a per-pillar score and top risk.

361. Run a lightweight Well-Architected self-review question set and capture the highest-priority improvement per pillar.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
aws wellarchitected list-lens-review-improvements --workload-id
<workload-id> --lens-alias wellarchitected --query
'ImprovementSummaries[].{Pillar:PillarId,Risk:RiskCounts}' --
profile devops
```

Expected: For each pillar you record the top improvement and its risk (HIGH/MEDIUM) in operational-review.md. (If you did not create a Workbench workload, complete the checklist manually.)

362. Open evidence-pack-checklist.md and map every learning outcome (LO1–LO6) to a specific artifact from the prior labs.

```
cat evidence-pack-checklist.md
```

Expected: Each LO row lists the lab, the artifact (screenshot/CLI output/template) and where it is stored — no LO is left without evidence.

363. Assemble the evidence folder: gather the referenced screenshots, CLI outputs and templates into evidence/ organised by domain.

```
ls -R evidence/ | head -40
```

Expected: evidence/ contains one subfolder per domain (01-sdlc ... 06-security) each holding the artifacts cited in the checklist.

364. Validate completeness: confirm every LO in the checklist has at least one non-empty evidence artifact.

```
bash validate-evidence.sh
```

Expected: Prints 'LO1 OK ... LO6 OK' and 'EVIDENCE PACK COMPLETE' — every outcome has a mapped, present artifact.

365. Cross-check security posture one final time so the pack reflects a clean, compliant end state.

```
aws configservice describe-compliance-by-config-rule --query
'ComplianceByConfigRules[?
Compliance.ComplianceType==`NON_COMPLIANT`].ConfigRuleName' --
profile devops
```

Expected: The list of NON_COMPLIANT rules is empty (or only the deliberately-documented exceptions), matching the blueprint's stated posture.

366. Present the solution: walk the blueprint (system view), the Well-Architected review (risks & actions) and the evidence pack (proof) as one 10-minute story.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected: You can narrate how a commit flows through CI/CD, deploys to a resilient multi-AZ system, is observed and auto-remediated, and is secured/audited/compliant — backed at each step by an evidence artifact.

367. Package and hand over: zip the completed blueprint, operational review, checklist and evidence folder as the capstone deliverable.

```
zip -r capstone-deliverable.zip blueprint.md operational-review.md evidence-pack-checklist.md capstone-architecture.yaml evidence/
```

Expected: A single capstone-deliverable.zip is produced containing the blueprint, review, checklist, top-level template and all evidence.

Validation

The blueprint has all six domain sections completed and cross-referenced to the relevant labs; capstone-architecture.yaml validates and its change-set shows the nested Domain 1–6 stacks; operational-review.md has a score and top risk for every Well-Architected pillar; and evidence-pack-checklist.md maps each of LO1–LO6 to a present, non-empty artifact (validate-evidence.sh prints EVIDENCE PACK COMPLETE) with a clean final compliance posture.

Test it

You can present one coherent delivery system — a commit flowing through CI/CD to a resilient, observed, auto-remediating, secured and continuously-compliant environment — and, for every learning outcome LO1–LO6, point to a specific evidence artifact drawn from the prior 24 labs, with the whole solution scored against the Well-Architected pillars.

Note: Cost & safety: The capstone is mostly documentation and review, so it is nearly free if you reuse (or read from) the stacks from prior labs. If you deploy the top-level capstone-architecture.yaml to validate nesting, tear it down the same day; otherwise run it in --dry-run / validate-only mode to avoid any spend. Always run the clean-up step; never commit real credentials.

Note: Full source files, expected outputs, evidence requirements, troubleshooting and acceptance criteria are in labs/lab-25-capstone-blueprint-evidence-pack/README.md (mirrored in INSTRUCTIONS.pdf). Use only your own AWS account and placeholder credentials.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Exam & Practice Focus — Cross-Domain Topics

These cross-cutting topics appear throughout the course and the assessment but are not each a dedicated lab. Study them alongside the labs so you can answer the knowledge (SAQ) questions confidently.

Deployment strategies

- In-place — update the same instances; simplest, but downtime and slower rollback.
- Rolling — update in batches to limit impact; supported by CodeDeploy and ECS.
- Blue/green — deploy a new fleet and shift traffic; instant rollback by switching back.
- Canary — shift a small percentage first, watch metrics, then complete or roll back.

Resilience and disaster recovery

- Design across multiple Availability Zones by default; use ELB + Auto Scaling for self-healing.
- Choose a DR pattern by RTO/RPO: backup & restore (hours) → pilot light → warm standby → multi-site active/active (near-zero).
- Protect data with Multi-AZ RDS, DynamoDB global tables, S3 cross-Region replication and automated snapshots.

Observability

- The three pillars are metrics, logs and traces — CloudWatch Metrics/Logs and AWS X-Ray.
- Turn logs into signals with metric filters, then alarm on thresholds or anomaly detection.
- Correlate on a dashboard; use Logs Insights, Container Insights and Lambda Insights to find root cause.

Security and compliance

- Enforce least privilege with IAM roles, permission boundaries and SCPs; avoid long-lived keys.
- Encrypt everywhere with KMS (at rest) and TLS (in transit); manage secrets in Secrets Manager.
- Use CloudTrail (audit), AWS Config (compliance), GuardDuty (threat detection) and Security Hub (posture).

Assessment Preparation

- First pass: complete every lab end to end on your own AWS account, reading each lab README.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Second pass: rebuild the labs from memory until the AWS CLI/console workflow is automatic.
- Review the 'Test it' check and the key concepts for any domain you find hard.
- Be ready for the Written Assessment (SAQ, 1 hour, open book) — short-answer knowledge questions across the six domains.
- Be ready for the Practical Performance (PP, 2 hours, open book) — hands-on AWS DevOps tasks drawn from the labs.
- Remember the 75% attendance requirement and take the digital attendance (AM, PM and assessment).

Glossary

- **CI/CD —** Continuous Integration / Continuous Delivery (or Deployment) — automating build, test and release.
- **CodePipeline —** AWS service that orchestrates the stages of a release pipeline.
- **buildspec.yml —** The build specification CodeBuild uses to install, build, test and output artifacts.
- **appspec.yml —** The application specification CodeDeploy uses to deploy and run lifecycle hooks.
- **Infrastructure as Code (IaC) —** Provisioning infrastructure from version-controlled templates (e.g. CloudFormation).
- **CloudFormation stack —** A collection of AWS resources created and managed together from a template.
- **Change set —** A preview of the changes CloudFormation will make before you execute them.
- **Drift detection —** Checking whether deployed resources still match the template.
- **Auto Scaling group (ASG) —** A group of EC2 instances that scales and self-heals to a desired capacity.
- **Availability Zone (AZ) —** An isolated location within an AWS Region; multi-AZ designs survive one AZ failure.
- **RTO / RPO —** Recovery Time / Point Objective — how fast you recover and how much data you can lose.
- **CloudWatch —** AWS monitoring service for metrics, logs, dashboards and alarms.
- **EventBridge —** Serverless event bus that routes events to targets such as Lambda and SNS.
- **Auto-remediation —** Automatically fixing an issue in response to an event, e.g. via Lambda or SSM Automation.
- **IAM role —** A set of temporary, assumable permissions — preferred over long-lived access keys.
- **AWS Config —** Service that records resource configuration and evaluates it against compliance rules.
- **CloudTrail —** Service that records AWS API activity for audit and forensics.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **KMS —** AWS Key Management Service — managed encryption keys for data at rest.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg